
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 16:01:57
Файлов исходного кода: 111
Суммарный объём кода: 1.97 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot: ')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends imagemagick
          # На Ubuntu политика ImageMagick иногда запрещает конвертировать текст в картинку
          # разрешаем то, что нужно пайплайну (text -> png)
          for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
              sed -i 's/<policy domain="text" rights="none"/<policy domain="text" rights="write"/g' $policy
            fi
          done
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискретке
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, height=device-height" />
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
     <title> Дискретка для тупых – Интерактивный гид по дискретке
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

 return (
 <div className="min-h-screen bg-slate-950 text-slate

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
````tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeT
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```

```

+         <button
+             id="tab-compiler-btn"
+             onClick={() => setActiveTab('compiler')}
+             className={`flex-1 lg:flex-none flex items
duration-200 whitespace-nowrap ${
+                 activeTab === 'compiler'
+                 ? 'bg-emerald-600 text-white shadow-lg
+                 : 'text-slate-400 hover:text-white'
+             }`}
+         >
+         <Terminal className="w-4 h-4" /> Python
+     </button>
+     <a
+         id="download-pdf-btn"
+         href="/export/full_code_all_pages.pdf"
+         href={`${import.meta.env.BASE_URL}export/full
+         download="full_code_all_pages.pdf"
+         target="_blank"
+         rel="noreferrer"
@@ -73,7 +84,7 @@ export const Navbar: React.FC<NavbarP
    </a>
    <a
        id="download-txt-btn"
        href="/export/full_code_all_pages.txt"
        href={`${import.meta.env.BASE_URL}export/full
        download="full_code_all_pages.txt"
        target="_blank"
        rel="noreferrer"
    ....

```

`src/components/PythonCompiler.tsx`

Полное содержимое после изменений

```

````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Pla

```

```

export function PythonCompiler() {
 const [code, setCode] = useState(STARTER_CODE);
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

 const runCode = useCallback(async () => {
 if (running) return;

 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется");

 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не выполнила");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
 } finally {
 setRunning(false);
 }
 }, [code, stdin]);
}

```

```

 <span className="text-[11px] text-slate
 </div>
 <div className="flex items-center gap-2">
 <button
 type="button"
 onClick={reset}
 className="inline-flex items-center gap-2 text-slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
 </button>
 </div>
</div>

<textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.key === "Enter") {
 event.preventDefault();
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
 placeholder="Напишите Python-код..."

```

```
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-indigo-600">
 Ctrl или Cmd + Enter запускает код. Введите код в поле ввода.

 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-amber-600">
 Первый запуск скачивает примерно несколько МБ данных.

 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-emerald-600">
 Код выполняется в браузере и не отправляется на сервер.

 </div>
 </div>
 </div>
 </main>
);
}
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и библиотеки
+name = "алгоритмы"
+
+def main():
+    print("Привет, мир!")
+    return "Результат работы программы"`;
+
+const [code, setCode] = useState(STARTER_CODE);
+const [output, setOutput] = useState("");
+const [loading, setLoading] = useState(false);
+const [error, setError] = useState("");
+const [isReady, setIsReady] = useState(false);
+
+const handleRun = async () => {
+  setLoading(true);
+  setError("");
+
+  try {
+    const pyodide = await loadPyodide({
+      indexURL: PYODIDE_INDEX_URL,
+    });
+
+    const jsCode = `
+      ${code}
+      pyodide.run_code(main)
+    `;
+
+    const result = pyodide.runCode(jsCode);
+    setOutput(result);
+    setIsReady(true);
+  } catch (err) {
+    setError(err.message);
+  }
+
+  setLoading(false);
+};
```



```

+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput("");
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout
+     runtime.setStderr({ batched: (message) => stderr
+     runtime.setStdin({ stdin: () => (inputLines.length
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выв
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("")
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```

+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...

```

`vite.config.ts`

Полное содержимое после изменений

```

````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
},

```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты **\*\*строго\*\*** придерживаешься готовых шаблонов разметки, описанных в файле ``.ai_skills/universal_guide_authoring/SKILL.md``.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-шаблоны, описанные в файле ``.ai_skills/universal_guide_authoring/SKILL.md``.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть описана в файле ``.ai_skills/universal_guide_authoring/SKILL.md`` для вставки в ``.src/data/content.json`` или ``.src/data/content.ts``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык и формальный язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 },
}
```

Универсальное пособие • полный исходный код

```

npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
````
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
````
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каж**
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full\_code\_all\_pages`
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья сегментов**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока не реализована

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни кода
8. Графы. Компоненты связности	`graph-components`	Нет кода
24. Классы сложности, сведение задач	`complexity-classes`	Нет кода

## Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 },
 });
}
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
```

```

 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "co
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []));
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.inn
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefin
const next = index < chapters.length - 1 ? chapters[in

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
 {/* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}` }
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 {/* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-1
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать содер
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className="
 <span className="text-[10px] font-bold [wri
 </button>

 <main id="main-content" className="flex-1 min
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсальн
 </div>
 <h2 className="text-base sm:text-xl font
 {activeChapter.title}
 </h2>
 </div>
 </div>

```

```
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
))
 </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile`
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5));
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9));
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}
```

```
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```



```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```

```
main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины
">i. Любой отрезок покрывается двумя
перация идемпотентна: min,
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, покрывающий
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700">
 <p class="text-[10px] uppercase tracking-wider">
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{kx} \times 2^{ky}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950">
 <div>
 $n \ O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 ерению) → 2D: построение (уровни <span
 ос (прямоугольник раскладывается на чет
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <p class="text-slate-300 text-sm mb-4">Эти
 за
 они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 цы, у разреженной таблицы – размер, умножен
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; border-radius: 5px; background-color: #f0f0f0; text-align: center; vertical-align: middle;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; border-radius: 5px; background-color: #f0f0f0; text-align: center; vertical-align: middle;">ум на его части не вычислишь. Приходится хранить матрицу.  
<span class="font-mono text-emerald-300">log N × log M</span>. На ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: сложность алгоритма – перекрывающиеся блоки посчитают сложность.  
</b> – у минимума нет обратной операции. Строим таблицу

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500/50">

<h3 class="text-xl font-bold text-amber-400">Задача 5. Разреженная таблица

<div class="bg-rose-950/30 p-5 rounded-lg border-rose-500/50">

<ul class="list-disc pl-5 space-y-2 text-slate-300">

<li><b>Сумма через разреженную таблицу. (билет 5).</b></li>

<li><b>Память.</b> N = M = 1000 даём массив, считайте ячейки, потом пишите код.</li>

<li><b>Один log вместо двух.</b> Вспомогательная таблица.  
<span class="font-mono text-emerald-300">str[i][j] = str[i][j-1] + str[i-1][j]</span></li>

<li><b>Выход за границу в запросе.</b> Вспомогательная таблица.  
<span class="font-mono text-emerald-300">str[i][j] = str[i][j-1] + str[i-1][j]</span></li>

<li><b>Нужны обновления.</b> Разреженная таблица. (билет 1), а не она.</li>

</ul>

```

 else
 st[r][c][kx][ky] = min(

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму на нескольких сотен на сторону), а запросов очень много.

Вторая половина этого сравнения:

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
х». Спуск от корня: узел, который вместе с
скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого $O(h)$ сравнений
 </div>
 </div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «У6
```



```

<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
 Код: вставка, merge, erase (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
def insert(root, key, pri):
 if root is None:
 return Node(key, pri)
 if key <= root.key: # равные x — левый
 root.left = insert(root.left, key, pri)
 else:
 root.right = insert(root.right, key, pri)
 return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
 root = insert(root, key, pri)</pre>
 <pre class="bg-slate-950 p-4 rounded">
def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
 if key < root.key: root.left = erase(root.left, key)
 elif key > root.key: root.right = erase(root.right, key)
 else: root = merge(root.left, root.right)
 return root</pre>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 }

```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево β (между x и p) – единств
x, и под p.</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

 / \ \
 x y
 / \ \
 α β
 / \ \
 x y
 / \ \
 α β

 x выше
 -----> α
 / \ \
 p r
 / \ \
 β γ

 p выше
 <-----
 / \ \
 p r
 / \ \
 β γ
 </pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x): 3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x): 3 × Zig-Zig + Zig → высота 8
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<
```

```
 <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mon
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + s
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None</pre>
 <p class="text-xs text-slate-400">В
 Одна строка – и вся амортизация.</p>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сумма подматрицы за O(1): 1D и 2D, дв
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 <td class="py-2 pr-3 font-mono">
 </td>
 </tr>
 </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
 ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-500
 </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Тонкая: одна матрица
 </div>
 <p class="text-slate-300 text-sm mb-2">
 одной матрицы (плюс рамка из нулей). Никаких вычитаний, только сложение. Прямоугольника от угла. Всё остальное получаем из
 </div>
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычислить с помощью
 </div>
 <p class="text-slate-300 text-sm mb-2">
 максимума или gcd вычитания не существует:
 е запросы – это
 платит она за это памятью
 </div>
 </div>

<div class="bg-slate-800/70 p-5 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 </tr>
 <tr class="border-b border-slate-700">

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 1D (тот же, что выполняет пане
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">П
 - часть демонстрации: лента подсвечивает, из
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 2D – построение и запрос прямо
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
 <p class="text-xl font-mono text-white">
 1. Если граф планарный, то он можно вложить в плоскость так, чтобы никакие две
 его ребра не пересекались.
 2. Если граф можно вложить в плоскость, то он планарный.
 3. Если граф можно раскрасить 4 цветами, то он планарный.
 4. Если граф планарный, то он можно раскрасить 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Если карта мира раскрасить 4 цветами, то
 границы не пересекаются в одной точке по-
 тому, что границы не сливались цветами, Всегда можно всего 4
 цвета раскрасить карту мира.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",

```



```

 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая.",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">В strongly connected component (SCC) графа — это максимальный подграф, в котором каждая вершина достижима из каждой другой вершины. Другими словами, в SCC для любых двух вершин u и v существует путь от u к v и от v к u.
 <p class="text-xl font-mono text-white">Алгоритм Косарая позволяет найти все SCC в орграфе, инвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где улицы имеют одностороннее движение. Если вы можете добраться из точки A в точку B по односторонним улицам, но не наоборот, то A и B не принадлежат к одному SCC. SCC — это такие группы точек, между которыми можно двигаться в обе стороны по односторонним улицам. Косарай находит все такие группы на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 </div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Пример: сортировка чисел</p>
 <p class="text-base sm:text-lg font-mono">
 P: класс decision-задач, решаемый за полиномиальное время.
 NP: класс decision-задач с сертификатом.
 </p>
 <p>).

 Сведение (Reduction) A ≤_p B: задача A сводится к задаче B.
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судоку (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример сортировки чисел. Класс NP – это класс задач, для которых существует быструю сетку (ответ), он БЫСТРО (за класс P) решается.
 </div>
 </div>
 </div>
 </div>
`
 }
]

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic"
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded"
 <p class="text-sm text-blue-300"
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg bor"
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>

```

```

 </div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл",
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков <str
 Единственная ниточка. Порвётся – граф р
 </p>
 </div>
</div>

```



```

<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем
 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад
 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный в
 low[v] = min(low[v], low[to]);
 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
}
```

```

 <p><span class="text-emerald-400 font-
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 и K3,3 – д
 Если внутри графа спрятано подра
 – плоской раскладки не будет.
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2" style="background-color: #f0f0f0; padding: 10px; border-radius: 10px; border: 1px solid #ccc; box-shadow: 5px 5px 0px #ccc;">
 <div class="bg-slate-800 p-6 rounded-lg" style="background-color: #2d3748; color: white; padding: 10px; border-radius: 10px; border: 1px solid #ccc; box-shadow: 5px 5px 0px #ccc;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 2px solid #2e7d32; box-shadow: 5px 5px 0px #2e7d32;">
 Аналогия 1: Расписание экзаменов
```

```
 </div>
 <p class="text-slate-300 text-sm">Вот таблица, где каждая строка — это предмет, а каждый столбец — это слот времени. Раскрасить граф = составить расписание. Задача — «берём предметы в каком-то порядке и ставим их в слоты».
```

```
 <div class="bg-slate-900 p-6 rounded-lg" style="background-color: #1a202c; color: white; padding: 10px; border-radius: 10px; border: 1px solid #ccc; box-shadow: 5px 5px 0px #ccc;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-fuchsia-500 shadow-md" style="border: 2px solid #e91e63; box-shadow: 5px 5px 0px #e91e63;">
 Аналогия 2: Частоты вышек
```

```
 </div>
 <p class="text-slate-300 text-sm">См. задачу о раскраске графа. Цвет — это частота вышки. Интерференция = ребро. Диапазон частот — это диапазон частот вышек. Задача — «найти частоты для каждой вышки, чтобы не было интерференции».
```

```
 </div>
```

```
<div class="overflow-x-auto my-6" style="background-color: #f0f0f0; padding: 10px; border-radius: 10px; border: 1px solid #ccc; box-shadow: 5px 5px 0px #ccc;">
 <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b" style="background-color: #f0f0f0; border: 1px solid #ccc;">
 <th class="py-2 pr-3 font-b" style="width: 30%;">
 <th class="py-2 pr-3 font-b" style="width: 30%;">
 <th class="py-2 font-b" style="width: 40%;">
 </tr>
 </thead>
```

```
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200" style="border: 1px solid #ccc;">
 <td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ </td>
 <td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
 <td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
 </tr>
```

```
 <tr class="border-b border-slate-200" style="border: 1px solid #ccc;">
 <td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ </td>
 <td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
 <td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
 </tr>
```

```

 веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободны
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жад

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
```

```
 Важно: Это работает то-
 еплятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-slate-800
 rder-rose-500 shadow-md">
```

```
 Аналогия: Главный перекресток
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь город, где два района соединены
 её перекроют (удалят вершину) — районы будут
 g>хрупкость системы.
```

```
 </p>
```

```
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-emerald-900
 er border-emerald-500 shadow-md">
```

```
 Телеком: Центральный свитч
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У тебя несколько компьютеров в одной комнате
 абелем (мостом). Сами свитчи — это точки со-
```

```
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
 <summary class="p-4 font-bold text-slate-400 border-
 order-slate-700/50">
```

```
 Душный мод: Код и корень DFS (Скрыто)
```

```
 </summary>
```

```

 <ul class="list-disc list-inside text-sm text-rose-400">
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
 if (!toRemove.includes(c.id)) {
 dedup.set(c.id, c);
 }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],

```

```

const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s
 "dynamic-programming", "johnson-algo", "graph-articulat
 "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

-----  
ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 207 | РАЗМ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера"""
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else None
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]
 return __safe(v)

```

```

PEP 709: с 3.12 компрехеншны встроены во фрейм
на своей строке каждую итерацию. Склеиваем по
if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
Финальные локалы программы (после последней строки)
if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ

---

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

"Не отсортированный список, а «мешок, из которого  
formal:

"Полное бинарное дерево в массиве: дети узла  $i$  ле

complexity: "вставка/извлечение  $O(\log n)$ , «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за  $O(1)$ . На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push\_back / pop\_front за  $O(1)$ . Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти  $O(1)$  — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре",
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "то"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизир",
 "амортизированное", "амортизированного", "амортизир",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи
```

```

{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: " $O(n + m)$ ",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи",
 "аждой букве.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i$ ",
 complexity: " $O(n)$ ",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин",
 "мемоизация", "мемоизацию", "подзадач", "подзадач",
],
 title: "Динамическое программирование",

```

```

 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас"
 formal:
 "Корректность обычно доказывается «леммой о разрезе»"
 demo: "mst",
 simulator: "mst",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь"
 formal:
 "Связный граф: эйлеров цикл все степени чётные; эйлеров путь все степени нечётные"
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного", "планаризация"],
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги"
 formal:
 "V - E + F = 2 для связного планарного графа. Курсивом выделены вершины, рёбра и области"
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "всесторонний поиск"
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают» к корню"
 formal:

```



```

 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок вершин произвольный.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
 },
 {
 id: "boruvka",

```

```
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "граф",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двусторонняя улица).",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить только порядковый номер.",
 formal:
 "sort + unique + binary search: значение $\rightarrow$ индекс",
 complexity: " $O(n \log n)$ ",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова д"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить через них. Ожидаемая высота $O(\log n)$).",
 complexity: "ожидаемо $O(\log n)$ ",
}
```

```

 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива

 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строкам"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где

 (Z-функция) и где встречаются слова словаря (бордер-лист).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция — $O(N)$.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворотом"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встанет на

 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.

 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 },

```

Универсальное пособие • полный исходный код

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id }))
}).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 411 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное
 type: "html",
 category: "Введение",
 content: `
 <section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6 flex-wrap gap-2">
 Введение
 <h2 class="text-3xl font-bold text-white">Базовая база</h2>
 </div>

 <div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 <b class="text-indigo-300">Базовая база — это структура данных, которая
 работает как массив, куча, очередь и стек, бинарное дерево.
 Кодим по минимуму: там, где за нас уже написали.
 </p>
 </div>

 <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за $O(\log n)$
```

<p class="text-slate-300 text-sm">Куча – движок для сортировки элементов, раз, и список с <span class="font-mono text-sm">heapq</span>.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4 border-slate-700">  
<h3 class="text-xl font-bold text-sky-400 mb-4">Очередь</h3>  
<p class="text-slate-300 text-sm mb-4"><b>Очередь</b> (BFS) отличается от кучи (DFS) тем, что первый вышел) отличаются одной операцией снятия элемента</span>: оба конца работают за  $O(1)$ . Обычный список сдвигает весь хвост и стоит  $O(n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700">from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre>
```

<p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4 border-slate-700">  
<h3 class="text-xl font-bold text-amber-400 mb-4">Сортировка</h3>  
<p class="text-slate-300 text-sm mb-4">Если массив отсортирован – понял, в какой половине жить, – и это 20 сравнений. В Python за это отвечает модуль bisect.</p>

```
<pre class="bg-slate-950 border border-slate-700">from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньше 30
```

<p class="text-slate-300 text-sm">Без сортировки –  $O(n^2)$ .</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4 border-slate-700">  
<h3 class="text-xl font-bold text-rose-400 mb-4">Сортировка</h3>  
<p class="text-slate-300 text-sm mb-4">Писать сортировку можно по-разному. В Python есть встроенная функция sorted, которая возвращает отсортированную копию списка. Если нужно отсортировать список на месте, то можно использовать метод sort().</p>

```

left(x) = z # z становится левым р
высоты пересчитываются только у z и x — ост
</div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-teal-400 mb-4
 <p class="text-slate-300 text-sm mb-4">Оба обхо
 тару». Вся разница в таре: стек уходит вглуби
 ext-emerald-300">0(V + E): каждая верш
 <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank): # tank: список-стек
 seen, tank = {start}, tank([start])
 while tank:
 v = tank.pop() if isinstance(tank, list) else t
 for to in graph[v]:
 if to not in seen:
 seen.add(to); tank.append(to)
 return seen</pre>
 <p class="text-slate-300 text-sm">Дальше эта па
 надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-slate-300 mb-
 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 border-indigo-500 shadow-md">
 Аналогия 1: Схема кинозала
 </div>
 <p class="text-slate-300 text-sm">Дерев
 азателей-«usher'ов» не нужно. Место n, дети
 рху вниз и слева направо.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 rounded
 <h2 class="text-3xl font-bold text-white">Дейкст
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1:
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Аналогия 1:
 <p class="text-xl font-mono text-white">Аналогия 1:
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstra" class="h-40">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 охождение улицы), Дейкстра сломается, так как

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-2">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-2">
 г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Флойд

```



```

 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы двигаемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является сло
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);
 }
}

</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие",
 "Да, если заранее хранить суммы в нулевом уровне",
],
 correctIndex: 1,
 explanation: "Два (в 2D – четыре) блока накрывают
 обратная операция, но применить её к перекрытию
 ммы, билет 5."
},
{
 question: "Какие операции подходят разреженной та
 options: [
 "Любые ассоциативные",
 "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
 "Только сумма и произведение"
],
 correctIndex: 1,
 explanation: "Нужно $op(x, x) = x$: тогда дважды на
 приём и уходит в префиксные суммы."
},
{
 question: "Между запросами элементы матрицы меняю
 options: [
 "Разреженную таблицу: она перестраивается за $O(n^2)$ ",
 "Префиксную матрицу: достаточно поправить одну
 "Дерево отрезков: и таблица, и префиксы статичны",
],
 correctIndex: 2,
 explanation: "Обе структуры статичны: изменение э
 ксной. С обновлениями работает дерево отрезков,
 }
},
"prefix-sums-2d": [
 {
 question: "Сколько памяти занимает двумерная преф
 options: [
 "N × M – как исходная матрица, плюс рамка из ну

```

```

 ной операции нет. Для min, max и gcd берут разро
 },
 {
 question: "Изменили один элемент матрицы. Что ста
 options: [
 "Поправится одна клетка за $O(1)$ ",
 "Нужен пересчёт $O(N \cdot M)$: от этой клетки зависит
 "Ничего не станет: таблица остаётся верной"
],
 correctIndex: 1,
 explanation: "Каждая клетка $S[i][j]$ включает в се
 я частые, это задача дерева отрезков (билет 1).
 }
],
"euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечёт
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это
 ётной, заканчиваем в другой."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная
 options: [
 "Потому что Гамильтон — болезнь, и лечить её т
 "Потому что для эйлерова цикла есть простой кри
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критери
 ого критерия. Это NP-полная задача, которую не
 },
 {

```

```

 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
 question: "Планарный граф имеет V=6, E=12. Возможн
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с V=
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, $3V - 6 = 9$. $10 > 9$
 }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...s
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventD
 chapterId ? latestByChapter.get(chapterId) ?? null
);

 useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDet
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !==
 }
 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, ha
 }, [chapterId]);

```

```

 }, [chapterId, currentStep, maxStep, goToStep, selectStep]
 }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
 typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
 typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
 Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>)
 : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**
 * Нормализованная матрица, пришедшая из выполненного Рунима
 *
 * Размерность берётся из самих данных: если пользователь
 * ввёл 5 вершин, визуализация рисует 5x5, а не подставляет его
 * 7x7 (раньше ячейки «не той» размерности превращались в 0)
 *
 * Поддерживаются все учебные способы хранения:
 * list[list] d = [[0, 4], [999, 0]]
 * list[dict] d = [{0: 0, 1: 4}, {1: 0}]
 * dict[dict] d = {0: {0: 0, 1: 4}}
 * dict[(i, j)] d = {(0, 1): 4} — ключ после запятой
 */

```

```

export interface LiveMatrix {
 rows: number;
 cols: number;
 /** Значение ячейки; null — таких данных в коде пользователя нет
 at: (i: number, j: number) => number | null;
}

```

```

 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 cols = Math.max(cols, j + 1);
};

const asArray = vizArray(value);
if (asArray) {
 asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
 const inner = vizArray(row);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
 return;
 });
 }
 const record = vizRecord(row);
 if (record) {
 for (const [key, cell] of Object.entries(record)) {
 const j = Number(key);
 if (Number.isInteger(j)) put(i, j, cell);
 }
 }
 });
} else {
 const record = vizRecord(value);
 if (!record) return null;
 for (const [key, cell] of Object.entries(record)) {
 const pair = parsePairKey(key);
 if (pair) {
 put(pair[0], pair[1], cell);
 continue;
 }
 const i = Number(key);
 if (!Number.isInteger(i)) continue;
 const inner = vizArray(cell);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
 continue;
 });
 }
 }
}

```



-----  
/\* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные */
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>({
 detail: { chapterId, demoId }
 }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
 if (typeof window === "undefined") return () => {};
 const latest = latestDemoByChapter.get(chapterId);
 if (latest !== undefined) cb(latest);
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
 if (detail && detail.chapterId === chapterId) cb(detail.demoId);
 };
 window.addEventListener(DEMO_CHANNEL, handler);
 return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

## Универсальное пособие • полный исходный код

```

tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; листы"},
 { name: "i", role: "индекс элемента массива; его левый и правый дети"},
 { name: "v", role: "текущая вершина дерева отрезка"},
 { name: "l, r", role: "границы отрезка, за которыми лежат левый и правый дети"},
 { name: "m", role: "середина отрезка m = (l + r) // 2"},
],
},
treap: {
 vizTitle: "Декартово дерево: build → split → merge",
 stepNote: "Шаг = один спуск в split/merge/erase (перезаписывается целиком.",
 code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 3]]
n = len(points) # сколько пар (x; y) – столько вершин
mid = 7 # разрез для split: x <= mid < y
key = 9 # какую точку удаляем в erase
step = 0 # номер шага – листает кадры дерева
height = 0 # высота построенного дерева
root = None

--- тело алгоритма ---
class Node:
 def __init__(s, x, y):
 s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
 """Декартово дерево из отсортированных по x точек:
 pts = sorted(pts, key=lambda p: p[0])
 stack = []
 for x, y in pts:
 cur = Node(x, y)
 last = None
```



```

while x.p:
 p = x.p; g = p.p
 step += 1
 if g is None:
 case = "zig" # родители
 rot(x)
 elif (g.l is p) == (p.l is x):
 case = "zig-zig" # x и p -
 rot(p); rot(x)
 else:
 case = "zig-zag" # змейка
 rot(x); rot(x)
 xk, pk, gk = x.k, p.k, (g.k if g else 0)
 print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
 root = x
 print(" корень теперь", x.k, "| обход:", inorder(root))
 return x

```

# 1) Zig: родитель x – уже корень (дерево 40 -> 20)  
 root = N(40); root.l = N(20); root.l.p = root  
 splay(root.l)

# 2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые  
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)  
 splay(root.l.l)

# 3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)  
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)  
 splay(root.l.r),

```

variables: [
 { name: "case", role: "какой случай сработал: \"zig-zig\", \"zig-zag\", \"zig\""},
 { name: "step", role: "номер применения случая – 1, 2, 3"},
 { name: "xk / x", role: "ключ узла, который поднимается"},
 { name: "pk / p", role: "ключ родителя x перед поворотом"},
 { name: "gk / g", role: "ключ деда – нужен в Zig-Zig"},
 { name: "root", role: "корень дерева после расшейки"},
]

```

```

 stepNote: "Шаг = заполнение очередного $P[i] = P[i-1]$
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] # $P[0] = 0$ – пустой префикс
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)`
variables: [
 { name: "a", role: "исходный массив 1D-демонстрация" },
 { name: "n", role: "длина массива a; префиксов будет n" },
 { name: "P", role: "префиксные суммы: $P[i] = a[0] + \dots + a[i-1]$ " },
 { name: "i", role: "граница префикса: на шаге считаем до i" }
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизация)",
 stepNote: "Шаг = событие внутри fibМемо(k): вызов, возврат",
 code: `n = 5
memo = {}

def fib(k):
 if k in memo:
 return memo[k]
 if k <= 2:
 return 1
 print(f"считаем fib({k-1}) + fib({k-2})")
 r1 = fib(k - 1)
 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))`,
variables: [
 { name: "n", role: "целевой номер числа Фибоначчи" },
 { name: "k", role: "аргумент текущего рекурсивного вызова" },
 { name: "memo", role: "кэш уже посчитанных значений" }
]

```

```

code: `from collections import deque

q = deque(["A"]) # очередь BFS
u = "B" # сосед, найденный из текущ
q.append(u) # enqueue соседа — в конец
front = q[0] # голова очереди: её обрабо
v = q.popleft() # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
 variables: [
 { name: "front", role: "голова очереди — вершина,
 { name: "v", role: "вершина, извлечённая из очере
 { name: "u", role: "сосед вершины v, которого кла
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на
 code: `from collections import deque

st, q = [], deque() # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0 # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
 i += 1
print("тарелка:", top, "| первый из очереди:", first, "
 variables: [
 { name: "top / front", role: "верх стопки (стек)
 { name: "i", role: "индекс в массиве кучи; родите
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к неп
 code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

## Универсальное пособие • полный исходный код

```

v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага — двигает по какому-то ребру

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => новая компонента
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
 { name: "n", role: "число вершин; в демонстрации — 10" },
 { name: "adj", role: "списки смежности — тот же граф, что в примере" },
 { name: "comp", role: "разметка: номер компоненты", range: "-1 ... comps" },
 { name: "used", role: "булев вариант разметки — для BFS" },
 { name: "comps", role: "счётчик найденных компонент" },
 { name: "c", role: "номер компоненты, которую красим сейчас" },
 { name: "v", role: "вершина, где обход находится" },
 { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
 { name: "i / step", role: "номер шага — перематываем кадр" },
],
},
"top-sort": {

```

```
def dfs1(v):
 used[v] = True
 for to in adj[v]:
 if not used[to]:
 dfs1(to)
 order.append(v) # выходим из вершины — она

for v in adj:
 if not used[v]:
 dfs1(v)
 print("DFS из", v, "закончен | order:", order)

транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
 adjT[v] = []
for v in adj:
 for to in adj[v]:
 adjT[to].append(v)
print("обратный граф:", adjT)

фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
 v = order[i]
 if v in comp:
 continue
 stack = [v]
 comp[v] = c
 while stack:
 u = stack.pop()
 for to in adjT[u]:
 if to not in comp:
 comp[to] = c
 stack.append(to)
 print("SCC", c, "собрана из", v, ":", comp)
```



```

 },
 "bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v, to) проверяется, является ли мостом. Если да, то его можно удалить, не нарушая связности графа.",
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1, "B": 2, "G": 3}
timer = 3
v, to = "B", "G"
is_bridge = low[to] > tin[v]
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range: "A, B, G"},
 { name: "to", role: "сосед; ребро (v, to) проверяется", range: "A, B, G"},
 { name: "tin[v]", role: "время входа в вершину", range: "1, 2, 3"},
 { name: "low[v]", role: "минимум tin, куда можно вернуться", range: "1, 2, 3"},
 { name: "timer", role: "глобальный счётчик времени", range: "1, 2, 3"}
],
 },
 "euler-path-vs-cycle": {
 vizTitle: "Эйлеров маршрут: алгоритм Иера",
 stepNote: "Шаг = переход по неиспользованному ребру, если такой есть. Если нет, то путь закончен.",
 code: `n = 5
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]
adj = {v: [] for v in range(n)}
for i, (u, v) in enumerate(edges):
 adj[u].append((v, i))
 adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)}
odd = [v for v in range(n) if deg[v] % 2 == 1]
start = odd[0] if odd else 0
used = [False] * len(edges)
path = [start]
last = start
stack = [start]
v = start
step = 0`
 },
}

```

```

V = n # вершин для формулы
E = sum(len(a) for a in adj) // 2 # рёбер: в списке смежн
F = 2 - V + E # граней: из $V - E + F = 2$

--- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "| $V - E + F = 2$ ")
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
 { name: "n", role: "число вершин графа на демонстрации" },
 { name: "adj", role: "списки смежности — тот же граф" },
 { name: "color", role: "массив цветов вершин (0 = не занят)" },
 { name: "v", role: "вершина, которую красим сейчас" },
 { name: "u", role: "сосед вершины v, чей цвет уже занят" },
 { name: "delta", role: "максимальная степень $\Delta = \max \deg v$ " },
 { name: "chi", role: "сколько цветов реально понадобится" },
 { name: "i / step", role: "номер шага — перематываем" },
 { name: "V, E, F", role: "для вкладки «Планарность»" },
],
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины",
 code: `import heapq

```

## Универсальное пособие • полный исходный код

```

nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes) # 7 вершин => n - 1 = 6 работ
m = len(E) # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0 # старт – База
```

```
--- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n): # итерация i = 1 ... n-1
 last_changed = ""
 for u, v, w in E: # шаг: релаксация ребра (u, v)
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 last_changed = v
 if last_changed == "":
 print(f"итерация {i}: изменений нет – сошлось для всех вершин")
 break
 print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}:" for k in nodes))
```

```
контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
 if dist[u] != INF and dist[u] + w < dist[v]:
 cycle_edge = (u, v, w)
 break
```

```
if cycle_edge:
 print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
 print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
 variables = [
 { name: "n", role: "число вершин графа – ровно n-1"},
 { name: "m", role: "число рёбер, по которым проходят"},
 { name: "i", role: "номер итерации (после i итераций)"},
 { name: "u, v, w", role: "текущее ребро: откуда, куда"},
 { name: "dist[v]", role: "расстояние; если обновилось"},
],
```

```

 на-Форда" },
 { name: "u, v", role: "ребро (u, v) перевзвешивает",
 { name: "d[v]", role: "расстояние Дейкстры в новом графе"
 },
],
},
mst: {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "C"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "D", "F"),
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`
variables: [
 { name: "n", role: "длина строки s", range: "n = len(s)" },
 { name: "i", role: "позиция, для которой считаем z[i]" },
 { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
 { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[0:z[i]]" }
],
},
"aho-corasick": {
 vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
 stepNote: "Шаг = один символ текста: state = go(state, c). Если не найдено, то переходят в корень. Если совпадают с картинкой «водопада».",
 code: `words = ["HERS", "HIS", "SHE", "HE"] # образы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, в которую нужно перейти, если не найдено

```

```

 queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
 """Переход автомата: сначала по бору, иначе спускаем
 while v != 0 and (v, ch) not in nxt:
 v = fail[v]
 return nxt.get((v, ch), 0)

for i, c in enumerate(text):
 state = go(state, c)
 v = state
 while v != 0: # собираем все образцы
 if v in term:
 matches.append((i, term[v]))
 v = fail[v]
 print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
 { name: "words", role: "набор образцов – по нему"},
 { name: "text", role: "сканируемый текст – подста"},
 { name: "state", role: "текущее состояние автомата"},
 { name: "i / c", role: "индекс и сам символ текста"},
 { name: "nxt", role: "переходы бора: nxt[(v, ch)]"},
 { name: "fail", role: "fail[v] – вершина наибольшей"},
 { name: "term", role: "в каких вершинах заканчивае"},
 { name: "queue", role: "очередь BFS, которой натя"},
 { name: "matches", role: "найденные вхождения: (ин"},
 { name: "nodes", role: "сколько вершин в боре (ко"},
],
},
"complexity-classes": {
 vizTitle: "Классы сложности: рост по n и ops",
 stepNote: "Шаг = одна точка (n, ops): визуализация",
 code: `n = 1 # размер входа

```

```

 },
 intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
 },

 /* — Дополнительные демонстрации внутри страниц (вкл.)

 "planarity-euler-formula#greedy": {
 vizTitle: "Жадная покраска вершин ($\chi \leq \Delta + 1$)",
 stepNote: "Шаг = вершина v получает наименьший цвет",
 code: `n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj) # максимальная степень
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

--- тело алгоритма ---
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
 variables: [
 { name: "color", role: "цвета вершин (0 = не покрашено)", range: "0 ... n" },
 { name: "v", role: "текущая вершина", range: "0 ... n-1" },
 { name: "delta / chi", role: "максимальная степень / количество цветов", range: "0 ... n" },
 { name: "i / step", role: "номер кадра", range: "0 ... n-1" },
]
 },

```

```

],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)",
 stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
 code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

--- тело алгоритма ---
for e, (a, b) in enumerate(edges):
 v, u = a, b
 used = set(inc[a].values()) | set(inc[b].values())
 c = 1
 while c in used:
 c += 1
 ecolor[e] = c
 inc[a][e] = c
 inc[b][e] = c
 chi_e = max(chi_e, c)
 print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= \Delta")
variables: [
 { name: "ecolor", role: "цвета рёбер – рёбра на графе", range: "0..delta" },
 { name: "e", role: "номер текущего ребра", range: "0..len(edges)-1" },
 { name: "v, u", role: "концы текущего ребра – подмножества V", range: "0..n-1" },
 { name: "inc", role: "для каждой вершины: какие цвета имеют инцидентные рёбра", range: "0..delta" },
 { name: "chi_e / delta", role: "χ' – сколько цветов", range: "0..delta+1" }
],
},
"graph-components#dfs": {
 vizTitle: "Компоненты связности: DFS-покраска",

```



```

 },
 "graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает",
 code: `n = 10 # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n)) # DSU: parent[v] = представи
rank = [0] * n # высота дерева – по ней реш
comps = n # сначала каждая вершина – о
e = -1 # номер обрабатываемого ребра
u, v = -1, -1 # концы текущего ребра
step = 0 # номер шага – двигает по ка

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

def union(a, b):
 global comps
 ra, rb = find(a), find(b)
 if ra == rb:
 return False # ребро внутри
 if rank[ra] < rank[rb]:
 ra, rb = rb, ra
 parent[rb] = ra
 if rank[ra] == rank[rb]:
 rank[ra] += 1
 comps -= 1
 return True

for e, (a, b) in enumerate(edges):
 u, v = a, b
 step = e + 1
 union(a, b)
print("компонент:", comps)

```

```

k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
 print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
 { name: "step", role: "номер кадра сведения (0..6)" },
 { name: "cls", role: "класс задачи A, которую сведём к КЛИКА" },
 { name: "n", role: "размер входа x; после сведения к КЛИКА" },
 { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
 vizTitle: "Верификатор 3-SAT: проверка сертификата",
 stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
 code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0] # сертификат: x1=1, x2=0, x3=1
k = 0 # номер проверяемого дизъюнкта
ok = True # вердикт верификатора

--- тело алгоритма ---
for k, cl in enumerate(clauses):
 hit = False
 for l in cl:
 val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
 if val == 1:
 hit = True
 ok = ok and hit
 print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
 variables: [
 { name: "cert", role: "сертификат – назначение переменных" },
],
},

```

```

edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представител
rank = {v: 0 for v in "ABCDEFGHI"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе 51)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое рассматриваем

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
 variables: [
 { name: "i", role: "номер текущего ребра – перебираем"},
 { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
 { name: "parent", role: "DSU-массив: цвета компонент"},
 { name: "mst", role: "рёбра, уже взятые в остов", "51"},
 { name: "total", role: "вес остова (51 на этом графе)"},
],
},
"mst#prim": {
 vizTitle: "Остовное дерево: Прим (рост из вершины E)",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

```

```

vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА"
code: `n = 9 # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"} # DSU: корень компоненты
cheapest = {} # корень компоненты -> самое дешёвое ребро
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
phase = 0 # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None # ребро-кандидат

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len(mst) < n - 1:
 phase += 1
 cheapest = {}
 for (w, a, b) in edges: # КАЖДАЯ компонента ОДНОВЕРХОВА
 ra, rb = find(a), find(b)
 if ra == rb:
 continue
 u, v = a, b
 if ra not in cheapest or w < cheapest[ra][0]:
 cheapest[ra] = (w, a, b)
 if rb not in cheapest or w < cheapest[rb][0]:
 cheapest[rb] = (w, a, b)
 if not cheapest:
 break # граф несвязен
 for (w, a, b) in cheapest.values():
 ra, rb = find(a), find(b)
 if ra == rb:
 continue # две компоненты

```

```

 { name: "k", role: "уровень: блоки $2^k \times 2^k$ (шаг
 { name: "st2[{r,c,k,k}]", role: "минимум квадратн
 },
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем
 { name: "kx, ky", role: "уровень самого крупного
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
 S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (

```

```

const keep: string[] = [];
for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, demoId: string) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}

```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

-----  
ФАЙЛ 26/111: src/data/tickets/ticket1\_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМ: 1.5 КБ

-----  
import { Chapter } from '../types';

```

span class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
– это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ – любая операция, котор
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается – возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента – спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 <div class="border border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

 </div>
 <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v])</div>
свой баланс. Это и есть отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
 кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
за $O(1)$, но строятся один раз и умирают при
ми.</p>
```

```
 <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
 <p class="text-slate-400 text-sm mb-1">
n ячеек (не $2n!$) – иначе на $n = 6$ поймать п
n ... $2n-1$] и n – степень двойки (или использ
 <p class="text-slate-400 text-sm">•
ть лень во всё дерево – это $O(n)$ и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
 в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

```
 Код: построение и запрос суммы (C
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

 y[4 * MAXN]; // «обещание»: прибавит

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;
 int m = (l + r) / 2;
 tree[2 * v] += lazy[v] * (m - l + 1); // п
 lazy[2 * v] += lazy[v];
 tree[2 * v + 1] += lazy[v] * (r - m); // и
 lazy[2 * v + 1] += lazy[v];
 lazy[v] = 0; // о
}

void add(int v, int l, int r, int L, int R, int x) { /
 if (r < L || R < l) return;
 if (L <= l && r <= R) {
 tree[v] += x * (r - l + 1);
 lazy[v] += x; // д
 return;
 }
 push(v, l, r); // о
 int m = (l + r) / 2;
 add(2 * v, l, m, L, R, x);
 add(2 * v + 1, m + 1, r, L, R, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр  
 г накопиться. Забыл push – получил правильн  
 </div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">  
 Код: итеративное дерево (2n, снизу)  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounded  
 # для удобств



```
ext-emerald-300">w ≥ 0}, зафиксирова
ию расстояния!</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Ограничения: минус ломает жадно

```
</div>
```

```
<p class="text-slate-300 text-sm mb
lass="font-mono text-emerald-300">s→a = 2</
ono text-emerald-300">b→a = -2. Верн
начала вытащит <span class="font-mono text-
o text-emerald-300">3 - 2 = 1 придёт
Джонсона (билет 17).</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-blue-400 m
```

```
<p class="text-slate-300 text-sm mb-4">Демон
D, Дом 2 E, ТЦ F, Метро G) и 11 одностор
ршины <span class="font-mono text-emerald-3
(u → v): переменные <span class="fon
>,
ают картинку напрямую.</p>
```

```
<ol class="list-decimal pl-5 space-y-2 text
 Инициализация. <span class="
text-emerald-300">∞; в кучу кладём <
emerald-300">prev пуст.
```

```
Извлечь минимум. Достаём пар
"font-mono text-emerald-300">du. Если
xt-emerald-300">done – пара устарела
```

```
Зафиксировать. <span class="
меняется.
```

```
</div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
y-2 pr-2">лучший известный путь $s \rightarrow v$ </td><td>
 <tr class="border-b border-slate-
y-2 pr-2">предок v на текущем лучшем пути</td><td>
 <tr class="border-b border-slate-
lass="py-2 pr-2">вершины с окончательным $dist[v]$ </td></tr>
 <tr class="align-top"><td class="py-2">исходная вершина, вершина)</td><td class="py-2">извлечение v из очереди</td></tr>
 </tbody>
 </table>
 </div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">Извлечение из кучи

 <p class="text-slate-300 text-sm mb-4">Инициализация
 $s \rightarrow u$. Отсюда
 $dist[v] = \delta(s, v)$ для всех достижимых v
 $dist[v] = \infty$ для недостижимых v
```

```
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">Извлечение из кучи
```

```

 </div>
 </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">Дейкстра на графе
 <div class="overflow-x-auto my-4">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-b">Взвешенный ребро
 <th class="py-2 pr-3 font-b">Взвешенный узел
 <th class="py-2 font-bold">Путь
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-600">
 <td>priority_queue (C++)</td><td class="py-2 pr-3">
 й выбор</td></tr>
 <tr class="border-b border-slate-600">
 <td>ease-key</td><td class="py-2 pr-3 font-mono">
 очереди</td></tr>
 <tr class="border-b border-slate-600">
 <td>поиск минимума</td><td class="py-2 pr-3 font-mono">
 </tr>
 <tr class="border-b border-slate-600">
 <td>><td class="py-2 pr-3 font-mono">0(V + E)</td></tr>
 <tr class="align-top"><td class="font-mono">0(E + V · C)</td><td class="py-2">ц
 </tr>
 </tbody>
 </table>
 </div>

<details class="bg-slate-900/40 rounded-lg border-2 border-slate-800">
 <summary class="p-4 font-bold text-slate-300 border-b border-slate-700/50">» Код: Дейкстра на
 <div class="p-5 text-sm text-slate-300"

```

```

 "E": [{"F", 1}],
 "F": [],
 "G": [{"D", 1}, {"E", 2}],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
 <p class="text-slate-400 text-xs">П
 же вершины, потому что читает ваши <span c
nt-mono">done.</p>
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (priority
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 priority_queue<pair<long long,int>;,
 vector<pair<long long,int>>&
 greater<pair<long long,int>>
 pq.push({0, s});
 while (!pq.empty()) {
 auto [du, u] = pq.top(); pq.pop();
 if (du > d[u]) continue; // устарев
 for (auto [v, w] : g[u]) {

```

```
used[u] = True # фиксируем d[u]
for v in range(n): # релаксируем v
```

```
 if d[u] + w[u][v] < d[v]:
 d[v] = d[u] + w[u][v]
```

Т. Память – матрица

```
</div>
</details>
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">

```

```
<div class="bg-rose-950/30 p-5 rounded-lg border">
 <p class="text-sm font-bold text-rose-300">

```

```
<ul class="list-disc pl-5 space-y-2 text-rose-300">
 Отрицательные веса. Алгоритм релаксации вершины необратима. Нужен Форд–Беллмановский цикл.
 Отрицательные циклы. При наличии отрицательного цикла не существует минимального числа. Сначала проверьте граф Фордом–Беллманом.
 Ленивое удаление. Без предварительной обработки вершина обрабатывается несколько раз:
 Переполнение INF. u уходит в отрицательное число и «улетит» в u.
```

```
Ориентированность. Демонстрация алгоритма на графе: каждое ребро добавляется в оба списка.
 Дейкстра ≠ BFS. BFS – частный случай Дейкстры. С разными весами FIFO ломается, нужна куча.
 «Одна пара». Алгоритм с двумя приоритетами и выход при извлечении цели (билет 17), но не для всех графов.

```

```
</div>
```

```
<p class="text-slate-300 text-sm mt-4 mb-4">
 о скоростью O(V^3); б
 после перевзвешивания потенциалов). Представим граф с
```

```

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Патруль Бэтмена на
 </div>
 <div class="my-3 rounded-lg overflow-hidden">

 ночей подряд, объезжая абсолютно все ули
 овно 1 выходной в году, когда он отде
 -ю ночь Бэтмен делает контрольный проезд: е
 о-то района снова уменьшилось – значит, Джо
 </div>

```

```

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Отрицательный цикл: временная п
 </div>

```



```

 </table>
 <p class="text-slate-400 text-xs mt-3">
бном» порядке: S→A,
порядок рёбер местами – и понадобится до ше
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <p class="text-slate-300 text-sm mb-3">
тава A имеет суммарный вес <span cla
веет» на 2.</p>
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2 pr-2 fo
td></tr>
 <tr class="border-b border-slate
-2 font-mono">-2</td><td class="py-2 pr-2 f
/td></tr>
 <tr class="border-b border-slate
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>
 <tr class="border-b border-slate
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
 <tr class="align-top"><td class
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
 </tbody>

```

```

<div class="overflow-x-auto">
 <table class="w-full text-xs text-l
 <thead>
 <tr class="text-slate-500 b
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 font-l
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
 <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
 </tr>
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётся
emerald-300">обнаруживает (d[i][i] < 0)<
 <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
 <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
 </tbody>
 </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900/60 p-4 rounded-lg l
 <p class="text-xs font-bold text-amber-:
 :</p>
 <pre class="bg-slate-950 p-3 rounded text-xs font-mono

```

```

if changed_vertex == -1:
 return dist, prev, None # отрицательны

changed_vertex улучшен на n-й итерации:
поднимаемся по prev на n шагов – гарантированно з
y = changed_vertex
for _ in range(n):
 y = prev[y]

cycle = [y]
v = prev[y]
while v != y:
 cycle.append(v)
 v = prev[v]
cycle.append(y)
cycle.reverse()
return dist, prev, cycle

демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
 {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D",
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None –

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D",
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # все
 <p class="text-slate-400 text-xs">П
 имая вершина с INF<
 ля float('inf')</sp
 </div>
</details>

```

```

 vector<int> cycle;
 for (int v = y;; v = p[v]) {
 cycle.push_back(v);
 if (v == y && cycle.size() > 1) break;
 }
 reverse(cycle.begin(), cycle.end());
 return cycle;
 }

```

```

int main() {
 int s = 0;
 int x = bellman_ford(s);
 if (x == -1) cout <<< "отрицательных циклов не";
 else {
 cout <<< "цикл: ";
 for (int v : negative_cycle(x)) cout <<< v << " ";
 cout <<< "\n";
 }
}
</code></pre>

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg">

<summary class="p-4 font-bold text-slate-300"> Оптимизации: ранний выход из цикла

<div class="p-5 text-sm text-slate-300">

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
 rt deque

```

```

def spfa(n, adj, s):
 """adj[u] = [(v, w), ...]. Релаксируем только «живые» вершины.
 INF = float("inf")
 dist = [INF] * n
 in_queue = [False] * n
 counter = [0] * n # сколько раз вершина попала в очередь
 dist[s] = 0
 q = deque([s])
 in_queue[s] = True

```

```

отрицательным циклом
 Переполнение INF.
 INT_MAX превращается в отрицательное значение, что приводит к переполнению
 запасом и long long
 Неориентированный граф с минимальным циклом -3 – это цикл длины 3 в
 отрицательном цикле. Минусы в неориентированном графе.
 Цикл есть, но не от старта. -3
. Нужна проверка всего графа – добавление ребра (17) и запустите от неё.
 «Ответ $-\infty$ ». Если отрицательный цикл, то dist[v] = -\infty: печатать ответ
 по ходу в обратном графе.
 SPFA как «быстрый Форд–Беллман» в условии V, E \le 10^5

</div>

<p class="text-slate-300 text-sm mt-4">Сложные задачи
 Потенциалы
 Алгоритм Дейкстры. Флойд–Уоршелл (16) решает все задачи на кратчайшие пути.
 Порядок для DAG – билет 9.</p>
</div>
</div>
</section>`
},
{
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы. Пути
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">16. Графы. Поиск кратчайшего пути. Флойд</h2>
 </div>

```

```

<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Таблица пересадок в
 </div>
 <p class="text-slate-300 text-sm mb
це только соседние станции. Затем говорите:
 через 1 там, где
плакат окончательный. Порядок «разрешений»
an>.</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Почему k обязан быть
 </div>
 <p class="text-slate-300 text-sm mb
s="font-mono">k один раз и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞, х
пользует «поздние» посредники). Проверка пер
 </div>

```

```

 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
mono text-emerald-300">0 → {0, 4, 7, 2, 6, 1}
"font-mono">0 → 3 → 4 → 6 (2 + 4 + 1
всего в финальной матрице 11 таких пар. Диа
</div>

<div class="bg-slate-800/70 p-5 rounded-lg border">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">конечное</td><td class=
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">INF</td><td class="py-2
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">0</td><td class="py-2">
 <tr class="align-top"><td class=
se-300">< 0</td><td class="py-2 text-rose-400">
делены</td></tr>
 </tbody>
 </table>
 </div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">

 <details class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-300">

```

```

d = [row[:] for row in M]
for a in range(n):
 for b in range(n):
 for c in range(n):
 k, i, j = {"kij": (a, b, c),
 "ijk": (c, a, b),
 "ikj": (c, a, b)}[order]
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
return d

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1]) # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good
4 клетки отличаются; при порядке (i,k,j) их 6</code><
 <p class="text-slate-400 text-xs">В
угих «почти работают» (большинство клеток с
 </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-4

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">> Код: Флойд–Уоршелл.
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te

def floyd_warshall(n, edges, directed=True):
 d = [[INF] * n for _ in range(n)]
 nxt = [[-1] * n for _ in range(n)]
 for i in range(n):
 d[i][i] = 0 # диагональ об

 for u, v, w in edges:

```



```

print(neg) # [] — от

</div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">> Код: C++ (матрица
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <algorithm>;
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
 for (int i = 0; i < n; ++i) d[i][i] = 0;
 for (int k = 0; k < n; ++k) // k
 for (int i = 0; i < n; ++i) {
 if (d[i][k] == INF) continue; // защи
 for (int j = 0; j < n; ++j) {
 if (d[k][j] == INF) continue;
 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
 }
 }
}

bool has_negative_cycle() {
 for (int i = 0; i < n; ++i) if (d[i][i] < 0)
 return false;
}

long long diameter() { // сама
 long long res = 0;
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)

```

```

 <tr class="border-b border-
ой сети</td><td class="py-2 pr-2 font-mono">
и отсутствии положительных циклов</td></tr>
 <tr class="align-top"><td c
o">if d[i][k]+d[k][j] == d[i][j]: cnt[i][j]
а переполнением</td></tr>
 </tbody>
</table>
 <p class="text-slate-400 text-xs mt
> при r[i][k]
V³).</p>
 </div>
</details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-amber-400

```

```

<div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-3
 <ul class="list-disc pl-5 space-y-2 tex
 Порядок циклов. <span cl
ono">(i, j, k) на демо-графе 4 клетки
] = 999 вместо 4).
 Диагональ. <span class="
«починит» её только при наличии цикла, а п
 Переполнение INF. <span c
уходит в минус и «улучшает» всё подряд. Пр
no">long long.
 Параллельные рёбра. Во в
no">d[u][v] стартует с худшего веса.
 Отрицательные циклы. Фор
ых с циклом, ответ
t-mono">d[i][j] = -∞, если существуе
 d[i][k] < ∞</sp
 Недостижимость ≠ 0. В ор
таких пар из 49). Выводить 999 или 0 как «р
 Память. <span class="fon

```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейк
в лоб. На экзамене спрашивают, что сдела
строчки перевзвешивания» до промышленных у
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
 <div class="border border-emerald-500 shadow-md">
```

Аналогия 1: Такси с навигатором

```
</div>
```

```
<p class="text-slate-300 text-sm mb
ы одинаково. А* — тот же таксист, но с нави
/span> прибавляется оценка остатка пути по
т туда, где сумма меньше. Навигатор обязан
/span>): занижать можно, завышать нельзя —
0">h = 0 это снова Дейкстра.</p>
```

```
</div>
```

```
<div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
 <div class="border border-emerald-500 shadow-md">
```

Аналогия 2: Две бригады туннеля

```
</div>
```

```
<p class="text-slate-300 text-sm mb
пускают две: от А и от В. Каждая зона роста
я не в момент встречи, а когда сумма
— иначе можно пропустить более дешёвый обх
ирует.</p>
```

```
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left
 <thead>
```

```
<tr class="text-slate-500 borde
```

```
<th class="py-2 pr-3 font-b
```

```
<th class="py-2 pr-3 font-b
```

```

 <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
er border-amber-500 shadow-md">
 ⚡ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-3">
 Дейкстра (билет 14) быстрая, но
ленный. Алгоритм Джонсона делает финт: пере
ь теми же.
 </p>
 <ol class="text-sm text-slate-300 sp>
 Добавляем фиктивную вершн
 Запускаем от s Форда-Бел
ажный факт:</i> Джонсон НЕ «удаляет» отрица
кой и прекращает работу, так как при отрица
 Перевзвешиваем: <span class=
тво следует из того, что $h(v) \leq h(u) + w(u,$
 Магия: для любого пу
) – $h(\text{финиш})$, то есть все пути между фик
еняется.
 Запускаем быструю Дейкстр
lass="font-mono text-emerald-300"> $d(u, v) =$

 <p class="text-slate-400 text-xs mt-3">
 еженном графе ($E \approx V$) это $O(V^2 \log V)$ проти
ойд – для маленьких или очень плотных.</p>
 </div>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Если
азад из t (по перевёрнутым рёбрам). Волны р
: в «широком» графе (дорожная сеть) это эконо
огда волны «встретились» в одной вершине; н
огда

```

```

 <p class="text-slate-400 text-sm mb-1">
именимой к отрицательным весам и даёт все п
 <p class="text-slate-400 text-sm mb-1">
in_f + min_b ≥ μ. Остановка по встрече даёт
 <p class="text-slate-400 text-sm mb-1">
ок сломается. Там нужен Dial.</p>
 <p class="text-slate-400 text-sm">•
ктке меньше бесполезных релаксаций.</p>
</div>

<p class="text-slate-300 text-sm mt-4">с
py-0.5 rounded text-xs font-mono text-emera
class="bg-slate-950 px-2 py-0.5 rounded tex
м числом раскрытий, 0-1 BFS <code class="bg
te-800">0(V + E)</code>, Dial <code class="
late-800">0(E + V·C)</code>, Дейкстра с куч
merald-300 border border-slate-800">0(E + V
o text-rose-300 border border-slate-800">0('

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: Джонсон целиком (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 heapq
INF = float("inf")

def johnson(n, edges):
 # edges: (u, v, w)
 # 1) потенциалы: Форд–Беллман от фиктивной вершины
 h = [0] * (n + 1)
 for s in range(n):
 # из s все рёбра
 # V итераций: V-
 changed = False
 for u, v, w in edges:
 if h[u] + w < h[v]:
 h[v] = h[u] + w
 changed = True

```

```

def bidirectional(g, rg, s, t): # rg – граф с пер
 INF = float("inf")
 df = {s: 0}; db = {t: 0}
 qf = [(0, s)]; qb = [(0, t)]
 mu = INF
 while qf and qb:
 if qf[0][0] + qb[0][0] >= mu: # вот прав
 return mu
 dist, v = heapq.heappop(qf)
 if dist > df.get(v, INF): continue
 for to, w in g[v]:
 nd = dist + w
 if nd < df.get(to, INF):
 df[to] = nd; heapq.heappush(qf, (nd, to))
 if to in db:
 mu = min(mu, nd + db[to]) # встреча:
 dist, v = heapq.heappop(qb)
 if dist > db.get(v, INF): continue
 for to, w in rg[v]:
 nd = dist + w
 if nd < db.get(to, INF):
 db[to] = nd; heapq.heappush(qb, (nd, to))
 if to in df:
 mu = min(mu, nd + df[to])
 return mu

```

# --- A\*: приоритет f = g + h ---

```

def astar(g, s, t, h): # h(v) – допустим
 INF = float("inf")
 gdist = {s: 0}
 heap = [(h(s), 0, s)] # (f, g, v)
 closed = set()
 while heap:
 f, gv, v = heapq.heappop(heap)
 if v == t:
 return gv # ранний выход:
 if v in closed:
 continue

```

```
 <p>• Зачем нужен ранний выход в Дейкстры
 смысла нет.</p>
 <p>• Почему 0-1 BFS работает без кучи, если
 же отсортирована.</p>
 </div>
</details>
</div>
</div>
</section>
 },
];
```

---

ФАЙЛ 28/111: src/data/tickets/ticket18\_20\_mst.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 231 | РАЗМЕР: 1.1 КБ

---

```
import { Chapter } from '../../types';

/**
 * Билеты 18 · 19 · 20 – одна страница на три алгоритма
 * минимального остоного дерева (MST). Так же, как в с
 * KruskalSimulator, три алгоритма живут на одном графе
 * переключаются вкладками «Краскал / Прима / Борувка».
 *
 * Синхронизация с компилятором: PAGE_SYNC["mst"] + под
 * "mst#kruskal", "mst#prim", "mst#boruvka" (вкладка со
 * через emitVizDemo, код в панели следует за ней).
 */
export const tickets18to20Mst: Chapter[] = [
 {
 id: "mst",
 title: "18–20. Графы. Остовное дерево: Краскал, При
 type: "html",
 description:
 "Три способа построить минимальное остовное дерев
 .",
 }],
```

```

 <tr class="text-slate-500 border-b" >
 <th class="py-2 pr-3 font-b" >
 <th class="py-2 pr-3 font-b" >
 <th class="py-2 pr-3 font-b" >
 <th class="py-2 pr-3 font-b" >
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-500" >
 <td class="py-2 pr-3 font-b" >
 <td class="py-2 pr-3">Рёбра
 <td class="py-2 pr-3 font-m" >
 <td class="py-2 pr-3 font-m" >
 <td class="py-2">рёбра даны
 </tr>
 <tr class="border-b border-slate-500" >
 <td class="py-2 pr-3 font-b" >
 <td class="py-2 pr-3">Пастёр
 <td class="py-2 pr-3 font-m" >
 <td class="py-2 pr-3 font-m" >
 </td>
 <td class="py-2">плотный гр
 </tr>
 <tr class="align-top">
 <td class="py-2 pr-3 font-b" >
 <td class="py-2 pr-3">Все к
 <td class="py-2 pr-3 font-m" >
 <td class="py-2 pr-3 font-m" >
 <td class="py-2">параллельны
 </tr>
 </tbody>
</table>
</div>

<div class="grid grid-cols-1 xl:grid-cols-3" >
 <div class="bg-slate-800 p-6 rounded-lg" >
 <div class="absolute -top-3 left-4" >

```



```

 дом вместо почти-константного find/union.</p>
 <p class="text-slate-400 text-sm mb-1">
 и Краскал построил минимальный остовный <i>
 <p class="text-slate-400 text-sm">•
ин раз, DSU спасает от дубля (find(u) == find(v))
</div>

<p class="text-slate-300 text-sm mt-4">
py-0.5 rounded text-xs font-mono text-emerald-300
бесплатный); Прим с двоичной кучей <code>
rder-slate-800">O(E log V)</code>, на плотн
text-xs font-mono text-emerald-300 border border-1
d text-xs font-mono text-emerald-300 border border-1
ex <code class="bg-slate-950 px-2 py-0.5 rounded">
/p>

<details class="bg-slate-900/40 rounded-lg border border-1
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код 18: Краскал + DSU (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
 lt;pair<int, pair<int,int>>>>
vector<int> p, rnk;

int find(int v) { return p[v] == v ? v : p[v] = find(p[v]); }
bool unite(int a, int b) {
 a = find(a); b = find(b);
 if (a == b) return false; // уже в одном компоненте
 if (rnk[a] < rnk[b]) swap(a, b);
 p[b] = a;
 if (rnk[a] == rnk[b]) rnk[a]++;
 return true;
}

sort(edges.begin(), edges.end()); // O(E log E)
long long cost = 0; int taken = 0;

```

а – другая задача.</p>

</div>

</details>

<details class="bg-slate-900/40 rounded-lg">

    <summary class="p-4 font-bold text-slate-300">

**-b border-slate-700/50">**

        Код 20: Боровка (Скрыто)

    </summary>

    <div class="p-5 text-sm text-slate-300">

        <pre class="bg-slate-950 p-4 rounded">

        = list(range(n))

def find(v):

    while parent[v] != v:

        parent[v] = parent[parent[v]]

        v = parent[v]

    return v

comps = n

total = 0

while comps > 1:

    best = {}

# компонента

    for w, u, v in edges:

        cu, cv = find(u), find(v)

        if cu == cv:

            continue

# уже внутри

        if cu not in best or w < best[cu][0]:

            best[cu] = (w, u, v)

        if cv not in best or w < best[cv][0]:

            best[cv] = (w, u, v)

    if not best:

        break

# граф несвя.

    for w, u, v in best.values():

        if find(u) == find(v):

            continue

# дубль: два

        parent[find(u)] = find(v)

        comps -= 1

        total += w

```
export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb">
 Мы идём по строке слева направо.
 строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слова
 x</code>. Тупой алгоритм начнет искать зано
 cab</code> суффикс <code>ab</code> – это ро
 буквы шаблона». Это поиск за O(N) без возв
 </p>
 </div>
 </div>
 `
 }
]
```

```

</details>
<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
</details>

```

```

<h3 class="text-lg font-bold text-white mt-4">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 p>
 </div>
 <p class="text-slate-300 text-sm mb-4">Префикс
 для шаблона (pattern) один раз, затем идём
 пало — $j += 1$. Не совпало — откатываемся на
 у заново сравнивать его не нужно. Когда j до
 ждения), делаем $j = \pi[j - 1]$.</p>
 <p class="text-slate-300 text-sm mb-4">П
 ждый откат j уменьшает. Значит суммарно отк
 классическая амортизационная оценка, её люб
 <div class="bg-slate-900/40 rounded-lg border-1
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 льшего собственного префикса, равного суффи
 <p class="text-slate-400 text-sm mb-1">
 ый алгоритм.</p>
 <p class="text-slate-400 text-sm mb-1">

```

```

тот же результат одной префикс-функцией: π от pattern
def kmp_via_concat(text, pattern):
 s = pattern + "#" + text
 pi = prefix_function(s)
 m = len(pattern)
 return [i - 2 * m for i in range(m + 1, len(s)) if pi[i] == m]

<p class="text-slate-400 text-xs">Результат префикс-функции для строки "abababab"
рез границу и вхождения будут ложными.</p>
</div>
</details>
</div>
</div>
</section>`
},
{
 id: "string-z-func",
 title: "22. Z-функция — Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Z-функция
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">22. Z-функция — Сдвиг и наложение
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">Определение
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Z-функция строки s — это массив z из целых чисел, где $z[i]$ — длина наибольшей общей префикса строки s с её подстрокой, начинающейся с i .
 <p class="text-xl font-mono text-white">Например, для строки "abababab" значения z будут: [0, 0, 4, 0, 4, 0, 4, 0].
 </div>
 <div class="bg-slate-800 p-4 rounded-lg">
 <div class="absolute -top-3 left-4">
 <div class="bg-slate-700/50 p-2 rounded">
 <div class="text-2xl font-bold text-emerald-400">Задание
 <p class="text-sm text-emerald-300">Найти Z-функцию для строки "abababab".
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
```

```

 <code>Z[3]</code> (abcd
а дальше s[6]='d' против s[3]='a' — мимо.
 <code>Z[4], Z[5], Z[6]</code>

</div>
</details>
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код C++ (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded

int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
</details>
</div>
</div>
</section>`
 },
 {
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 roun
 <h2 class="text-2xl sm:text-3xl font-bold text-white
 </div>

 <div class="space-y-8">

```

```

<pre class="bg-slate-950 p-4 rounded overfl
int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch)
 }
 return t[v].link;
}

```

```

int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v), t[v].pch);
 }
 return t[v].go[c];
}
</pre>

```

```

</div>
</details>
</div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description:
 "P, NP, co-NP, NP-трудные и NP-полные, PSPACE и E.
 ли задача NP-полна.",
 category: "Теория сложности",
 content: `

```

```

<section id="complexity-classes" class="mb-12 scroll-mt
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-purple-400

```

```

 <p class="font-mono text-sm sm:text-bas
 $P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$

 NP-н
доказано)
 </p>
</div>
<div class="overflow-x-auto">
 <table class="w-full text-xs text-left l
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-2 pr-3 font-b
 <th class="py-2 pr-3 font-b
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slat
 py-2 pr-3">детерминированно за $O(n^k)$ </td><td
удольности, максимальный поток</td></tr>
 <tr class="border-b border-slat
 py-2 pr-3">сертификат проверяется за полином
ешающая версия), коммивояжёр «есть ли тип $\leq$
 <tr class="border-b border-slat
 s="py-2 pr-3">сертификат есть у ответа «НЕТ:
ель)</td></tr>
 <tr class="border-b border-slat
 lass="py-2 pr-3">не проще любой задачи из NP
фа, TSP-оптимизация (не решающая версия)</td>
 <tr class="border-b border-slat
 ass="py-2 pr-3">NP-трудные И из NP</td><td
сумм</td></tr>
 <tr class="border-b border-slat
 "py-2 pr-3">полиномиальная память, время не
й на полиномиальном поле</td></tr>
 <tr class="align-top"><td class=
за $2^{(n^k)}$ </td><td class="py-2">обобщённые
 </tbody>
 </table>

```



```

 стной трудной A к B ($A \leq_p B$), а не н
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm mb-1">
 дит по времени на $n = 40$. NP-полнота запрещ
 <p class="text-slate-400 text-sm">•
 ит RSA. «Раз она в NP, значит NP-полная» —
 </div>

```

```

<p class="text-slate-300 text-sm mt-4"><b c
ass="bg-slate-950 px-2 py-0.5 rounded text-
еребор сертификатов <code class="bg-slate-9
^poly(n))</code>; сведение обязано работать
0 border border-slate-800">0(poly(n))</code>

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">

```

Код: верификатор сертификата и пр

```

</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded

```

ьтонов цикл NP-полон: сертификат = порядок

```

def verify_hamilton(n, edges, cert):
 if sorted(cert) != list(range(n)): # каждая
 return False
 allowed = {(u, v) for u, v in edges} | {(v, u) for
 for i in range(n):
 a, b = cert[i], cert[(i + 1) % n] # замык
 if (a, b) not in allowed:
 return False
 return True

```

# 3-SAT  $\leq_p$  КЛИКА: формула с  $k$  дизъюнктов  $\rightarrow$  граф из  $3k$  в

```

def reduce_3sat_to_clique(clauses):
 # вершина = (номер дизъюнкта, литерал); ребро, если
 verts = [(ci, lit) for ci, cl in enumerate(clauses)
 edges = []
 for i in range(len(verts)):

```



```

<div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = 0
</div>
<p class="text-slate-300 text-sm mb
 Что делает: Поиск в глуб
да попадает в тупик – делает шаг назад по с
</p>
<ul class="text-xs text-indigo-300
 Где используется:
 Топологическая сортировка (l
 Поиск мостов и точек сочлен
 Компоненты сильной связности
 Детектирование циклов (чере

</div>

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border border-emerald-500 shadow-md">
 BFS (Поиск в ширину) = Волна
</div>
<p class="text-slate-300 text-sm mb
 Что делает: Распространен
соседей. Работает через Очередь (FIFO). Гар
</p>
<ul class="text-xs text-emerald-300
 Где используется:
 Кратчайший путь в Невзвешенн
 Проверка двудольности графа
 Алгоритм Ахо-Корасик (постр
 Двунаправленный BFS для уск
йкстра).

</div>
</div>

```

```

{
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Планарные графы: формула Эйлера, K5 и K4",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы. Планарные. Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700/80">
 <h3 class="text-xl font-bold text-blue-400">Планарные графы
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Планарный граф — граф, который можно нарисовать в плоскости так, чтобы никакие две его
 <p class="text-2xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
 </div>

 <div class="space-y-4 text-slate-300 text-s">
 <p>Главные неравенства на число рёбер</p>
 <ul class="list-disc list-inside space-y-2">
 Для простого планарного графа
 <i>Доказательство:</i> Каждая грань
 . Подставляя в формулу Эйлера:

 Для простого двудольного планарного графа
 . Доказательство: В двудольном графе
 2E ≥ 4F ⇒ F ≤ E/2

 <div class="bg-slate-950 p-4 rounded-lg">
 <p class="text-amber-300 font-bold">
 <p class="text-slate-300 text-xs le

```

```


 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description:
 "Компоненты связности: обходами DFS/BFS за $O(V + E)$ ",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 8
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы. Компоненты связности
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Компоненты связности
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Определение:
 <p class="text-xl font-mono text-white">
 рой вершин есть путь. Внешний цикл «для каж-
 » даёт и разметку, и количество компонент.<
 </div>

 <p class="text-slate-300 text-sm mb-4">Два
 ной запускаем DFS или BFS и присваиваем все
 mp[v] = c. Счётчик c увеличивается р
 Время $O(V + E)$, память $O(V)$. Чере
 каждое ребро делаем union(u, v), в конце чи
 ту и после каждого добавления спрашиваю

 <div class="grid grid-cols-1 xl:grid-cols-2" style="margin-top: 20px;">
 <div class="bg-slate-800 p-6 rounded-lg" style="width: 100%;">

```

```

 <tr class="align-top"><td class="text-slate-400 text-xl font-mono text-white" rowspan="2">до кого дойдёт старт</td><td class="py-4 text-slate-400 text-xl font-mono text-white">
 V + E)</td></tr>
 </tbody>
 </table>
 </div>
 <p class="text-slate-400 text-xs mt-3">Классический алгоритм SCC. Обход из s даёт только вершины,
</div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Топологическая сортировка ДАГ и детект",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 9. Графы. Топологическая сортировка
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Топологическая сортировка
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-blue-400">Топологическая сортировка
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Топологическая сортировка — это алгоритм, который позволяет упорядочить вершины графа так, чтобы каждая вершина была помещена перед всеми вершинами, к которым она имеет directed edge.
 <p class="text-xl font-mono text-white">Аналогия 1: Учеба в вузе
 </div>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 </div>
 </div>
 </div>
</section>

```



```

 AG конденсации.
 Переворачиваем все ребра в
 Идем по <code>order</code>:
 тот запуск – образуют очередную SCC!

</div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых увеличивает ч",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord">
 <h3 class="text-xl font-bold text-rose-400 m">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic m">
 <p class="text-xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
 </div>

 <div class="bg-slate-950 p-4 rounded-lg bor">
 <p class="text-amber-300 font-bold text">
 <p class="text-slate-300 text-xs leadin">
 Проверка <span class="font-mono text-
 тиграфе между u и v может быть 2 параллельны
 азуют цикл длиной 2.


```



```

 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых увеличивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3"
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-rose-300 italic m
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Особый случай: Корень дерева DFS
 моего ребёнка в дереве DFS.
 </p>

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код поиска точек сочленения (Скры

```

```


 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-base sm:text-lg font-monospace">
ности!</p>
 </div>

 <div class="grid grid-cols-1 md:grid-cols-2">
 <!-- Неориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg">
 <p class="text-indigo-300 font-bold">
 <ul class="list-disc list-inside space-y-2">
 Эйлеров цикл: связан
-emerald-300">deg[v] % 2 == 0.
 Эйлеров путь: связан
о 2 – путь начинается в одной нечётной и за

 </div>

 <!-- Ориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg">
 <p class="text-indigo-300 font-bold">
 <ul class="list-disc list-inside space-y-2">
 Эйлеров цикл: слабос
ald-300">in[v] == out[v].
 Эйлеров путь: слабос
s] = 1 (старт), у одной <span class="
an class="font-mono text-emerald-300">in[v]

 </div>
 </div>

 <h3 class="text-lg font-bold text-white mt-4">

```

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points {3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
```

```
const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
};
```

recordStep("Начало DFS (Тарьян) для поиска точек со

```
const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}]`,
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
 v,
);
 }
 }
};
```

```

 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
 };

 useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
 }, [isPlaying, currentStepIndex, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"

```

```

 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : "1"}
 />
);
 }}}

```

```

{/* Nodes */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

```

```

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";

```

```

 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";

```

```

 const r = isAp ? 24 : 20;

```

```

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}

```

```

 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
 })
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Посещена
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Текущая

```

```
 </div>
 </div>
);
};
```

---

ФАЙЛ 32/111: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from './viz';
import { LiveDistTable } from './LiveDistTable';
```

```
interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 и листать вручную.";
```

```
export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);
```



```
const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start'
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0'
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations'
 { line: 4, text: ' for (u, v, weight) in E:'
 { line: 5, text: ' if dist[u] + weight < dist[v]'
 { line: 6, text: ' dist[v] = dist[u] + weight'
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл'
 { line: 8, text: ' if dist[u] + weight < dist[v]'
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ'"
 { line: 10, text: ' return dist' },
];
```

```
useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999'
 activeCodeLine: 2,
 });
```

```
const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }
 }
}
```

```

 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev }
 log: ' Проверка завершена. Ни одно ребро не
 activeCodeLine: 10,
 });
 }
 }

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
 }, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => setCurrentStepIndex((p)
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== nu
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 iteration: liveI ?? 0,
 checkingEdge: liveU && liveV ? { u: liveU, v: l

```

```
<LiveDistTable
 dist={liveDist}
 demoIds={nodes.map((node) => node.id)}
 title="Ваши расстояния из кода (Форд-Беллман)"
/>

<div className="flex flex-col lg:flex-row gap-6 m-0 auto w-full">
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-blue-950/20 p-6 rounded-lg shadow-sm">
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue-950/20 p-6 rounded-lg shadow-sm">
 Итерация {currentStep.iteration} (Шаг {currentStep.step})
 </div>

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-red-950/20 p-6 rounded-lg shadow-sm animate-pulse">
 О отрицательном цикле!
 </div>
)}

 <svg className="w-full h-auto max-h-[500px]">
 <defs>
 <marker id="arrow-bf-normal" markerWidth=100 markerHeight=100 viewBox="0 0 100 100">
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 </marker>
 <marker id="arrow-bf-active" markerWidth=100 markerHeight=100 viewBox="0 0 100 100">
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-bf-cycle" markerWidth=100 markerHeight=100 viewBox="0 0 100 100">
 <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </marker>
 </defs>

 <g>
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking = currentStep.checkingEdges.includes(idx);
 return (
 <div key={idx} className="relative h-0 w-0">
 <div style={{position:'absolute',left:-10px;top:0}}>{uNode.name}</div>
 <div style={{position:'absolute',right:-10px;bottom:0}}>{vNode.name}</div>
 <div style={{position:'absolute',left:0;right:0;bottom:0;transform:translateX(50%);height:10px;transform-origin:center bottom;}}>
 {isChecking ? <div style="width:100%;height:100%;background-color:#f59e0b;border-radius:50%;border:1px solid #e11d48;">

```

```

 }}}}
 </svg>

 <div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2 text-sm font-mono text-white">
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap gap-2">

 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === edge.v
 return (
 <span key={idx} className={`px-2 py-1 rounded-sm ${
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
 }`}>
 {edge.v} ({edge.w})

)
 })}
 </div>
 })}
 </div>
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10 p-4 rounded-2xl">
 <div>
 <h4 className="text-lg font-bold text-rose-400">
 <div className="grid grid-cols-4 gap-2 text-sm font-mono text-white">

```

-----  
ФАЙЛ 33/111: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

-----  
import { useState, useEffect, useCallback } from 'react'  
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
 6: []
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}
```

```
const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
```

```

const neighbors = ADJ[curr];
if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добав.
 });
 }
 }
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
}
}

steps.push({
 activeLine: 12,

```

```

<div className="flex flex-col md:flex-row items-c
 <div>
 <h3 className="text-2xl font-bold text-white
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Ви
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-bl
 title={isPlaying ? 'Пауза' : 'Воспроизведе
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-
 ed:hover:bg-transparent"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-2 g
 {/* Визуал графа */}
 <div className="bg-slate-800 border border-slat
 50px]">

```

```

 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors durati
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
 }}}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-sl
 er">
 <div className="bg-slate-800 text-slate-400
 se tracking-wider">
 bfs(graph, start)
 Mar {step
 </div>
 }

```



```
))
 })
 </div>
 <div className="mt-2 text-sm text-amber-300 text-center justify-center">
 "{step.logs}"
 </div>
 </div>
</div>
</div>
</div>
);
}
```

---

ФАЙЛ 34/111: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-freakazoid.png';
import bellmanImg from '../assets/bellman-ford-batman.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string; caption: string; borderColor: string; captionColor: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Декстер Дуглас и Фриказоид',
 caption: 'Декстер Дуглас и Фриказоид – волна вируса',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Бэтмен на машине Генри Форда',
 caption: 'Бэтмен на машине Форда – n-1 ночей патрулирует Gotham City',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Сеть Флойда',
 caption: 'Сеть Флойда – алгоритм поиска кратчайшего пути',
 borderColor: 'border-teal-500/30',
 captionColor: 'text-teal-400',
 },
};
```

```

 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
 }

/**
 * Переходы «предыдущая / следующая тема» – как на Code
 * Компактный вариант живёт в шапке главы, крупный – по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 }
};

```

ФАЙЛ 36/111: src/components/ChapterQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { ArrowRight, CheckCircle2, RotateCcw, XCircle, } from "lucide-react";
import { quizzes } from "../data/quizzes";
```

```
interface Props {
 chapterId: string;
}
```

```
/**
```

```
 * Блиц по прочитанной теме: 3–5 коротких вопросов с несколькими вариантами ответа
```

```
 * Вопросы живут в `src/data/quizzes.ts` (по id главы).
```

```
 * компонент-викторина существовали, но не были подключены к
```

```
 * контент был, а добраться до него было нельзя. Теперь
```

```
 * демонстрации: прочитал → посмотрел, как работает → протестировал
```

```
 * В текстовой версии (`?lite=1`) и в PDF те же вопросы
```

```
 * с ответами и разбором – там интерактив не нужен.
```

```
 */
```

```
export const ChapterQuiz: React.FC<Props> = ({ chapterId }) => {
 const list = quizzes[chapterId];
 const [idx, setIdx] = useState(0);
 const [picked, setPicked] = useState<number | null>(null);
 const [score, setScore] = useState(0);
 const [finished, setFinished] = useState(false);
```

```
 // другая тема – другой блиц с нуля
```

```
 useEffect(() => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 }, [chapterId]);
```

```

 {finished ? `итог ${score}/${list.length}`

 </div>

 {finished ? (
 <div className="text-center py-3">
 <p className="text-sm text-slate-300 mb-1">
 {score === list.length
 ? "Все верно: тему можно закрывать."
 : score * 2 >= list.length
 ? "Половина и больше верно – стоит пересмотреть."
 : "Стоит вернуться к тексту страницы: "
 }
 </p>
 <p className="text-2xl font-bold text-white">
 {score} / {list.length}
 </p>
 <button
 type="button"
 onClick={restart}
 className="inline-flex items-center gap-1
 font-bold text-indigo-200 hover:bg-indigo-500/20"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Прогресс
 </button>
 </div>
) : (
 <div>
 <p className="text-sm font-semibold text-white">
 <div className="space-y-2">
 {q.options.map((option, i) => {
 const isRight = i === q.correctIndex;
 const chosen = picked === i;
 const tone = !answered
 ? "border-slate-600 bg-slate-800/70 text-white"
 : isRight
 ? "border-emerald-500 bg-emerald-500/50 text-white"
 : chosen
 ? "border-rose-500 bg-rose-500/15 text-white"

```

```
 </button>
 </div>
 </>
 })
</div>
</section>
);
};

export default ChapterQuiz;
```

---

ФАЙЛ 37/111: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
```

```

* чтобы на бумагу попал весь текст (доказательства,
* а после печати возвращаем их в исходное состояние.
*/
useEffect(() => {
 const each = () => Array.from(contentRef.current?.q
 const openAll = () => {
 each().forEach((d) => {
 d.dataset.wasOpen = d.open ? "1" : "0";
 d.open = true;
 });
 };
 const restore = () => {
 each().forEach((d) => {
 if (d.dataset.wasOpen === "0") d.open = false;
 delete d.dataset.wasOpen;
 });
 };
 window.addEventListener("beforeprint", openAll);
 window.addEventListener("afterprint", restore);
 return () => {
 window.removeEventListener("beforeprint", openAll);
 window.removeEventListener("afterprint", restore);
 };
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {

```

```

};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-1.5">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-small border border-gray-500 p-3 300px hover:text-white hover:border-emerald-500" />
 >
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 </button>
 <button
 onClick={() => window.print()}
 title="Распечатать или сохранить эту тему в PDF-формате (без шапки, содержания и панели комментариев)"
 className="flex items-center gap-1.5 text-x-small border border-gray-500 p-3 300px hover:text-white hover:border-indigo-500" />
 >
 <Printer className="w-3.5 h-3.5" /> PDF этой страницы
 </button>
 <a
 href={`?lite=1&topic=${encodeURIComponent(chapter.id)}`}
 target="_blank"
 rel="noreferrer"
 >

```





## Универсальное пособие • полный исходный код

```

const TABS: { id: Tab; label: string; icon: React.ReactElement } = [
 { id: "hierarchy", label: "1 · Иерархия классов", icon: <IconHierarchy /> },
 { id: "reduce", label: "2 · Сведение A $\leq_p$ B", icon: <IconReduce /> },
 { id: "verify", label: "3 · Верификатор (NP)", icon: <IconVerify /> },
 { id: "growth", label: "4 · Рост по n и ops", icon: <IconGrowth /> },
];
```

```
/* ----- Иерархия -----
```

```
const CLASSES = [
 { id: "EXP", label: "EXP", sub: "2(nk)", color: "#0000FF",
 {кванторные формулы}"}],
 { id: "PSPACE", label: "PSPACE", sub: "полиномиальная
 с полной информацией"}],
 { id: "NP", label: "NP", sub: "сертификат проверяется
 AT", "гамильтонов цикл", "раскраска в 3 цвета", "клика"},
 { id: "P", label: "P", sub: "решается за O(nk)", color: "#00FF00",
 {ид", "MST", "2-SAT, двудольность"}],
];
```

```
const NPHARD_NOTE = "NP-трудные – всё, к чему сводится
 амильтонов цикл. Проблема остановки NP-трудна, но в NP".
```

```
/* ----- Сведение -----
```

```
const REDUCE_STEPS = [
 { title: "Дано: задача A, для которой быстрого алгоритма нет
 – то есть B NP-трудная.", boxes: ["A"], active: 0 },
 { title: "Берём вход x задачи A", note: "x – конкретный экземпляр
 "x"]}, active: 1 },
 { title: "Строим f(x) = y за полиномиальное время", note: "y строится
 шинами из разных дизъюнктов, если литералы не противоречат",
 ve: 2 },
 { title: "Решаем B на входе y", note: "Гипотетический алгоритм B
 – но допустим, что он есть.", boxes: ["A", "x", "f(x)"]}, active: 3 },
 { title: "Переводим ответ обратно: A(x) = B(f(x))", note: "Если B(y)
 $\Rightarrow$ формула выполнима. Полиномиальное время сохранилось",
 5 },
];
```

```

const GROWTH_LINES = [
 { id: "n", label: "n", fn: (n: number) => n, color: "#000000" },
 { id: "n log n", label: "n·log2 n", fn: (n: number) => n * Math.log2(n), color: "#000000" },
 { id: "n2", label: "n2", fn: (n: number) => n * n, color: "#000000" },
 { id: "n3", label: "n3", fn: (n: number) => n * n * n, color: "#000000" },
 { id: "2n", label: "2n", fn: (n: number) => Math.pow(2, n), color: "#000000" },
];

export const ComplexityViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("hierarchy");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [samples, setSamples] = useState<Sample[]>([]);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

 const maxStep = tab === "reduce" ? REDUCE_STEPS.length : 0;
 useEffect(() => {
 setStep(0);
 setPlaying(false);
 }, [tab]);

 useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1400);
 }, [maxStep, playing]);
}

```

```

let den = 0;
for (let i = 0; i < xs.length; i++) {
 num += (xs[i] - mx) * (ys[i] - my);
 den += (xs[i] - mx) ** 2;
}
const slope = den === 0 ? 0 : num / den;
// экспонента на лог-лог растёт неограниченно: сравн
const half = Math.floor(pts.length / 2);
const slopeTail = (() => {
 const q = pts.slice(half);
 if (q.length < 2) return slope;
 const lx = q.map((p) => Math.log(p.n));
 const ly = q.map((p) => Math.log(p.ops));
 const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
 const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
 let nn = 0;
 let dd = 0;
 for (let i = 0; i < lx.length; i++) {
 nn += (lx[i] - ax) * (ly[i] - ay);
 dd += (lx[i] - ax) ** 2;
 }
 return dd === 0 ? slope : nn / dd;
})();
const cls =
 slopeTail > slope + 0.8 || slopeTail > 4.5
 ? { name: "экспонента / суперполином", color: "#f59e0b",
 : slope <= 1.25
 ? { name: "≈ линейный, $O(n)$ ", color: "#10b981",
 : slope <= 1.8
 ? { name: "≈ $O(n \log n)$ ", color: "#06b6d4", hint: "log"
 : slope <= 2.4
 ? { name: "≈ квадратичный, $O(n^2)$ ", color: "#f59e0b",
 : slope <= 3.4
 ? { name: "≈ кубический, $O(n^3)$ ", color: "#fb9238",
 : { name: "полином высокой степени", color: "#fb9238"
 : slope > 4.5
 };
return { slope, slopeTail, ...cls };
}, [samples]);

```

```

 lg bg-slate-900 border border-slate-700 text-slate-500
 /></button>
 <button onClick={() => setPlaying((p) => !p)}
 text-xs font-bold hover:bg-purple-500">
 {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 </button>
 <button onClick={() => { setPlaying(false); setVolume(0.5); }}
 text-xs font-bold rounded-lg bg-slate-900 border border-slate-700
 hover:bg-slate-800 hover:shadow-lg e="w-4 h-4" /></button>
 {volume}
</div>
})

```

```

{/* ----- ИЕРАРХИЯ ----- */}
{tab === "hierarchy" && (
 <div className="space-y-4">
 <div className="bg-slate-950 rounded-xl p-3 border border-slate-800">
 <svg viewBox="0 0 500 380" className="w-full h-380">
 {CLASSES.map((c) => {
 const on = liveCls ? liveCls.toUpperCamelCase(c.id) : null
 return (
 <g key={c.id}>
 <ellipse cx={c.x} cy={c.y} rx={c.rx} ry={c.ry}
 {on ? 3 : 1.5} strokeDasharray={on ? undefined : [4, 4]}
 fill={on ? "#e2e8f0" : "#334155"} stroke="black" />
 <text x={c.x} y={c.y - c.ry + 18} textAnchor="middle">{c.name}</text>
 <text x={c.x} y={c.y - c.ry + 32} textAnchor="middle">{on}</text>
 </g>
)
 })}
 </svg>
 </div>
 <div>
 <h3>Примеры задач – внутри своих овалов</h3>
 {CLASSES.map((c) => {
 c.tasks.map((t, i) => {
 <text key={t} x={c.x} y={c.y + (c.id === "root" ? 10 : 20) + i * 15}
 textAnchor="middle" fill="#e2e8f0" fontSize="8">
 {t}
 </text>
 })
 })}
 </div>
 </div>
)}

```

```

 }}}
 </div>
</div>
<div className="bg-slate-900/70 border border-
 <p className="text-sm font-bold text-white
 text-purple-400" />{reduceFrame.title}</p>
 <p className="text-xs text-slate-400 leading-
</div>
</div>
)}

{/* ----- ВЕРИФИКАТОР -----
{tab === "verify" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <p className="text-[10px] uppercase tracking-
 <div className="flex flex-wrap gap-2 mb-3">
 {CERT.map((v, i) => {
 <span key={i} className={"font-mono tex
 ld-600/50 text-emerald-300" : "bg-slate-900
 x{i + 1} = {liveCert && typeof liveCe

)}}
 {liveCert && <span className="font-mono t
 le-950/40">cert из кода}
 </div>
 <div className="space-y-2">
 {CLAUSES.map((cl, i) => {
 const checked = liveK !== null ? i < li
 const current = liveK !== null ? i ===
 const st = VERIFY_STEPS[i];
 return (
 <div
 key={i}
 className={"flex items-center gap-3
 amber-950/20" : checked ? "border-
 slate-800 bg-slate-900/40"}}
 >

```

```

e0ps ?? "-"}
```

```

 <button onClick={() => setSamples([])} className="rounded-lg px-2 py-1">очистить график</button>
</div>
<svg viewBox="0 0 500 240" className="w-full h-40">
 {/* оси в лог-лог */}
 <line x1={44} y1={200} x2={486} y2={200} />
 <line x1={44} y1={14} x2={44} y2={200} stroke="black" />
 <text x={265} y={222} textAnchor="middle">log</text>
 <text x={16} y={110} fill="#64748b" fontStyle="italic">GROWTH</text>
 {GROWTH_LINES.map((g) => {
 const pts: string[] = [];
 for (let i = 0; i <= 20; i++) {
 const n = 1 + i * 0.8;
 const v = g.fn(n);
 const x = 44 + (Math.log(n) / Math.log(2)) * 442;
 const y = 200 - (Math.log(Math.max(1, v)) / Math.log(2)) * 196;
 pts.push(`${x.toFixed(1)},${Math.max(1, v).toFixed(1)}`);
 }
 return <polyline key={g.id} points={pts} stroke="black" strokeOpacity={0.55} />;
 })}
 {samples.map((s) => {
 const x = 44 + (Math.log(Math.max(1, s.n)) / Math.log(2)) * 442;
 const y = 200 - (Math.log(Math.max(1, s.v)) / Math.log(2)) * 196;
 return <circle key={`${s.n}`} cx={x} cy={y} r={10} />;
 })}
 {GROWTH_LINES.map((g, i) => (
 <g key={g.id + "l"}>
 <rect x={330 + (i % 3) * 58} y={16 + i * 8} width={58} height={8} />
 <text x={343 + (i % 3) * 58} y={24 + i * 8} text="GROWTH" />
 </g>
))}
</svg>
{samples.length === 0 && (
 <p className="text-xs text-slate-500 text-center">Пусто. Запусти в компиляторе код с переменной GROWTH_LINES</p>
)}

```

```
 }}
 </div>
</div>
});
};

export default ComplexityViz;
```

---

ФАЙЛ 39/111: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "react-icons/bs";
```

```
interface CodeLine {
 line: string;
 highlight: string;
}
```

```
const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
```

```

 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3.",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 }
];

```



```

 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мост,
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост,
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост,
 activeCodeLine: [13,14,15,16,17]
},
{
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост,
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм
 activeCodeLine: [11,13,16,17,18]
},
{
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <div className="flex items-center gap-1 bg-slate-700 rounded">
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === 0}
 aria-label="Шаг назад" title="Шаг назад"
 className="p-1.5 hover:bg-slate-700 rounded"
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px]
 dfsAuto ? "bg-rose-600 text-white" : "bg-
 }`>
 {dfsAuto ? <><Pause className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === DFS_STEPS.length-1}
 aria-label="Шаг вперёд" title="Шаг вперёд"
 className="p-1.5 hover:bg-slate-700 rounded"
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 aria-label="Сбросить DFS" title="Сбросить D
 className="p-1.5 hover:bg-slate-700 rounded"
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>
 </div>

 <div className="grid grid-cols-1 md:grid-cols-5 g
 { /* Graph visualization */ }
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relativ

```

```

 label}</text>
 {vis && (
 <>
 <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor:
} l:{step.low[n.id]||"?"}</text>
 </>
)}
 </g>;
 }}}
</svg>
<div className="flex items-center justify-betw
 Mar {dfsIdx+1}/{DFS_STEPS.length}</sp
 <div className="flex-1 mx-2 bg-slate-950 h-
 <div className="bg-indigo-600 h-full tran
 </div>
 </div>
</div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg borde
 <div className="bg-slate-900 px-3 py-1.5 te
 >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono lea
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includ
 return <div key={i}
 className={`px-1 transition-all dur
 hl ? "bg-indigo-600/20 text-indig
 }`}>
 <span className="text-slate-600 mr-1
 {cl.line || ' '}
 </div>;

```

```
 </div>
 </div>
);
 }
}
```

---

ФАЙЛ 40/111: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 и листать вручную.";

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false };
 const prev: Record<string, string | null> = { A: null };

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пиратского
 activeCodeLine: 2,
 });

 for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещёнными
 activeCodeLine: 5,
 });
 }
});

```

```

 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
 }, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg</div>;
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(vars?.done);
const compilerLinked = liveU !== null || liveV !== null;
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
 distances: Object.fromEntries(nodes.map((node) => [node, 0])),
 previous: Object.fromEntries(nodes.map((node) => [node, null])),
 visited: Object.fromEntries(nodes.map((node) => [node, false])),
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}, dist=${liveDist}, prev=${livePrev}, done=${liveDone}`
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 { /* Шапка симулятора */ }
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white rounded">

```

```

 </div>
 </div>

 <LiveDistTable
 dist={liveDist}
 prev={livePrev}
 done={liveDone}
 demoIds={nodes.map((node) => node.id)}
 title="Ваши расстояния из кода (Дейкстра)"
 />

 <div className="flex flex-col lg:flex-row gap-6 md:gap-10" >
 {/** Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/20 p-6 rounded-2xl flex flex-direction column align-items-center justify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indigo-950/20 p-3 font-bold shadow-sm">
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-dh-normal" markerWidth="10" markerHeight="10" fill="#475568" >
 <path d="M0,0 L6,3 L0,6 z" fill="#475568" />
 </marker>
 <marker id="arrow-dh-active" markerWidth="10" markerHeight="10" fill="#f59e00" >
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e00" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6" markerHeight="6" fill="#10b98f" >
 <path d="M0,0 L6,3 L0,6 z" fill="#10b98f" />
 </marker>
 </defs>
 {/** Рисуем */}
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking =
 currentStep.checkingEdge === edge.id
 })}
 </svg>
 </div>
 </div>

```

```

const isVisited = currentStep.visited[node.id]
const dist = currentStep.distances[node.id]

return (
 <g key={node.id} className="transition-
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#f1c40f' : '#f1c40f'}
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f1c40f' : '#f1c40f'}
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="white"
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white"
 "middle" className="bg-indigo-950 px-1 py-0"
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill="white"
 {node.name.split(' ')[0]}
 </text>
 </g>
);
}}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded"
 <p className="font-semibold text-amber-400"
 <p className="min-h-[40px] flex items-center"
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {

```



```

<div>
 <h4 className="text-lg font-bold text-rose-900">
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-900">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред. шаг)</th>
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-b border-rose-900 ${
 isCurr ? 'bg-indigo-950/60 font-weight-bold' : ''
 }`} >
 <td className="py-2.5 px-3 flex items-center">

 {n.name}
 </td>
 <td className="py-2.5 px-3 font-weight-bold">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-rose-900">
 {prev || '-'}
 </td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>

```

```

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' |
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',

```

```

 const res1 = simulateFib(n - 1);
 const res2 = simulateFib(n - 2);
 memoMap[n] = res1 + res2;

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: memo[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
 }

 const finalRes = simulateFib(targetN);

 generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 }
}, [isPlaying, currentStepIndex, speed]);

```

Смотри, как таблица DP кэширует результаты

</p>

</div>

<div className="flex flex-wrap items-center gap-

<div className="flex items-center gap-2 bg-sl

<span>N:</span>

<select

value={targetN}

onChange={(e) => setTargetN(Number(e.targ

disabled={isPlaying}

className="bg-transparent text-white font

>

<option value={4}>N = 4</option>

<option value={5}>N = 5</option>

<option value={6}>N = 6</option>

<option value={7}>N = 7</option>

</select>

</div>

<div className="flex items-center bg-slate-95

<button

onClick={() => { setIsPlaying(false); set

className="p-2 text-slate-400 hover:text-v

title="Сбросить"

>

<RotateCcw className="w-4 h-4" />

</button>

<button

onClick={() => { setIsPlaying(false); set

disabled={currentStepIndex === 0}

className="p-2 text-slate-400 hover:text-v

title="Шаг назад"

>

<ChevronLeft className="w-5 h-5" />

</button>

<button

onClick={() => setIsPlaying(!isPlaying)}

```

 : 'bg-slate-900/50 border-slate-800
 }` }
 >
 <span className="text-xs text-slate-400
 <span className={`text-xl md:text-2xl f
 {isCached ? val : '0'}

 {num <= 2 && <span className="text-[9px
 </div>
 });
 }}}}
</div>

{currentStep && (
 <div className="mt-6 pt-4 border-t border-sla
 <div className="text-sm font-medium text-sla
 <span className="inline-block w-2 h-2 rou
 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-
 Текущий N: <strong className="text-emeral
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }` }
 >
 <Eye className="w-4 h-4" />
 Степ-бай-степ состояние

```

```

 </div>
 <pre className="p-6 font-mono text-sm space
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return {
 <div
 key={lineNum}
 className={`flex items-center px-4 py-2
 ${isActive
 ? 'bg-emerald-600/20 border-l-4 border-emerald-600'
 : 'text-slate-300 hover:bg-slate-800'}
 `}
 >
 {lineNum}
 {line}
 </div>
);
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-l-4 border-emerald-600">
 {currentStep?.desc}
 </div>
 </div>
)}
</div>

```

```

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800">
 <div className="flex items-center justify-between">
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-emerald-600"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 >

```

```

*
* Две части:
* 1. игра «пройди все рёбра» кликами (Эйлер не прощает
* 2. критерий: степени вершин и число нечётных (0 → ц
*
* Синхронизация с Python-компилятором (PAGE_SYNC["eule
* path – маршрут (массив вершин) рисуется поверх графа
* пройдены, deg/odd – степени и нечётные вершины, las
* step – номер шага.
*/
export function EulerSimulator() {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину
 const [c1Done, setC1Done] = useState(false);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "walk");
 }, [chapterId]);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v
 if (!edge) {

```

```

 const val = degRec[String(n.id)] ?? degRec[n.id]
 const num = typeof val === "number" ? val : val.toString()
 return num ?? DEG[n.id];
 })
 : null;
const oddRaw = vizArray(vars.odd);
const odd = oddRaw ? oddRaw.map((x) => (typeof x === "number" ? x : x.toString())) : null;
const last = vizNumber(vars.last) ?? vizNumber(vars.last.toString());
const step = vizNumber(vars.step) ?? vizNumber(vars.step.toString());
if (path.length === 0 && usedFlags.length === 0 && !vars.complete) {
 // рёбра маршрута: пары соседних вершин
 const keys: string[] = [];
 for (let i = 0; i + 1 < path.length; i++) keys.push(`${path[i]} ${path[i + 1]}`);
 usedFlags.forEach((f, i) => {
 if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i]));
 });
 return {
 path,
 keys: Array.from(new Set(keys)),
 deg,
 odd,
 last: last !== null && last >= 0 && last < C1_NODES.length ? last : null,
 step,
 complete: keys.length >= C1_EDGES.length,
 };
}, [vars]));

const path = live && live.path.length ? live.path : c1Path;
const visitedKeys = live ? live.keys : c1VisitedEdges;
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[path.length - 1] : null;
const done = live ? live.complete : c1Done && visitedKeys.length >= C1_EDGES.length;
const msg = live
 ? live.complete
 ? `Компилятор прошёл все ${C1_EDGES.length} рёбра графа. Вершин ${odd.length} → это ${odd.length === 0 ? "цикл" : "маршрут"}.`
 : `Маршрут из кода: ${live.path.length ? live.path : c1Path}`;

```



```

return <g key={i}>
 <line x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={isLastEdge ? "#a5b4fc" : used ?
 strokeWidth={isLastEdge ? 5 : used ? 4
 className="transition-all duration-300"
 {order !== null && (
 <text x={({u.x + v.x} / 2} y={({u.y + v.y}
 }</text>
)}
 </g>;
}})
{C1_NODES.map(n => {
 const isLast = lastId === n.id;
 const visited = path.includes(n.id);
 const isOdd = odd.includes(n.id);
 return <g key={n.id} className="cursor-point
 {isLast && <circle cx={n.x} cy={n.y} r={11}
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200
 <text x={n.x} y={n.y + 4} textAnchor="mid
 label}</text>
 <text x={n.x} y={n.y - 16} textAnchor="mi
 one">deg {deg[n.id]}{isOdd ? " · нечёт" : "
 </g>;
 })}
</svg>
<div className="absolute top-2 left-2 bg-slate-
 Pe6ëp: {visitedKeys.length}/{C1_EDGES.length}
</div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 done && visitedKeys.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 te
 : c1Done && !live

```

```
 });
}
```

---

ФАЙЛ 43/111: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { liveMatrix, pickLiveVar, useVizRuntime, vizNums
```

```
interface Step {
 k: number; i: number; j: number;
 matrix: (number | null)[][]; updated: boolean;
 log: string; activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
ва листать вручную.";
```

```
export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 // k/i/j/d приходят напрямую; номер строки произвольн
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
```

```
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
```

```

k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
log: ' Инициализация матрицы прямых путей, ∞ (99
activeCodeLine: 2,
});

const N = 7;
for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через пром
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 &
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]), u
 log: ` Улучшение пути [${i}]→[${j}] чере
 ∞ '.`,
 activeCodeLine: 7,
 });
 }
 }
 }

 simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
 log: ' Все пары отработаны. Алгоритм Флойда завер
 activeCodeLine: 8,
 });
}

```

```

const sizeMismatch = compilerLinked && liveMat && (liveMat.size !== demoSize)
 ? `Демо-схема графа нарисована для ${demoSize} вершин. Если количество вершин не совпадает с количеством вершинности; схема графа остаётся демонстрационной.`
 : null;
const labelOf = (idx: number) => nodeNames[idx]?.split(' ')?.[0];

const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 k: liveK ?? -1,
 i: liveI ?? -1,
 j: liveJ ?? -1,
 matrix: shownMatrix,
 updated: !!runtime?.changed?.some(({name}) => name === 'matrix'),
 log: `Компилятор: k=${liveK ?? "-"}, i=${liveI ?? "-"}, j=${liveJ ?? "-"}\nПодсветка взята из реальных переносов` : "",
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-4">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-md">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Получить матрицу
 </p>
 </div>
 </div>
 <div>
 <div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false); TEXT : "Вернуться к первому шагу" } } className="text-sm text-emerald-300 hover:text-white cursor-not-allowed">Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying)} className="text-sm text-emerald-300 hover:text-white cursor-not-allowed">Начать новую демонстрацию</button>
 </div>
 </div>
 </div>
 </div>
);

```

```

 markerEnd={marker} strokeDasharray={isTarget
 <circle cx={({uNode.x + vNode.x) / 2}
 isVia ? '#818cf8' : '#64748b'} strokeWidth={
 <text x={({uNode.x + vNode.x) / 2} y={
 8fafc'} fontSize="10" fontWeight="bold" text={
 </g>
 });
 });
}}}
{nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a8a8a8'; }
 else if (isI) { fill = '#b45309'; stroke = '#a8a8a8'; }
 else if (isJ) { fill = '#047857'; stroke = '#a8a8a8'; }
 return (
 <g key={node.id} className="transition-colors">
 <circle cx={node.x} cy={node.y} r={isTarget
 sJ ? 4 : 2} className="transition-colors duration-200">
 <text x={node.x} y={node.y + 5} fill="white">{node.k}</text>
 <text x={node.x} y={node.y + 32} fill="white">{node.i}</text>
 </g>
);
})}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">Список смежности</p>
 <p className="min-h-[40px] flex items-center"></p>
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50">
 <h4 className="text-lg font-bold text-emerald-400">Список смежности</h4>

```

```

 {sizeMismatch}
 </p>
 } : null}
 <div className="overflow-x-auto">
 <table className="w-full text-center border">
 <thead><tr className="bg-rose-950/40 text-white">
 <th className="p-2 border-r border-rose-950/40">
 {Array.from({ length: cols }, (_, idx) => {
 <th key={idx} className={`p-2 border-rose-950/40`}>
 {labelOf(idx)} {idx}</th>
 </th>
 </th>
 </thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => {
 <tr key={i} className="border-b border-rose-950/40">
 <td className={`p-2 bg-rose-950/40 text-white`}>
 {currentStep.k === i ? 'text-amber-400' : ''}</td>
 {labelOf(i)} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i && currentStep.k !== j
 const isVia = currentStep.k !== -1 && currentStep.j === j);
 let bg = 'bg-slate-900/40 text-slate-400';
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-400';
 else if (isVia) bg = 'bg-indigo-950/60 text-indigo-400';
 else if (currentStep.i === -1 && currentStep.j === -1)
 bg = 'bg-indigo-950/60 text-indigo-400';
 else if (i === j) bg = 'bg-slate-900/40 text-slate-400';
 const shown = val === null ? '' : val;
 return (<td key={j} className={`p-2 ${bg}`}>
 {shown}
 </td>
 </td>
 </tr>
 </tr>
 </tbody>
 </table>
 </div>
 </div>

```

```

* comps уменьшается.
*
* Синхронизация с Python-компилятором (PAGE_SYNC["graph"])
* - comp / color / used – массив разметки: вершины к
* - parent / p – массив DSU: рисуем множества
* - v – текущая вершина (пульсирует)
* - stack / dq / q – фронт обхода (подсвечивает)
* - comps / c – счётчик компонент (большая)
* - i / e / step – номер шага: двигает по кадру
*/

```

```

type Mode = "dfs" | "dsu";

```

```

interface Node {
 id: number;
 label: string;
 x: number;
 y: number;
}

```

/\*\* Граф из 10 вершин и трёх компонент – ровно как в те

```

const NODES: Node[] = [
 { id: 0, label: "0", x: 90, y: 80 },
 { id: 1, label: "1", x: 175, y: 45 },
 { id: 2, label: "2", x: 205, y: 140 },
 { id: 3, label: "3", x: 105, y: 175 },
 { id: 4, label: "4", x: 330, y: 70 },
 { id: 5, label: "5", x: 420, y: 45 },
 { id: 6, label: "6", x: 380, y: 145 },
 { id: 7, label: "7", x: 130, y: 290 },
 { id: 8, label: "8", x: 230, y: 320 },
 { id: 9, label: "9", x: 330, y: 275 },
];

```

```

const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
 [4, 5], [5, 6], [4, 6], // компонента
 [7, 8], [8, 9], // компонента
];

```

```

 push(start, stack, c, `Новый запуск: c = ${c}, стар
 цвет ${c} и кладём в стек.`);
 while (stack.length) {
 const v = stack[stack.length - 1];
 const next = ADJ[v].find((to) => comp[to] === 0);
 if (next === undefined) {
 stack.pop();
 push(v, stack, c, `Из ${v} идти некуда – возвра
 ;
 continue;
 }
 comp[next] = c;
 stack.push(next);
 push(next, stack, c, `${v} → ${next}: красим в цв
 }, кладём ${next} в стек.`, [v, next]));
 }
 push(null, [], c, `Компонента ${c} готова`, `Стек п
 } вершин. Ищем следующую серую вершину.`);
 }
 push(null, [], c, "Готово: компонент = " + c, `Внешни
 вязности.`);
 return frames;
}

```

```

interface DsuFrame {
 title: string;
 note: string;
 parent: number[];
 comps: number;
 edge: [number, number] | null;
 merged: boolean;
}

```

```

function buildDsuFrames(): DsuFrame[] {
 const frames: DsuFrame[] = [];
 const parent = NODES.map((n) => n.id);
 const find = (v: number): number => (parent[v] === v
 let comps = NODES.length;

```



```

const runtime = useVizRuntime();
const [mode, setMode] = useState<Mode>("dfs");
const [step, setStep] = useState(0);
const [playing, setPlaying] = useState(false);
const timer = useRef<number | null>(null);

const frames = mode === "dfs" ? DFS_FRAMES : DSU_FRAMES;
const maxStep = frames.length - 1;
const frame = frames[Math.min(step, maxStep)] as DfsFrame;

/* — вкладка сообщает компилятору, какой код подстав. */
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode);
}, [chapterId, mode]);

/* — автопроигрывание — */
useEffect(() => {
 if (!playing) return;
 timer.current = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 900);
 return () => {
 if (timer.current) window.clearInterval(timer.current);
 };
}, [playing, maxStep]);

const goTo = useCallback((n: number) => setStep(Math.min(n, maxStep)));

/* — переменные компилятора, которые рисуются НАПРЯМУЮ */
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя рендеризация */
const liveComp = useMemo(() => {

```

```

const compOf = (id: number): number => {
 if (liveComp && liveComp[id] !== undefined && liveComp[id] !== 0) return liveComp[id];
 if (parentNow) return setOf(parentNow, id);
 return frame.comp?.[id] ?? 0;
};

const frontier = liveFrontier ?? (mode === "dfs" ? {
const cur = liveCur ?? frame.cur;
const comps = liveComps ?? frame.comps;
const edge = frame.edge;
const title = runtime && vars ? `${frame.title} • Контур` : "Контур";

const switchMode = (m: Mode) => {
 setMode(m);
 setStep(0);
 setPlaying(false);
};

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-3xl">
 <Network className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold">Контур
 <p className="text-xs text-emerald-300">10 минут
 </div>
 </div>
 <div className="flex items-center bg-slate-950 p-2.5 rounded-3xl">
 <button
 onClick={() => switchMode("dfs")}
 className={"flex items-center gap-1.5 px-3 py-2 bg-slate-800/90 text-white shadow" : "text-slate-400 hover:text-white"}
 >
 <Layers className="w-3.5 h-3.5" /> 06ход (DFS)
 </button>
 </div>
 </div>
 </div>
);

```

```

 { /* поле */ }
 <div className="bg-slate-950 rounded-xl p-3 border"
 <div className="absolute inset-0 bg-[radial-gradient(
 <svg viewBox="0 0 500 370" className="w-full h-full"
 {EDGES.map(([u, v], i) => {
 const a = NODES[u];
 const b = NODES[v];
 const active = edge && ((edge[0] === u && edge[1] === v) || (edge[0] === v && edge[1] === u));
 const sameComp = compOf(u) > 0 && compOf(u) === compOf(v);
 return {
 <line
 key={i}
 x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 stroke={active ? "#fbbf24" : sameComp ? "#fbbf24" : "#fbbf24"}
 strokeWidth={active ? 4 : sameComp ? 2 : 1}
 strokeDasharray={sameComp ? undefined : [4, 4]}
 className="transition-all duration-300"
 opacity={sameComp || active ? 1 : 0.55}
 />
 };
 })}
 {NODES.map((n) => {
 const c = compOf(n.id);
 const color = c > 0 ? COLORS[(c - 1) % COLORS.length] : "#fbbf24";
 const isCur = cur === n.id;
 const inFrontier = frontier.includes(n.id);
 return {
 <g key={n.id} onClick={() => goTo(step)}
 {isCur && <circle cx={n.x} cy={n.y} r={13} fill={color} />
 {inFrontier && !isCur && <circle cx={n.x} cy={n.y} r={13} fill="none" stroke={color} />
 <circle cx={n.x} cy={n.y} r={13} fill={color} />
 <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
 {c > 0 && <text x={n.x} y={n.y - 19} textAnchor="middle">c{c}</text>}
 </g>
 };
 })}
 </div>
 </div>
);
}

```

```

 <span className="text-[10px] uppercase tracking
 {vars ? {
 ["v", "comps", "c", "i", "e", "comp", "parent
 vars[name] === undefined ? null : {
 <span key={name} className="text-[11px] fo
 py-1">
 {name} = {JSON.stringify(vars[name]).sl

 }
 }
 } : {

 открой панель Python – переменные <code cla
 erald-400">parent</code>, <code className="
 comps</code> покрасят этот граф сами

 })
</div>
</div>
);
};

```

```
export default GraphComponentsViz;
```

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...
import { Play, Pause, SkipForward, Undo, RefreshCw } from

```

```

type Node = { id: string; label: string; x: number; y:
type Edge = { u: string; v: string };

```

```
const nodes: Node[] = [
```

```

function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зашли в узел ${v}`, visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Обработка узла ${v}`, stack: [...stack], visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Вышли из узла ${v}`, ueOrStack: [...stack], visitedNodes: Array.from(vis) });
}

dfs(start);
return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начали с узла ${start}`, visitedNodes: Array.from(vis) });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Обработка узла ${v}`, q: [...q], visitedNodes: [...Array.from(vis)] });
 q.shift();
 }
}

```

```

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-700 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>
 </div>
);

```

```

 if (current && step.type !== 'edge') {
 strokeColor = "stroke-amber-500";
 textColor = "fill-amber-500";
 radius = 22;
 }

 return (
 <g key={n.id} className="graph-node">
 <circle cx={n.x} cy={n.y}
 className={`stroke-${strokeColor} fill-${fillColor} stroke-width=2`}
 />
 <text x={n.x} y={n.y}
 className={`text-${textColor} font-size=14`}
 {n.label}
 />
 {
 /* Ripple for BFS */
 current && mode === 'bfs' && n === current ? (
 <circle cx={n.x} cy={n.y}
 className="stroke-amber-500 stroke-width=4"
 />
) : null
 }
 />
)
)
 }
 }
 </svg>
</div>

/* Structure View (Stack / Queue) */
<div className="bg-slate-900 border border-slate-700 p-10" >
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'border border-emerald-500' : 'border border-slate-700'}
 ${mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 `}>
 {step.queueOrStack && step.queueOrStack.length > 0 ? (
 <div key={idx} className="w-full text-white text-left p-2">
 {step.queueOrStack[idx]}
 </div>
) : null}
 </div>
 <div className="flex flex-col gap-2 mt-10">
 {step.queueOrStack && step.queueOrStack.length > 0 ? (
 <div key={idx} className="w-full text-white text-left p-2">
 {step.queueOrStack[idx]}
 </div>
) : null}
 </div>
</div>

```

```
 <RefreshCw strokeWidth={3} size={20} />
 </button>
 </div>

 { /* Description Log */ }
 <div className="flex-1 bg-slate-900/50 p-4 rounded-md shadow-sm
ow-hidden">
 <div className="text-[10px] text-slate-400">
 {steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
 </div>
</div>
);
}
```

---

ФАЙЛ 46/111: src/components/HearViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../hooks';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```



```

 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐱' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩸' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐱' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩸' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();

let uid = 300;

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const runtime = useVizRuntime();

```

```

 const finalSymptom = preset.symptom;

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 };

 const nextHeap = heapInsert(heap, newPatient).map(x => {
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === preset.priority);
 const marked = nextHeap.map(x => x.id === (target?.id));

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

 const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
 };

 const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
 };

```

```

 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);
}, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
 setLog([' База данных скорой помощи перезагружена.']);
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 }))), 1000);
};

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < visualHeap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
});

```

```

<form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder
 />
 <div className="flex flex-col items-center min-w-[300px]
 Приоритет
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Везти
 </button>
</form>

{/* Забрать самого тяжелого */}
<button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium transition-all cursor-pointer flex items-center justify-center"
 >
 Забрать
 </button>

```

```

 ${patient.animState === 'in' ? 'anim
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isCompilerNode ? 'ring-4 ring-amb
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full r
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
);
 }}}
</div>

```

```

 <div className="w-full h-3 bg-gradient-to-r f
 </div>

```

```

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 roun
 <div className="absolute top-4 left-4 text-[1
 Операционная койка

```

```
 <div className="bg-slate-900 border border-slate-500" data-bbox="173 101 1000 121">
 <div className="text-[10px] font-mono text-slate-500" data-bbox="206 126 1000 146">
 {log.map((entry, idx) => (
 <div data-bbox="240 173 1000 261">
 key={idx}
 className={`text-xs font-mono flex items-center justify-between gap-2`} data-bbox="273 221 1000 241">
 >
 {idx === 0 ?
 {entry}
 : {entry}
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
);
};
```

---

ФАЙЛ 47/111: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```

 <div className="text-[10px] text-slate-400 text-center"
 {caption}
 </div>
))
 <div className="text-[9px] text-slate-600 text-center"
 {paused ? "пауза – курсор на карточке" : "наведение"}
 </div>
</div>
);
};

```

```

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);

```

```

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 stroke-width={width}
 stroke-dasharray={dashed ? [4, 4]} : []
 />
);

```

```

 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done
 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>

```





```

const step = useLoop(frames.length, ms);
const f = frames[step];
const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "r" ? "#f08080" : "#f08080";

return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="middle">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: x под p, p под x"]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x", "x": "p"}]},
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p": "x", "x": "p"}]}
]}
 />
);

```

```

 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
 const order = ["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
 }}}
 </Svg>
);
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}}
)}

```

```

 <text x={224} y={74} textAnchor="middle" fontSize=
 </Svg>
);
};

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
 <Edge a={[60, 70]} b={[200, 70]} color={improved
 <text x={130} y={54} textAnchor="middle" fontSize=
 <Node x={60} y={70} r={20} fill={C.active} label=
 <Node x={200} y={70} r={20} fill={improved ? C.do
 <text x={60} y={106} textAnchor="middle" fontSize=
 <text x={200} y={106} textAnchor="middle" fontSize=
 <text x={130} y={22} textAnchor="middle" fontSize=
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился н
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} colo
 })}
);
};

```

```

const step = useLoop(chosen.length + 1, 800);
const taken = new Set(chosen.slice(0, step));
return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 })}
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={step
 })}
 </Svg>
);
};
};

```

```

 fill={i === items.length - 1 ? (popping ? C
 <text x={130} y={111 - i * 26} textAnchor="mi
 </g>
)))
 <text x={130} y={16} textAnchor="middle" fontSize=
</Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeigh
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХ
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХ
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} heigh
 <text x={53 + i * 54} y={71} textAnchor="midd
 >
 </g>
)))
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.d
 </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2
 const step = useLoop(parents.length, 950);
 const p = parents[step];

```



```

 о префиксные суммы P">
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim}>
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "stroke 0.2s" }}>
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для`}>
 {Array.from({ length: 8 }).map((_, i) => (
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={12}
 fill={C.dim}>
 <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
 </g>
))}
 {Array.from({ length: count }).map((_, i) => (
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}

```



```
export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? C.fill : C.idle}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">{idx}</text>
 </g>
))}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={12}>
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};
```

```
export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;

```

```
| "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
| "suffix-link" | "toposort" | "relax" | "prefix-sum"
| "segment-tree" | "bridge" | "articulation" | "mst"
| "prefix-function" | "dp" | "floyd" | "components" |
| "zig" | "zig-zig" | "zig-zag";
```

```
const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
};
```

```

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

 const viz = getViz(vizId ?? undefined);
 if (!vizId || !viz) return null;
 const Component = viz.Component;

 return createPortal(
 <div className="fixed inset-0 z-[110] flex items-en
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-l
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
 <VizLiveStatus demoTitle={viz.title} classN
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hove
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 </div>
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor, onClose }) => {
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>(null);

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left, [anchor]});
 }, [anchor]);

 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onKey, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onKey, true);
 };
 }, [anchor, onClose]);
};
```

```

 {entry.formal && {
 <p className="text-[11.5px] leading-relaxed" >
 {entry.formal}
 </p>
 }}

 {entry.complexity && {
 <div className="text-[11px] font-mono text-left" >
 Сложность: {entry.complexity}
 </div>
 }}

 {viz && simulatorId && {
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center gap-2
 rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
 }}
 </div>
</div>
</div>,
document.body
);
};

```

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

```

```

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-width-2px";
 if (step === 2) return "stroke-pink-500 stroke-width-2px";
 return "stroke-slate-700 stroke-dasharray-4 4 stroke-width-2px";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-width-2px";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-width-2px";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // отсюда и далее
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

```

```

 if (e.id === 'CA' && step > 6) return true;
 return false;
 };

 const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соеди
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mo
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>

```

```

<div className="flex flex-col md:flex-row gap-4" style={
 <div className="bg-[#0f172a] flex-1 min-h-[300px] p-4">
 <svg width="400" height="300" class="h-full w-full">
 <defs>
 <marker id="arrow-slate" viewBox="0 0 10 5" orient="vertical"
start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: slategray; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 5" orient="vertical"
uto-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: slategray; stroke-width: 2px; opacity: 0.5;" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 5" orient="vertical"
start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #f1c40f; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5" orient="vertical"
tart-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #f08080; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 5" orient="vertical"
o-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #27ae60; stroke-width: 2px;" />
 </marker>
 </defs>

 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

```



```

? 'fill-[#2
: 'fill-sla
/>
<text x={n.x} y={n.y}
-white">
 {n.label}
</text>

{pot !== null && {
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
</g>
)}
</g>
)
}}
</svg>
</div>

<div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
 {getDesc()}
 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 b
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}

```

```

 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;

```

```

 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
));
 });
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);
 for (const to of adj[u]) {

```

```

let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в ${sccMap[u]}`,
 u,
 false,
 sccMap[u],
);
 }
}

```

```

 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getScColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

```

```

 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1f77b4" stroke="#1f77b4" stroke-width="1" />
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666" stroke="#666666" stroke-width="1" />
 </marker>
 </defs>

```

```

{ /* Edges */ }
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u);
 const v = initialNodes.find((n) => n.id === edge.v);

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
 } else {
 if (

```

```

 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "done") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration-300`}
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 />
)}
 </g>
);
}

```

```

 <span className="text-slate-600 text-
) : {
 [...step.order].reverse().map(({val, i
 <div
 key={idx}
 className="bg-indigo-950 border b
ex items-center gap-1"
 >
 {val}
 </div>
))
 })
</div>
</div>

{/* List of actions log */}
<div>
 <span className="text-[10px] text-slate-5
 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 roun
 >
 {s.desc}
 </div>
 }}}
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-sla

```



```
interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'm
}

interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'reje
 color: string;
}

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
```

```

 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]);
},
// Step 2: Include A-B
() => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
},
// Step 3: Inspect D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветн
 type: 'info'
 }, ...prev]));
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цв
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',

```

```

 title: 'Шаг 5: Берем ребро В-Е (вес 6)',
 description: 'Н0: это ребро соединяет вершины В
 type: 'warning'
 }, ...prev]));
},
// Step 10: Reject В-Е
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро В-Е. Иначе п
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',

```

```

 type: 'info'
 }, ...prev]);
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в осто́ве',
 description: 'Вершина H перекрашена в фиолетовый',
 type: 'success'
 }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. С',
 type: 'warning'
 }, ...prev]));
},
// Step 20: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
},

```

```

// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-E (8)']);
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D: A-D (5), B-E (6), E-F (7), D-E (8)',
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вершины D',
 type: 'info'
 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D-E (8)']);
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в кучу',
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',

```

```

() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро В-Е (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет верши
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро В-Е отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (Е) тянет
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]));
},

```

```

 type: 'error'
 }, ...prev]);
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.'
 type: 'warning'
 }, ...prev]));
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{

```

```

// Node 5(F) -> E-F(7)
// Node 6(G) -> D-G(8)
// Node 7(H) -> G-H(10)
// Node 8(I) -> H-I(12)
setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбирает
description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
}, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Components)
() => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/olive)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' }
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' }
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы
description: 'Все выбранные дороги построены ра
 оз {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
}

```



```

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const introLog = {algo: 'kruskal' | 'prim' | 'boruvka'
 if (algo === 'kruskal') {
 return [{
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 type: 'info',
 }
];
 }
 if (algo === 'prim') {
 return [{
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем старто
 ь (Heap) и захватываем соседей!',
 type: 'info',
 }
];
 }
 return [{
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 дерев
 ы объединиться в колхоз.',
 type: 'info',
 }
];
};

const handleReset = {algo: 'kruskal' | 'prim' | 'boru

```

```

 return null;
 }, []));

useVizStepSync(stepIndex, goToStep, currentSteps.length);

/** A..I ↔ id 0..8 */
const idOf = (name: string | number | null | undefined) => {
 if (name === null || name === undefined) return null;
 if (typeof name === 'number') return Number.isInteger(name) ? name : null;
 const c = name.trim().toUpperCase().replace(/[^A-I]/g, '');
 return c ? c.charCodeAt(0) - 65 : null;
};

/** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
const parseEdge = (item: unknown): { u: number; v: number; w: number | null } | null => {
 if (item === null || item === undefined) return null;
 if (typeof item === 'string') {
 const nums = item.match(/-?\d+(?:\.\d+)?/g);
 const letters = item.match(/[A-Ia-i]/g);
 let u: number | null = null;
 let v: number | null = null;
 if (letters && letters.length >= 2) { u = idOf(letters[0]); v = idOf(letters[1]); }
 else if (nums && nums.length >= 2) { u = idOf(nums[0]); v = idOf(nums[1]); }
 if (u === null || v === null) return null;
 return { u, v, w: nums && nums.length ? Number(nums[2]) : null };
 }
 if (Array.isArray(item)) {
 const letters = item.filter((x) => typeof x === 'string');
 const nums = item.filter((x) => typeof x === 'number');
 if (letters.length >= 2) {
 const u = idOf(letters[0]); const v = idOf(letters[1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length ? nums[0] : null };
 }
 if (nums.length >= 2) {
 const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length > 2 ? nums[0] : null };
 }
 }
 return null;
};

```

```

 if (id > 8) return;
 feed(id, val);
 });
}
/**
 * Ключ множества вершины: если код дал DSU-массив
 * идём по цепочке до корня – иначе вершины одной к
 * разные цвета. Для произвольных меток («comp1») б
 */
const compOfVertex = new Map<number, string>();
rawComp.forEach((label, id) => {
 if (parentIdx.size === 0) {
 compOfVertex.set(id, label);
 return;
 }
 let cur = id;
 const seen = new Set<number>([id]);
 for (let hop = 0; hop < 16; hop++) {
 const nxt = parentIdx.get(cur);
 if (nxt === undefined || nxt === cur || seen.has(nxt)) {
 seen.add(nxt);
 cur = nxt;
 }
 compOfVertex.set(id, String.fromCharCode(65 + cur));
 });
const takenRaw = vizArray(vars.mst) ?? vizArray(vars.mst);
const takenEdges = (takenRaw ?? []).map(parseEdge);
const takenVertices = (vizArray(vars.taken) ?? vizArray(vars.taken))
 .map((x) => idOf(typeof x === 'string' ? x : vizNumber(x)))
 .filter((x): x is number => x !== null);
const curU = idOf(vizString(vars.u) ?? vizNumber(vars.u));
const curV = idOf(vizString(vars.v) ?? vizNumber(vars.v));
const curW = vizNumber(vars.w);
const heapRaw = vizArray(vars.heap) ?? vizArray(vars.heap);
const heapEdges = (heapRaw ?? []).map(parseEdge);
const cheapestRaw = vizRecord(vars.cheapest) ?? vizRecord(vars.cheapest);
const cheapestEdges = cheapestRaw ? Object.values(cheapestRaw)
 .filter(e => e !== null) : [];

```

```

const getColorClass = (color: string, id: number) =>
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 case 'purple': return 'bg-purple-600 border-purple
 case 'mold': return 'bg-emerald-500 border-emerald
 case 'gold': return 'bg-amber-400 border-amber-200
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn
 return 'bg-slate-700 border-emerald-500 text-
 }
 return 'bg-slate-700 border-slate-500 text-slate
 }
};

```

```

const getEdgeStroke = (status: string, color: string)
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

```

```

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-900" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >


```

```

 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md
 : 'text-slate-400 hover:text-slate-1
)
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-center gap-2
-200 font-semibold text-xs rounded-xl border
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
)
 >
 Следующий шаг
 </button>

```

```


 })
 {live.cheapestEdges.length > 0 && {

 cheapest: {live.cheapestEdges.length}

 })
 {live.total !== null && {

 вес остова = {live.total}

 })
 </>
) : (

 открой панель Python и запусти код вкладки
 'ка'>»: переменные
 {live.cheapestEdges}

 <code className="font-mono text-purple" style="font-family: monospace;">
 e className="font-mono text-purple-300">u</code>
 <code className="font-mono text-purple-300" style="font-family: monospace;">
 mono text-purple-300">w</code> перекрасят э

)
 </div>

<div className="grid grid-cols-1 lg:grid-cols-3" style="font-family: monospace;">
 {/* Graph Canvas */}
 <div className="lg:col-span-2 bg-slate-950 rounded-full relative min-h-[480px] overflow-hidden">

 {/* Legend */}
 <div className="absolute top-4 left-4 bg-slate-900 p-4 text-white" style="font-family: monospace;">
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-900">
 </div>
 {activeAlgo === 'kruskal' ? (
 <div className="flex items-center gap-1.5">

```

```

 { /* SVG Edges */
 <svg className="absolute inset-0 w-full h-full"
 spectRatio="none">
 {effEdges.map(edge => {
 const u = effVertices[edge.u];
 const v = effVertices[edge.v];
 const strokeColor = getEdgeStroke(edge);
 const isInspecting = edge.status === 'inspect';
 const isRejected = edge.status === 'rejected';
 const isInHeap = edge.status === 'in_heap';

 return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-200"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' :
 : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 })}
 </svg>
);
}

```

```

<div className="bg-slate-950 rounded-2xl border-2 border-b
 <div className="p-4 bg-slate-900 border-b border-r border-t
 <h4 className="font-bold text-slate-200 text-sm">
 Ход выполнения
 </h4>
 </div>
</div>

```

```

{activeAlgo === 'prim' && {
 <div className="p-3 bg-slate-900/60 border-2 border-b border-r border-t
 <p className="text-[11px] font-bold text-slate-200">
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5">
 {heapQueue.length === 0 ? {
 Очередь пуста
 } : {
 heapQueue.map((item, idx) => {
 <span key={idx} className={"px-2 py-1 text-xs font-mono" +
0 shadow-md animate-pulse" : "bg-slate-800 text-slate-200"}>
 {item}

 })
 })
 </div>
 </div>
}}

```

```

<div className="flex-1 p-4 overflow-y-auto">
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-all duration-200" +
 (log.type === 'success'
 ? 'bg-emerald-950/40 border-emerald-500'
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-500'
 : log.type === 'error'
 ? 'bg-red-950/40 border-red-500'
 : 'bg-slate-900/40 border-slate-500')
 >
 {log.message}
 </div>
 </div>
)}

```



```

 == 'prim' ? "bg-emerald-600 text-white" : "
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka
 className={"px-3 py-1.5 rounded text-xs fo
 == 'boruvka' ? "bg-rose-600 text-white" : "
 >
 Борувка (Билет 20)
 </button>
</div>
</div>

{activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
 <Clock className="w-3.5 h-3.5 text-in
 </p>
 <p className="text-2xl font-black text-
 <p className="text-slate-400 text-xs mt
 </div>
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
 <Award className="w-3.5 h-3.5 text-in
 </p>
 <p className="text-2xl font-black text-
 <p className="text-slate-400 text-xs mt
 </div>
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
 ожении:</p>
 <p className="text-xs text-slate-300 le
 ди и красим вершины в один цвет, бракуя цик
 </div>
 </div>
 </div>

```

```

 </p>
 <p className="text-2xl font-black text-white">
 <p className="text-slate-400 text-xs mt-2">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <p className="text-2xl font-black text-white">
 <p className="text-slate-400 text-xs mt-2">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 в предложении:
 <p className="text-xs text-slate-300 leading-5">
 не чистые вершины через самое дешевое доступное ребро
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

```

```
<Code className="w-3.5 h-3.5 text-rose-600">
</p>
<pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`def boruvka(n, edges):
 parent = list(range(n))
 mst = []
 while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост н
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2] < w:
 cheapest[ru] = [u, v, w]
 if cheapest[rv] == -1 or cheapest[rv][2] < w:
 cheapest[rv] = [u, v, w]
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge[1]):
 mst.append(bridge)`}
</pre>
</div>
</div>
</div>
);
};
```

ФАЙЛ 57/111: `src/components/LiteGuide.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
```

```

padding: 0 .25rem; }
.lite-wrap pre { background: #f3f4f6; border: 1px solid #d1d3d4; padding: 1.45rem; }
.lite-wrap table { border-collapse: collapse; width: 100%; }
.lite-wrap th, .lite-wrap td { border: 1px solid #d1d3d4; padding: .5rem; }
.lite-wrap th { background: #f3f4f6; }
.lite-wrap details { margin: .6rem 0; border: 1px solid #d1d3d4; padding: .5rem; }
.lite-wrap summary { cursor: default; font-weight: 700; }
.lite-toc { font-size: .92rem; }
.lite-toc ol { margin: .4rem 0 0 1.2rem; }
@media print {
 .lite-wrap { max-width: none; padding: 0; }
 h2 { page-break-before: always; }
 h2:first-of-type { page-break-before: avoid; }
 pre, table, details { page-break-inside: avoid; }
}

```

```

/** Раскрываем все спойлеры и вычищаем служебную разметку */
function toPlainTextHtml(html: string): string {
 return html
 .replace(/<details(?:[>]*\bopen\b)/g, "<details open")
 // кнопки/иконки внутри текста не нужны в текстовой версии
 .replace(/<button[\s\S]*?<\s*\/button>/gi, "")
 .replace(/<svg[\s\S]*?<\s*\/svg>/gi, "")
 // интерактивные подсказки терминов оставляем как есть
 .replace(/<sdata-term-id="[^"]*">/g, "")
 .replace(/<class="term-hint"[^"]*">/g, "")
 // ссылки на другие билеты должны вести в текстовую версию
 .replace(/<href="\?topic=/g, 'href="<?lite=1&topic=')
 .replace(/<class="ticket-link"/g, "");
}

```

```

interface Props {
 /** id одной темы; undefined – все темы подряд. */
 topicId?: string;
}

```

```

<article key={chapter.id} id={chapter.id}>
 <h2>{chapter.title}</h2>
 <p className="lite-topics">
 {topics.length > 0 && <>Билет{topics.leng
 {chapter.category ? <>Раздел: {chapter.ca
 id страницы: <code>{chapter.id}</code>
 </p>

 <div
 dangerouslySetInnerHTML={{
 __html: linkifyTicketsHtml(toPlainTextH
 }}
 />

 {viz && (
 <div className="lite-viz">
 <h4>Визуализация: {viz.title}</h4>
 {viz.hint && <p>{viz.hint}</p>}
 {syncKeys.map((k) => {
 const sync = PAGE_SYNC[k];
 const demo = k.includes("#") ? k.spli
 return (
 <div key={k}>
 <p>
 {demo ? `Вкладка «${demo}»`
 {sync.vizTitle ? ` – ${sync.viz
 {sync.stepNote ? <> Шаг демонстр
 </p>
 {sync.variables.length > 0 && (

 {sync.variables.map((v) => (
 <li key={v.name}>
 <code>{v.name}</code> – {
 {v.range ? ` (${v.range})

)}}

)}}
)}}
)}}
)}}
)}}
)}}
)}}

```

```
export default LiteGuide;
```

---

ФАЙЛ 58/111: src/components/LiveDistTable.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import type { DebugValue } from "../data/debugRunner";

interface LiveDistTableProps {
 /** dist из кода пользователя: имя вершины → расстояние */
 dist: Record<string, DebugValue> | null;
 /** prev/p – предок на кратчайшем пути (необязательно) */
 prev?: Record<string, DebugValue> | null;
 /** done/visited – зафиксированные вершины (необязательно) */
 done?: DebugValue[] | null;
 /** Идентификаторы вершин, нарисованных на схеме демо */
 demoIds: string[];
 /** Подпись таблицы (по умолчанию «Ваши расстояния»). */
 title?: string;
}

const fmt = (value: DebugValue | undefined): string => {
 if (value === undefined || value === null) return "–";
 if (typeof value === "number") {
 if (value === Number.POSITIVE_INFINITY) return "∞";
 if (value === Number.NEGATIVE_INFINITY) return "−∞";
 return String(value);
 }
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "infinity") return "∞";
 if (t === "-inf" || t === "-infinity") return "−∞";
 return value;
 }
 return String(value);
}
```

```

<div className="mt-2.5 flex flex-wrap gap-1.5">
 {shown.map(({key}) => {
 const parent = prev ? fmt(prev[key]) : null;
 const fixed = doneSet.has(key);
 return (
 <span
 key={key}
 className={`inline-flex items-baseline gap-1
 ${fixed
 ? "border-emerald-500/40 bg-emerald-500/10"
 : "border-slate-700 bg-slate-900/70 text-slate-400"}
 `}
 title={fixed ? "вершина зафиксирована (done)" : ""}
 >
 <b className={fixed ? "text-emerald-300" : ""}>
 =
 {fmt(dist[key])}
 {parent && parent !== "-" ?
 {fixed ?

 } : null}

);
 })}
 {keys.length > shown.length ? (

 +{keys.length - shown.length} ещё

) : null}
</div>
</div>
);
}

```

```

 highlightColor: 'text-emerald-400',
 rest: '}'. Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-500/40',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4`}>
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover grid" />
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc} {c.complexity}
 </p>
 <div className={`mt-3 text-xs font-mono px-2`}>{c.rest}</div>
 </div>
))}
 </div>
);
}

```

ФАЙЛ 60/111: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useState } from "react";
```



```

 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание • {index + 1} из {total}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950">
 <div className="absolute inset-y-0 left-0 w-[100px] h-[100px] bg-white te-[tocIn_.18s_ease-out]">
 <div className="flex items-center justify-between">

 Содержание
 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
 </div>
)}
 </div>
</div>

```

```

return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 h-10 w-10 rounded-sm">
 <Sparkles className="w-5 h-5 lg:w-6 lg:h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-lg lg:text-xl font-extrabold">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center flex-wrap sm:flex-wrap sm:gap-4">
 <div className="flex items-center gap-2">

 <div style="background-color: #4f4f9f; color: white; padding: 2px 5px; border-radius: 3px; font-size: 0.8em; font-weight: bold; white-space: nowrap">
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </div>

 <button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (настройка)"
 aria-label={compilerOpen ? "Скрыть панель Python-компилятора" : "Показать панель Python-компилятора"}
 className={`shrink-0 flex items-center justify-center gap-2 duration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg'
 : 'text-slate-400 hover:text-white bg-slate-800'
 }`}
 >
 <div style="font-size: 0.8em; font-weight: bold; white-space: nowrap">Python-компилятор
 </div>
 </div>
 </div>
 </div>
)

```

```

 title="Полный PDF-дамп исходного кода
 >
 <FileDown className="w-3.5 h-3.5" />

<a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}exp
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center gap-2 px
/20 border border-sky-500/30 transition-all
 title="Скачать полный TXT файл со все
 >
 <FileText className="w-3.5 h-3.5" />

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center gap-2 w-
te hover:bg-amber-600/20 border border-amber
ointer"
 title="Скачать всё пособие одним авто
 >
 {busy ? <Loader2 className="w-3.5 h-3
 </button>
</div>
</details>
</div>
</div>
</div>
</header>
);
};

```

```

edges: [number, number][];
adj: number[][];
pos: { x: number; y: number }[];
names: string[];
}

const PALETTE = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];
const colorOf = (c: number) => (c >= 1 && c <= PALETTE.length) ? PALETTE[c - 1] : "#ccc";

function makeGraph(key: GraphKey): Graph {
 if (key === "bip") {
 const edges: [number, number][] = [[0, 3], [0, 4], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]];
 const pos = [
 { x: 80, y: 60 }, { x: 80, y: 150 }, { x: 80, y: 240 },
 { x: 380, y: 60 }, { x: 380, y: 150 }, { x: 380, y: 240 }
];
 const g: Graph = { key, label: "Двудольный (6 вершин, 6 ребер)", n: 6, edges };
 g.adj = buildAdj(g);
 return g;
 }
 const edges: [number, number][] = [[0, 1], [0, 2], [1, 2], [1, 3], [1, 4], [1, 5], [2, 3], [2, 4], [2, 5], [3, 4], [3, 5], [4, 5]];
 const pos = [
 { x: 70, y: 60 }, { x: 175, y: 40 }, { x: 150, y: 185 },
 { x: 250, y: 185 }, { x: 365, y: 110 }, { x: 350, y: 60 }
];
 const g: Graph = { key, label: "Основной (8 вершин, 12 ребер)", n: 6, edges };
 g.adj = buildAdj(g);
 return g;
}

function buildAdj(g: Graph): number[][] {
 const adj: number[][] = g.edges.reduce((acc: number[][], [u, v]) => {
 acc[u].push(v);
 acc[v].push(u);
 return acc;
 }, Array.from({ length: g.n }, () => [] as number[]));
 adj.forEach((a) => a.sort((x, y) => x - y));
}

```

```

 color: [...color], v, u: null, e: null, used,
 });
}
const chi = Math.max(...color);
frames.push({
 title: `Готово: жадная покраска использовала ${chi}
 note: `χ_жадн = ${chi}. Истинное хроматическое число
 , а теорема Брукса ($\chi \leq \Delta$, если граф не полный и
 color: [...color], v: null, u: null, e: null,
});
return frames;
}

```

/\*\* Проверка двудольности BFS-ом: 2 краски    нет нечётных

```

function bipartiteFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const color = new Array(g.n).fill(0);
 const queue: number[] = [];
 let conflict: [number, number] | null = null;
 frames.push({
 title: "Старт: пробуем покрасить в 2 цвета",
 note: "BFS красит вершину в цвет, противоположный р
 двудольный ($\chi > 2$).",
 color: [...color], v: null, u: null, e: null,
 });
 for (let start = 0; start < g.n && !conflict; start++) {
 if (color[start] !== 0) continue;
 color[start] = 1;
 queue.push(start);
 frames.push({
 title: `Запускаем BFS из ${g.names[start]}: цвет
 note: `Вершина ${g.names[start]} ещё не покрашена
 и 2.`
 color: [...color], v: start, u: null, e: null,
 });
 while (queue.length && !conflict) {
 const v = queue.shift() as number;
 for (const u of g.adj[v]) {

```

```

const delta = Math.max(...g.adj.map((a) => a.length))
frames.push({
 title: "Старт: рёбра не покрашены",
 note: `Правильная рёберная покраска: у каждой верши
 бер одной вершины различны), а по теореме Визинга
 color: new Array(g.n).fill(0), ecolord: [...ecolor],
});
g.edges.forEach(([a, b], i) => {
 const usedA = Array.from(inc[a].values());
 const usedB = Array.from(inc[b].values());
 const used = Array.from(new Set([...usedA, ...usedB]
 let c = 1;
 while (used.includes(c)) c += 1;
 ecolord[i] = c;
 inc[a].set(i, c);
 inc[b].set(i, c);
 frames.push({
 title: `Рёбро (${g.names[a]}, ${g.names[b]}): цве
 note: `В вершине ${g.names[a]} заняты цвета ${us
 y) => x - y).join(", ") || "-"}}. Наименьший св
 да.`
 color: new Array(g.n).fill(0), ecolord: [...ecolor
 });
});
const chiE = Math.max(...ecolor);
frames.push({
 title: `Готово:  $\chi' = \chi_E$  при  $\Delta = \Delta$ `,
 note:
 g.key === "bip"
 ? `Двудольный граф: $\chi' = \Delta = \Delta$ – это теор
 сс 1 по Визингу.`
 : ` $\chi' = \chi_E$. Теорема Визинга: $\Delta \leq \chi' \leq \Delta + 1$
 $\chi' = \Delta + 1$ – классом 2 (например, K3 и любой
 color: new Array(g.n).fill(0), ecolord: [...ecolor],
});
return frames;
}

```

```

 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
}, 1300);
return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveColor = vars ? vizArray(vars.color) ?? vizArray(0) : null;
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vizArray(0) : null;
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumber(0) : null;
const linked = Boolean(liveColor || liveEcolor || (liveV || liveU || liveE || liveChi));

const frame = frames[Math.min(step, maxStep)] ?? ({ t: 0, color: 0, ecol: 0 });
const nodeColor = (id: number): number => {
 if (liveColor && typeof liveColor[id] === "number")
 return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
 if (liveEcolor && typeof liveEcolor[i] === "number")
 return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV) ? liveV : 0;
const curU = liveU !== null && Number.isFinite(liveU) ? liveU : 0;
const curE = liveE !== null && Number.isFinite(liveE) ? liveE : 0;
const delta = Math.max(...graph.adj.map((a) => a.length));

```

</div>

```
<div className="flex flex-wrap items-center gap-2"
 <button onClick={() => { setStep(0); setPlaying(
 rounded-lg bg-slate-900 border border-slate-
 <button onClick={() => go(-1)} disabled={step =
 00 border border-slate-700 text-slate-300 hove
 <button onClick={() => setPlaying((p) => !p)} c
 ext-xs font-bold hover:bg-indigo-500">
 {playing ? <Pause className="w-4 h-4" /> : <P
 </button>
 <button onClick={() => go(1)} disabled={step >=
 late-900 border border-slate-700 text-slate-3
 on>
 <span className="text-[11px] text-slate-500 ml-
 <span className="ml-auto flex flex-wrap items-c
 <span className="bg-slate-950 border border-s
 {tab === "edges"
 ? <span className="bg-slate-950 border bord
 rs}
 : <span className="bg-slate-950 border bord
 }}

</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-5 g
 <div className="lg:col-span-3 bg-slate-950 roun
 <svg viewBox="0 0 480 300" className="w-full l
 {graph.edges.map(([a, b], i) => {
 const ec = edgeColor(i);
 const isCur = i === curE;
 const isBad = Boolean(frame.bad && ((fram
));
 return (
 <g key={i}>
 <line
 x1={graph.pos[a].x} y1={graph.pos[a
 stroke={isBad ? "#f43f5e" : ec > 0
```



```

 <p className="text-[10px] uppercase tracking
 <div className="flex flex-wrap gap-1.5 font
 {(tab === "edges" ? graph.edges.map((_, i)
 <span key={i} className="px-1.5 py-0.5
 ? colorOf(c) : "#475569", background: c > 0
 {tab === "edges" ? `e${i}` : graph.na

)))
 </div>
</div>
<div className="bg-slate-900/70 border border
 {tab === "greedy" && <>Жадная покраска: <b
 : <b className="text-slate-200"> $\chi \leq \Delta$,
 в $\chi \leq 5$ (доказывается через $E \leq 3V - 6$ и це
 ром).</>
 {tab === "bipartite" && <>Двудольный граф
 /DFS за $O(V + E)$. Распознавание 3-раскрашив
 {tab === "edges" && <>Рёберная покраска: <b
 Δ (Кёниг). Задача « $\chi' = \Delta$?» NP-полна даже
 </div>
</div>
</div>
<div className="mt-3 flex flex-wrap items-center
 <span className="text-[10px] uppercase tracking
 /> из компилятора:
 {vars && linked ? (
 ["v", "u", "e", "color", "ecolor", "chi"].map
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] f
 y-1">
 {name} = {JSON.stringify(vars[name]).sl

)
)
) : (

 открой панель Python: массив <code classNam

```

---

ФАЙЛ 63/111: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотрим
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number) {
 return (by-ay)*(ax-bx)>0;
 }
 const o1=ccw(p1.x,p1.y,p2.x,p2.y),o2=ccw(p1.x,p1.y,p3.x,p3.y),o3=ccw(p1.x,p1.y,p4.x,p4.y),o4=ccw(p2.x,p2.y,p3.x,p3.y),o5=ccw(p2.x,p2.y,p4.x,p4.y),o6=ccw(p3.x,p3.y,p4.x,p4.y);
 return (o1>0&&o2>0&&o3>0&&o4>0&&o5>0&&o6>0);
 }
 })()}
 </svg>
 </div>
 </div>
 </div>
);
}
```

```

<div className="bg-slate-950 rounded-xl p-4 border"
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0"
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:160,y:220},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:60,y:220},
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
)}
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 </svg>

```

```

 ChevronLeft,
 ChevronRight,
 ChevronUp,
 CircleStop,
 Copy,
 Eye,
 EyeOff,
 HelpCircle,
 Link2,
 Loader2,
 Pause,
 Pencil,
 Play,
 Plus,
 RotateCcw,
 SkipBack,
 SkipForward,
 Terminal,
 Unlink,
 Variable,
 X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed

```

```

declare global {
 interface Window {
 __algv0PythonRuntime?: PythonRuntimeProvider;
 }
}

function getPythonRuntime(): Promise<PyodideRuntime> {
 const injected = typeof window !== "undefined" ? window : undefined;
 if (injected) return typeof injected === "function" ? injected : undefined;
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_PATH)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_PATH }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}

/**
 * Предел вывода в панель «Результат».
 *
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), но не по
 * напечатанной строки: print("x" * 10_000_000) или print("x" * 10_000_000)
 * — это один шаг, после которого весь текст уезжал в <div>.
 * Обрезаем по границе строки и честно пишем, сколько э
 */
const OUTPUT_CHAR_CAP = 20_000;

function clipOutput(text: string): string {
 if (text.length <= OUTPUT_CHAR_CAP) return text;
 const cut = text.slice(0, OUTPUT_CHAR_CAP);
 const lastBreak = cut.lastIndexOf("\n");
 const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.slice(0, lastBreak) : cut;
 return `${head}\n... вывод обрезан: показаны первые ${head.length} символов`;
}

```

```

 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матрицы",
 code: "n, m = 3, 4\nfor i in range(n):",
 },
 {
 label: "while ...",
 tip: "Классический while-счётчик – как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"итерация {i}\")",
 },
 {
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i – позиция, x – элемент",
 code: "for i, x in enumerate(['a', 'b', 'c']):",
 },
],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = take",
 code: "st = []\nst.append('A') # push\nst.pop() # take",
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
 code: "from collections import deque\nq = deque()\nq.append('A')\nq.popleft()",
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) – как очередь приоритетов",
 code: "import heapq\nh = []\nheapq.heappush(h, (1, 'A'))\nheapq.heappop(h)",
 },
],
}

```

```

const syncVarBases = useMemo(
 () =>
 [...new Set(
 sync.variables.flatMap((v) =>
 v.name
 .split(/\s*(?:,|\s\/)\s*/)
 .map(baseVarName)
 .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/
)
)
]),
 [sync]
);
const syncVarSet = useMemo(() => new Set(syncVarBases
/** Полный эталон: глаз сразу вставляет его в обычный
const template = useMemo(() => buildSyncTemplate(chap
/** Дефолт редактора: только инициализация демо; она
const initTemplate = useMemo(() => buildInitTemplate(

const [code, setCode] = useState(() => {
 const saved = savedEditors.get(chapterId);
 if (!saved) return initTemplate;
 if (saved.code === saved.base || saved.code.trim()
 return saved.code;
});
/** «Глаз» – переключатель: эталонный код демо ↔ иници
const [refOn, setRefOn] = useState(() => {
 const saved = savedEditors.get(chapterId);
 return !!saved && saved.code.trim() !== "" && saved
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState(
/** Принудительный повтор (глаз / Ctrl+Enter), даже е
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "hi

```

```

useEffect(() => {
 savedEditors.set(chapterId, { code, base: initTemplate,
}, [code, initTemplate, chapterId]);

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId(d)), [chapterId]);
useEffect(() => () => clearVizState(chapterId), [chapterId]);

// Смена страницы (или вкладки демо): нетронутый редактор требует
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
 const prev = prevTplRef.current;
 if (initTemplate === prev.base && template === prev.full) {
 prevTplRef.current = { base: initTemplate, full: template };
 setPlaying(false);
 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старого кода
 setRefOn(false);
 if (code === prev.base || code === prev.full || code === prev.full) {
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: template };
 setCode(initTemplate);
 setRefOn(false);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);

```



```

const variables = { ...afterValues };
const locals = { ...afterLocals };
for (const [name, value] of Object.entries(beforeValues)) {
 const scalar = value === null || typeof value !== 'object';
 if (scalar && !assigned.has(name) && name in afterValues) {
 variables[name] = value;
 if (step?.locals[name] !== undefined) locals[name] = step?.locals[name];
 }
}
const changed = changedVars(step?.locals, locals);
return {
 line: step?.line,
 func: step?.func,
 locals,
 variables,
 changed: Object.keys(changed).filter((name) => changed[name]),
};
}, []);

```

```

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) => {
 if (!aliveRef.current) return; // панель закрыта
 if (!hasViz || result.steps.length === 0) return;
 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,
 });
 },
 [chapterId, hasViz, snapshotAt]
);

```

```

// Единственная точка публикации: любое перемещение трекера
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, debug.source);
}, [debug]);

```

```

 return;
 }
 const t = window.setTimeout(() => stepBy(1), 650 / playbackSpeed);
 return () => window.clearTimeout(t);
}, [playing, debug, stepBy, playbackSpeed]);

const debugRun = useCallback(async (autoPlay = false) => {
 if (!running) return;

 const source = code;
 const input = stdin;
 const request = traceRequest;
 setRunning(true);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 clearVizState(chapterId);
 setOutputOpen(false);

 // Пустой редактор: Pyodide не гоняем, панель в режиме ожидания
 // связь с визуализацией снимаем — демо показывает, что не работает
 if (source.trim() === "") {
 setDebug({ result: { steps: [], error: "", truncated: false } });
 setOutput("В редакторе пусто — выполнять нечего.\n\nПустой код: связь с визуализацией снята — «глаз».");
 setRuntimeMessage("Пустой код: связь с визуализацией снята — «глаз».");
 setOutputOpen(true);
 setRunning(false);
 return;
 }

 setRuntimeMessage(runtimeReady ? "Обновляю трассу..." : "Запускаю трассу...");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = input.split(/\r?\n/);

 try {

```

```

 });
 if (result.error) setOutputOpen(true);
 setRuntimeMessage(
 result.truncated
 ? `Выполнение остановлено после ${DEBUG_STEPS} шагов`
 : "Трасса готова автоматически. ← → — шаги,
);
 }
} catch (error) {
 if (!aliveRef.current) return;
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, truncated: true },
 clearVizState(chapterId);
 setOutputOpen(true);
 setOutput(`Ошибка:\n${message}`);
 setRuntimeMessage("Не удалось выполнить код — попробуйте снова");
} finally {
 setRunning(false);
}
}, [chapterId, code, stdin, traceRequest, running, runtimeReady]);

// Код выполняется сам: debounce даёт спокойно допечать
// изменился во время загрузки Pyodide, после завершения
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin &&
 const timer = window.setTimeout(() => void debugRun, 100);
 return () => window.clearTimeout(timer);
}, [code, stdin, traceRequest, running, runtimeReady]);

// Стрелки ← → в режиме отладки листают ШАГИ (а не строки)
useEffect(() => {
 if (!debug || debug.result.steps.length === 0) return;
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName))
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {

```

```

 if (curDebugLine === undefined || !listRef.current)
 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
 }, [curDebugLine]);

 // производные данные шага: что изменилось, в каком п
 // Строка, которую назначает текущий шаг, подсвечивае
 // локали берём со следующего шага трассы (для послед
 // трейсером на возврате из фрейма). Значит на строке
 const shownLocals = useMemo(() => {
 if (!debug || !curDebug?.step) return {};
 return snapshotAt(debug.result, debug.idx, debug.sr
 }, [debug, curDebug, snapshotAt]);
 const debugChanged = curDebug?.step ? changedVars(cu
 const debugOrdered = curDebug?.step ? orderVars(shown

 /** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
 const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) ste
 setPlaying(true);
 }, [debug, debugRun, playing, running, stepTo]);

 /** Вставка сниппета из шпаргалки в позицию курсора.
 const insertSnippet = useCallback(({snippet: string}) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n")

```

```

 const next = Math.round(startH + startY -
 onHeightChange(Math.max(minHeight, Math.m
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 el.removeEventListener("pointercancel", on
 };
 el.addEventListener("pointermove", onMove a
 el.addEventListener("pointerup", onUp as Ev
 el.addEventListener("pointercancel", onUp a
 }}
>
 <div className="absolute left-1/2 top-1/2 h-1
 digo-400 transition-colors" />
 </div>
 })

 { /* Левый край меняет ширину на десктопе. */ }
 { onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину п
 title="Потяните, чтобы изменить ширину панели
 className="hidden lg:block absolute top-0 bot
 onPointerDown={(e) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX
 onWidthChange(Math.max(380, Math.min(next
);
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp

```

```

</Tooltip>
<Tooltip side="bottom" content="Скопировать код"
 <button
 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Скопировать код"
 >
 {copied ? <Check className="w-4 h-4 text-emerald-500" /> : <Copy className="w-4 h-4 text-slate-400" />}
 </button>
</Tooltip>
<Tooltip side="bottom" content="Вернуть комментарий"
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip side="bottom" content="Скрыть панель"
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Заккрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip
 side="bottom"
 content={
 refOn
 ? "Убрать эталонный код: редактор станет собственному встроенному демо. Кнопка ↶ вернет к исходному коду"
 : "Вставить полный эталонный код этого файла"
 }
 >
 <button
 type="button"
 onClick={toggleFullCode}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Вставить полный эталонный код"
 >
 {fullCode ? <RotateCcw className="w-4 h-4" /> : <Copy className="w-4 h-4 text-slate-400" />}
 </button>
 </Tooltip>

```

```

ld-400" />}
{running
<select
 aria-label="Скорость автопрохода"
 value={playbackSpeed}
 onChange={(event) => setPlaybackSpeed(Num
 className="cursor-pointer rounded bg-slate
>
 {PLAYBACK_SPEEDS.map((speed) => (
 <option key={speed} value={speed}>{speed
)))}
</select>
</label>
</div>
</div>

{/* — Мини-вкладки: переменные страницы / шпаргал
<div className="shrink-0 border-b border-slate-800
 <div className="flex items-center gap-1 px-2 pt
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5
 stripTab === "vars" ? "bg-slate-800 text-
 }`}
 >
 <Variable className="w-3.5 h-3.5" /> Перемен
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5
 stripTab === "hints" ? "bg-slate-800 text
 }`}
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подс
 </button>
 <div className="ml-auto pr-1">

```

```

text-slate-300 max-w-full">
 {chapterTitle}
 </div>
 {desyncedFrom !== null && (
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страница
т заменён). "
 >
 <button
 type="button"
 onClick={resync}
 className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
 >
 <Unlink className="w-3.5 h-3.5 shrink
Страница сменилась – подставить её пер
 </button>
 </Tooltip>
)}
 {hasViz && (
 <div className="flex flex-wrap items-cent
 {sync.variables.map((v) => (
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bol
 <code className="font-mono te

 {v.role
 {v.range && <span className="bl

 }
 }
 >

```



```

 >
 <Plus className="w-3 h-3 text-slate-600"
 {s.label}
 </button>
 </Tooltip>
)})
</div>
</div>
)})
<p className="text-[10px] text-slate-600">Начало
</div>
})
</div>

```

```

{/* — Редактор (всегда на месте) + трасса + ввод */}
<div className="flex-1 min-h-0 flex flex-col">
 {/* Заголовок редактора: сразу видно, что выполняется */}
 <div className="shrink-0 flex items-center gap-1">
 <Pencil className="w-3.5 h-3.5 text-emerald-400" />

 Редактор • выполняется этот текст

 {state && (
 <span
 className="inline-flex items-center gap-1"
 font-weight="bold"
 text="amber-300"
 shrink="0"
 title="Трасса ниже построена по предыдущему состоянию"
 >
 <Loader2 className="w-2.5 h-2.5 animate-spin" />

)}

 {code.trim() === "" ? "пусто — ничего не выведено" : ""}

 </div>

 <textarea
 ref={editorRef}
 />

```

```

 <Tooltip content="В начало трассы">
 <button type="button" aria-label="В нача
 }} disabled={!curDebug?.step} className="p-1
 transition-colors">
 <SkipBack className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←) – автопро
 <button type="button" aria-label="Шаг на
 bled={!curDebug?.prev} className="p-1 round
 tion-colors">
 <ChevronLeft className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг вперёд (→) – автопро
 <button type="button" aria-label="Шаг в
 abled={!curDebug?.step || debug.idx >= debu
 e hover:bg-slate-800 disabled:opacity-40 tr
 <ChevronRight className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="В конец трассы">
 <button type="button" aria-label="В кон
 result.steps.length - 1); }} disabled={!cur
 d text-slate-400 hover:text-white hover:bg-
 <SkipForward className="w-4 h-4" />
 </button>
 </Tooltip>
 <span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>" &
 </>
) : (
 "шагов нет – исправьте ошибку и начини
)}


```

```

 aria-label="Остановить прогон трассы"
 onClick={stopDebug}
 className="inline-flex shrink-0 items-center gap-1
ext-[10px] font-bold text-rose-300 transition-colors
 >
 <CircleStop className="w-3.5 h-3.5" />
 стоп
 <kbd className="rounded border border-gray-200
 </button>
 </Tooltip>
</div>

{traceOpen && (
 <>
 {/* листинг с подсветкой текущей строки */}
 <div ref={listRef} className="flex-1 min-h-[200px]
 {debug.src.split("\n").map((line, i) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.line;
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-3 py-2
 active ? "bg-emerald-500/15 text-emerald-300" :
 }`}
 >
 <span
 className={`w-7 shrink-0 select-none text-right
 active ? "text-emerald-300" : "text-gray-400"
 }`}
 >
 {active ? "▶" : ln}

 {line || " "}
 </div>
);
 })}
)}

```

```

 }}}
 </div>
 </div>
 </>
})
</div>
})
{debug && !showTrace && debug.result.error ? {
 <pre className="shrink-0 max-h-28 overflow-auto py-2 font-mono text-[11px] leading-snug text-gray-800">
 {debug.result.error}
 </pre>
} : null}
<div className="shrink-0 border-t border-slate-800">
 <label htmlFor="python-stdin" className="text-slate-800">
 stdin для input()
 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono text-slate-800"
 />
</div>
</div>

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800">
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between gap-2 text-slate-800"
 >
 Вывод
 {outputOpen ? "⌵" : "⌴"}
 </button>
 <pre
 className="shrink-0 max-h-40 overflow-auto py-2 font-mono text-[11px] leading-snug text-slate-800"
 style={outputOpen ? {} : {display: "none"}}
 >
 {output}
 </pre>
</div>

```

```

 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
 { name: 'Студент Вася', emoji: '🍷', color: 'from-blue',
 bubbleText: 'голода после матана!' },
 { name: 'Кодер Семён', emoji: '🍲', color: 'from-violet',
 bubbleText: 'ищу ИИ за порцию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 bubbleText: 'есть сметаны, плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
 bubbleText: 'есть сметаны, плиз.' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
 bubbleText: 'есть сметаны, плиз.' },
 { name: 'Бизнесмен', emoji: '💼', color: 'from-slate',
 bubbleText: 'есть сметаны, плиз.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍷', color: 'from-blue',
 bubbleText: 'голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'from-violet',
 bubbleText: 'ищу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 bubbleText: 'есть сметаны, плиз.', animState: 'idle' },
]);

 const runtime = useVizRuntime();
 const liveQ = vizArray(runtime?.variables?.q);
 const liveV = vizString(runtime?.variables?.v);
 const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: value.name,
 emoji: value.emoji,
 color: value.color,
 bubbleText: value.bubbleText,
 animState: value.animState
 })) : [];

```

```

 setQueue(prev => prev.map(c => c.id === newGuest.id ?
 }, 450);
 }, [queue, isServing]);

```

```

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('⚠ DEQUEUE: В очереди никого нет! Повар скучает');
 setTimeout(() => setShake(false), 400);
 return;
 }

```

```

 setIsServing(true);
 const headGuest = queue[0];

```

```

 addLog(`  DEQUEUE: Повар наливает суп первому – ${headGuest.name}`);

```

```

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {

```

```

 setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue shake
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {

```

```

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

```

```

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(`  ${headGuest.name} поел и ушел сытым. 🍴`);
 }, 400);
 }, 900);
 }, [queue, isServing]);

```

```

 }, [queue, isServing]);

```

```

const reset = () => {
 counter = 3;

```

```

 iver:scale-95 transition-all cursor-pointer
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 opacity-40 disabled:cursor-not-allowed text
 :scale-95 transition-all cursor-pointer fle
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12

 {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
 <div className={`md:col-span-9 bg-slate-900 rounded
 -[300px] overflow-hidden ${shakeEmpty ? 'anim
 <div className="absolute top-4 left-4 text-[1.2em]
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ && <span className="ml-2 text-emerald-500
 </div>

 <div className="flex-1 flex flex-col justify-between
 <div className="flex items-end justify-between

```

```

key={customer.id}
className={`
 flex flex-col items-center gap-4
 ${customer.animState === 'in' ? 'opacity-100' : 'opacity-0'}
 ${customer.animState === 'eat' ? 'opacity-100' : 'opacity-0'}
 ${customer.animState === 'out' ? 'opacity-100' : 'opacity-0'}
`}
>
 {/* Облачко мыслей */}
 <div className={`
 absolute -top-16 left-1/2 -transform translate-x-50%
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 border-1px solid black' : 'opacity-0'}
 `}>
 <div className="font-bold text-slate-200">
 {customer.bubbleText}
 <div className="absolute -bottom-16 left-1/2 -transform
 it rotate-45"></div>
 </div>

 {/* Персонаж */}
 <div className={`
 w-14 h-14 rounded-2xl border-1px solid black
 relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : 'ring-0'}
 `}>
 {customer.name}

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[0.8em]
 ${isHead ? 'bg-amber-500' : 'bg-slate-200'}
 `}>
 {isHead && isTail ? 'h+t' : isHead ? 'h' : isTail ? 't'}

)}
 </div>

```



```

 className="w-full py-1.5 px-3 rounded-
0 flex items-center gap-2 text-[11px] text-
 >
 {item}
 </div>
)))
 </div>
})
</div>

{/* Пол */}
<div className="w-full h-3 bg-gradient-to-r f
</div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-indigo-5
 {entry}
 </div>
 })}
 </div>
</div>
);
};

```

```

const vars = runtime?.variables;

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveWordsRaw = vars ? vizArray(vars.words) ?? v
const liveWordsSig = useMemo(() => (liveWordsRaw ? JS
const liveState = vars ? vizNumber(vars.state) ?? viz
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
const liveQueue = vars ? vizArray(vars.queue) : null;
const liveNodes = vars ? vizNumber(vars.nodes) : null
const liveFail = vars ? vizRecord(vars.fail) : null;

// набор образцов из кода перестраивает бор целиком
useEffect(() => {
 if (!liveWordsSig) return;
 const parsed = (JSON.parse(liveWordsSig) as unknown
 .map((w) => (typeof w === "string" ? w : typeof w
 .filter((w) => Boolean(w))
 .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g,
 .filter((w) => w.length > 0)
 .slice(0, 8);
 if (!parsed.length) return;
 if (parsed.join("|") === words.join("|")) return;
 setWords(parsed);
 buildInitialTrie(parsed);
 // eslint-disable-next-line react-hooks/exhaustive-
}, [liveWordsSig]);

// состояние автомата и шаг BFS – подсвечиваем вершин
useEffect(() => {
 if (liveState !== null && liveState >= 0 && nodes[l
 if (liveStep !== null && liveStep >= 0) {
 // simulationStep – грубый статус: 0 = бор готов,
 const total = Object.keys(nodes).length;
 const failReady = liveFail ? Object.keys(liveFail
 setSimulationStep(liveStep <= 0 ? 0 : failReady ?

```

```

 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {

```

```

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить!');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов!'
);
 }
};

```

```

 setSpeechText(logs);
 };

 return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border" >
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400" >
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2" >
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate-700 text-white
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
 </div>

 {/* Верхняя панель: Говорящий лосось и диалоговое
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4" >
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center justify-center" >
 <div className="w-40 h-40 relative flex items-center justify-center" >
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg

```

```

 <path d="M 165 105 Q 155 120 165 125 Q
) : (
 <path d="M 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="M 130 80 Q 125 100 130 120" stro
 <path d="M 120 85 Q 115 100 120 115" stro
</svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
</div>
 <span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
 tween min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -tra
 slate-800 border-b-[12px] border-b-transpar

 <div className="flex items-center space-x-2 t
 <Volume2 className={`w-4 h-4 ${isTalking ?
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leadin
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-sla
 <div className="text-xs text-slate-400 flex
 <Info className="w-3.5 h-3.5 text-blue-40

```

```

 { /* SVG Canvas for Automaton Tree */ }
 <div className="mb-8 bg-slate-900 border border-s
 <h5 className="text-white font-bold mb-3 flex i
 Дерево и Суффиксные (fail) ссы
 </h5>

 {(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 { /* Draw Edges */ }
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x ||

 const isCurrent = currentNode === node.

 return (
 <g key={`edge-${node.id}`}>
 <line

```

```

- 16}` }
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
}}

```

```

{/* Draw Nodes */}
Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node.id;
 const isInQueue = bfsQueue.includes(node.id);

```

```

 let fill = '#1e293b';
 let stroke = '#475569';

```

```

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

```

```

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}

```



```

<div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-
er gap-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:tex
 title="Удалить"
 >
 ×
 </button>

)}}
</div>
</div>

```

```

<form onSubmit={handleAddWord} className="fle
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.valu
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-sla
 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-60
 ition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>

```

```

 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }` }
>
<td className="py-2.5 px-3 flex items-center" >
 <span
 className={`w-2 h-2 rounded-full
 ${node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'}
 }` }
 >
 #{node.id}
 {node.id === 0 &&
 </td>
<td className="py-2.5 px-3">

 {node.char}

</td>
<td className="py-2.5 px-3 text-slate-500" >
 {node.id === 0 ? '-' : `#${node.id}</td>
<td className="py-2.5 px-3" >
 {node.id === 0 ? (

) : (

 #{node.fail}

)}
</td>
<td className="py-2.5 px-3" >
 {node.id === 0 || node.term line

```

```

 {compilerLinked ? (
 <
 {liveWordsSig && words = {words.join(", ")}}
 {liveNodes !== null && вершин в боре = {liveNodes}}
 {liveState !== null && state = {liveState}}
 {liveStep !== null && шаг BFS = {liveStep}}
 {liveQueue && queue = {JSON.stringify(liveQueue).slice(0, 100)}}
 </>
) : (

 открой панель Python: массив <code className="text-[11px] font-mono text-cyan-300">state</code>/<code className="text-[11px] font-mono text-cyan-300">step</code>

)}
 </div>
 </div>
);
};

```

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizStore';

```

```

// ===== TYPES =====
interface Step {
 id: number;
 desc: string;
}

```

```

 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
};
void CODE;

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r}`,
 lightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] =`,
 tate: { ...tree }, highlightNodes: [v], mergeChildren: [l, r] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень:`,
 Highlight: [0, n - 1] });
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];

```

```

let id = 0;
let l = qL + n;
let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, codeLine: 3, treeState: tree, highlightNodes: [l] });

while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный`, codeLine: 6, treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR], result: res });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, codeLine: 7, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res });
 l++;
 }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`, codeLine: 8, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

// ===== TREE RENDERING HELPER =====
interface VNode { v: number; l: number; r: number; val: number; }

```

```

" // -----",
" if (r & 1) res += tree[--r];",
" l >>= 1; r >>= 1;",
" return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-30 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' } }[color];

 return { step, idx, setIdx, playing, setPlaying, speed, clr };
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr: arr, arrLen: arrLen }) {
 const p = Player({ steps, color });

```

```

<div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'}
 {/* Header */}
 <div className={`px-4 py-2.5 flex items-center justify-between
 -emerald-950/50' : 'bg-amber-950/50'} border-bottom
 <h4 className="font-bold text-sm text-white flex-1"
 <span className="text-[10px] font-mono text-slate-500"
 </div>

```

```

 {/* Array strip */}
 <div className="px-3 py-2 bg-slate-950 border-bottom border-amber-500/30"
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= step.start
 return (
 <div key={i} className={`flex flex-col items-center justify-between
 go' ? 'bg-indigo-950 border-indigo-500' : 'bg-slate-950 border-amber-500/30'}
 -translate-y-0.5` : 'bg-slate-900 border-amber-500/30'}
 <span className="text-[7px] text-slate-500"
 <span className="text-[11px] font-bold font-sans text-white"
 </div>
);
 })}
 </div>

```

```

 {/* Tree */}
 <div className="flex-1 p-2 overflow-x-auto bg-slate-950"
 <div className="scale-[0.85] origin-top min-w-max"
 </div>

```

```

 {/* Code */}
 <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono"
 x">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 rounded
 2 border-indigo-500 text-indigo-200' : 'text-slate-500'}

```

```

 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
 </div>

 { /* Progress bar */ }
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
 </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const runtime = useVizRuntime();
 const liveI = vizNumber(runtime?.variables?.i);
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8,');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 }
}

```



```

</div>
<div className="flex flex-wrap gap-1.5 pt-1" >
 {arr.map(({v, i}) => (
 {v}
 {i}

))}
 {arr.length}
</div>
</form>

```

```

{/* Query range */}
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800" >
 <label className="text-[10px] font-bold uppercase" > Query range
 <div className="flex items-center gap-2" >
 <div className="flex items-center gap-1" >
 Range
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1" >
 Range
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 </div>

```

```

 arr={arr}
 />
 })

{view === 'queries' && {
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`}
 icon="↑"
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`}
 icon="↓"
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
})

{view === 'theory' && {
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl border"
 <h4 className="text-base font-bold text-white">
 Почему разбиение именно так для N={arr.length}?
 </h4>
 <p className="text-sm text-slate-300 leading-relaxed">
 Каждый узел берёт <code className="bg-slate-900 px-1">
 и получает <code className="bg-slate-900 px-1">
 -slate-900 px-1 rounded font-mono text-emerald-400
 евая часть забирает на 1 больше (округление

```



```

 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 {!compact && (
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute left-1/2 top-1/2 -tran
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам.."
 className="w-full bg-slate-800/70 border border-bo
 e-500 focus:outline-none focus:border-indigo
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -tran
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
 </div>
)}
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4">
 {groups.length === 0 && <p className="text-sm text-slate-500">
 Нет результатов
 }

 {groups.map(({ section, items }) => (
 <div key={section.id}>
 <div className="flex items-baseline gap-1.5">

 {section.from > 0 && (

 {section.from}

)}
 </div>
 <div className="space-y-1">
 {items.map((chapter) => {

```

```

 />
 })
 </button>
);
 }}}
</div>
</div>
)))
</div>

<div className={`shrink-0 m-3 mt-0 bg-gradient-to
 x] text-slate-300 space-y-1.5 hidden lg:block $
 <div className="font-bold text-indigo-400 flex .
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 <b className="text-slate-200">Порядок чтения
 номерам билетов 1 → 24, каждая опирается на п
 Следующая тема» продолжают этот же порядок.
 </p>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
</aside>
);
};

```

---

ФАЙЛ 69/111: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

```

```
// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный"',
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный"',
 { id: '3', text: 'Кандидат: "Привет, я испанский ин
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.1);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: '🥞' },
 { name: 'Кастрюля от супа', icon: '🍲' },
 { name: 'Чашка от кофе', icon: '☕' },
 { name: 'Миска с остатками салата', icon: '🥗' },
 { name: 'Тарелка от тако', icon: '🌮' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(`Положили поверх стопки: ${newPlate.name} ${randomPreset.icon}`);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name} ${topPlate.icon}`);
};
```

```

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:

```

```

 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">LIFO
 </div>
 </div>
 <span className="text-xs bg-blue-500/20 text-l
 DFS

 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between
 activeTab === 'queue'
 ? 'bg-indigo-600/20 border-indigo-500 tex
 : 'bg-slate-800/60 border-slate-700/60 te
 }`
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab :
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>
 </div>
 <span className="text-xs bg-indigo-500/20 tex
 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 t
 : 'bg-slate-800/60 border-slate-700/60 te
 }`
 >

```



```

border-blue-500/40 px-4 py-2.5 rounded-xl
>
 Помыть верхнюю
</button>
<button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
>
 <RotateCcw className="w-5 h-5" />
</button>
</div>
</div>

{popMessage && {
 <div className="bg-blue-950/60 border borde
imate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {popMessage}
 </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
>
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стон
 <p className="text-xs mt-1">Добавьте гр
 </div>
) : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 ani

```

```

{ /* QUEUE TAB */
{activeTab === 'queue' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция очереди за супом
 <span className="text-xs font-mono bg-in
 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Кто первым пришел, тот первым получил с
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex ite
 -2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex ite
 er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>

```

```

 isFirst
 ? 'bg-gradient-to-b from-indigo-500 to-slate-900'
 : 'bg-slate-900/90 border-slate-900'
 }` }
 >
 {isFirst && {

 Первый (Head)

 }}
 {
 <span className={`font-bold text-slate-900`}
 {person.name}

 <span className={`text-[10px] font-mono text-slate-900`}
 {isFirst ? 'bg-indigo-500/30 text-white' : 'bg-slate-900/30 text-slate-900'}

 }`}>
 </div>
 </div>
);
}}}

<div className="flex flex-col items-center justify-center gap-6 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 ПЕРВЫЙ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • ИЛИ ПЕРВЫМ ПОЛУЧИЛ → ПЕРВЫЙ ПРИШЕЛ (LIFO)
 </div>
</div>
)}
<div className="mt-6 text-xs text-slate-500">
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • ИЛИ ПЕРВЫМ ПОЛУЧИЛ → ПЕРВЫЙ ПРИШЕЛ (LIFO)
</div>
</div>
)}

```

```

<div className="md:col-span-6">
 <label className="block text-xs font-bold">
 <input
 type="text"
 value={newCustomText}
 onChange={e => setNewCustomText(e.target.value)}
 placeholder="например: "Привет, я лучший!"
 className="w-full bg-slate-900 border border-emerald-500 transition-colors"
 />
 </div>
<div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
</div>
<div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
</div>
</form>

```

```

 <span className={`flex items-center
 isTop ? 'bg-emerald-500 text-
 }`}>
 #{index + 1}

 <div>
 <div className={`font-bold te
 {item.text}
 </div>
 {isTop && (
 <span className="text-[10px
-0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Буд

)}
 </div>
 </div>

 <div className="flex items-center
 /* Custom progress bar */
 <div className="hidden sm:block
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%
 ></div>
 </div>
 <span className={`font-mono font
 isTop ? 'text-emerald-400 tex
 }`}>
 {percent}%

 </div>
</div>
);
 }}}}
</div>
))
<div className="mt-6 text-xs text-slate-500

```

```
}, [chapterId, tab]));
```

```
return (
```

```
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
```

```
 <div className="flex flex-wrap items-center justify-center" >
```

```
 <div className="flex items-center gap-3" >
```

```
 <div className="p-2.5 bg-indigo-600 text-white" >
```

```

```

```
 </div>
```

```
 <div>
```

```
 <h3 className="text-2xl font-bold text-white" >
```

```
 <p className="text-sm text-indigo-300" >Один
```

```
 </div>
```

```
 </div>
```

```
 </div>
```

```
 <div className="flex flex-wrap gap-2 mb-5" >
```

```
 {[
```

```
 { id: "cases" as Tab, label: "1 · Механика: Z"
```

```
 { id: "big" as Tab, label: "2 · Большое дерево"
```

```
 { id: "play" as Tab, label: "3 · Собери сам"
```

```
]).map((t) => {
```

```
 <button
```

```
 key={t.id}
```

```
 onClick={() => setTab(t.id)}
```

```
 className={`px-4 py-2 rounded-lg text-xs font-medium
```

```
 ${tab === t.id ? "bg-indigo-600 text-white" : ""}
```

```
 `}
```

```
 >
```

```
 {t.label}
```

```
 </button>
```

```
])
```

```
 </div>
```

```
 {tab === "cases" && <SplayRotationsViz />}
```

```
 {tab === "big" && <SplayWalkViz />}
```

```
 {tab === "play" && <SplayRotationSandbox />}
```

```

 t ? { key: t.key, left: clone(t.left), right: clone(t
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
 },
 {
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);

```

```

 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя)" };
}

/* ----- раскладка -----

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);

```



```

 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)
const expected = useMemo(() => (solved ? null : canonicalNext(tree, target));
setHistory((h) => [...h, clone(tree) as TNode]);
setMoves((m) => [...m, { node: key, correct: want?.correct }]);
setTree((t) => rotateUp(clone(t) as TNode, key));
});

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return (
 <div className="w-full bg-slate-950 rounded-2xl border-slate-700">
 <div className="flex items-start justify-between p-4">
 <div>
 <h4 className="text-sm sm:text-base font-bold">
 <p className="text-[12px] text-slate-400 lead">
 Клик по узлу = один поворот с его родителем

```

```

 const onPath = path.has(a) && path.has(b);
 return {
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 }
}

{nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return {
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && {
 <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
 }}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.has(a) && path.has(b) ? "#a5b4fc" : "#475569"}
 stroke={isTarget ? "#a5b4fc" : "#475569"}
 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle" />
 </g>
 };
})}

```

```


 </div>

 {showAnswer && !solved && expected && {
 <div className="mt-3 text-[12px] bg-amber-950/40" >
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
 }}

 {solved && {
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-50"
 >
 <div className="flex items-center gap-2 font-mono" >
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500" /> : <X className="w-4 h-4 text-rose-500" />}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов`}
 </div>
 {wrong > 0 && {
 <ul className="list-disc pl-5 space-y-0.5" >
 {moves.map(
 (m, i) =>
 !m.correct && {
 <li key={i}>
 ход {i + 1}: крутили {m.expected}

 }
)
)}

 }}
 >
 {wrong === 0 && moves.length > minimal && {

```

```

edges: [NodeId, NodeId][];
kind: "start" | "aim" | "result";
title: string;
text: string;
/** Пара, которую крутим на этом шаге: кадр-«прицел». */
focus?: [NodeId, NodeId];
/** Кто переехал на этом кадре – рисуем призрак и стрелку. */
moved?: NodeId[];
ms: number;
}

interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",

```

```
edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
] as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
] as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крутит
focus: ["g", "p"],
 ms: AIM_MS,
 },
 {
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему почти
наравне с 20"
```

```

key: "zig-zag",
label: "Zig-Zag",
when: "x и p – дети с РАЗНЫХ сторон (змейка)",
rotations: "2 поворота, первым – нижний",
keys: { x: 40, p: 20, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и",
 focus: ["g", "x"],
 }
]

```

```

<svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

```

*{/\* призраки: где узел стоял до поворота \*/}*

```

{ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill=
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
);
 })}
})))

```

```

{edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];

```

```

 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font
 {id}
 </text>
)}
 </g>
);
}}
</svg>
);
};

```

```

/** Имя случая из кода пользователя → вкладка демонстра
const CASE_BY_NAME: Record<string, number> = {
 zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-z
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

```

```

/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveCase = vars ? vizString(vars.case) ?? vizSt
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(
const lastSig = useRef<string | null>(null);

```



```

return (
 <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
 {CASES.map((c, i) => (
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-sm transition-colors
 i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"
 }`}
 >
 {c.label}
 </button>
))}
 </div>

 <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корреляция
 ({active.rotation}°)
 </div>

 {/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
 <div className="mb-3 rounded-xl border border-slate-800">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase transition-colors
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`}
 >
 {current.kind === "aim" ? "прицел" : current.kind === "start" ? "начало" : "конец"}

 Шаг {idx + 1} из {total}. {current.title}
 </div>
 </div>
 </div>
)

```

```
</div>
```

```
<div className="flex flex-wrap items-center gap-2"
```

```
 <button
```

```
 onClick={() => {
```

```
 setPlaying(false);
```

```
 setFrame((f) => (f - 1 + total) % total);
```

```
 }}
```

```
 disabled={idx === 0}
```

```
 className="px-3 py-2 rounded-lg bg-slate-800 l
```

```
 d: hover: text-slate-300 text-xs font-bold tr
```

```
 >
```

```
 ← Назад
```

```
</button>
```

```
<button
```

```
 onClick={() => {
```

```
 setPlaying(false);
```

```
 setFrame((f) => (f + 1) % total);
```

```
 }}
```

```
 className="px-4 py-2 rounded-lg bg-indigo-600
```

```
 w-indigo-900/40"
```

```
>
```

```
 {last ? "⏮ Сначала" : "Дальше →"}
```

```
</button>
```

```
<button
```

```
 onClick={() => setPlaying((p) => !p)}
```

```
 className="flex items-center gap-1.5 px-3 py-1
```

```
 ext-xs font-bold transition-colors"
```

```
>
```

```
 {playing ? <Pause className="w-3.5 h-3.5" />
```

```
 {playing ? "Стоп" : "Авто"}
```

```
</button>
```

```
<button
```

```
 onClick={() => setSlow((s) => !s)}
```

```
 title="Автопрокрутка ещё медленнее"
```

```
 className={`flex items-center gap-1.5 px-3 py-1
```

```
 slow
```

```
 ? "bg-emerald-600/20 border-emerald-500 tr
```

```

 })
 {liveX !== null && <span className="text-[1
py-1">x = {liveX}}
 {liveP !== null && <span className="text-[1
py-1">p = {liveP}}
 {liveG !== null && <span className="text-[1
py-1">g = {liveG || "нет (Zig)"}}
 {liveStep !== null && <span className="text
-2 py-1">step = {liveStep}}
 </>
) : (

 запусти код из панели Python: переменная <с
случай,
 а <code className="font-mono text-indigo-300

)}
</div>

<p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры н
</p>
</div>
);
};

```

ФАЙЛ 73/111: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useMemo, useRef, useState } from 'react';
```

```
/**
```

```
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
```

```
 *
```

```
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный пов
```

```

 if (!cur.left) {
 cur.left = { key, left: null, right: null, parent: r };
 return r;
 }
 cur = cur.left;
 } else {
 if (!cur.right) {
 cur.right = { key, left: null, right: null, parent: r };
 return r;
 }
 cur = cur.right;
 }
}
}
}

```

/\*\* Поднять child над его родителем (один атомарный пов

```

export function rotateUp(child: SN): void {
 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

```

/\*\* Рёбра в виде "p→c" — для diff между кадрами. \*/

```

 rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
 rootIn: SN | null,
 key: number,
 isInsert = false,
): { steps: SplayStep[]; found: boolean } {
 let root: SN | null = cloneWithParents(rootIn);
 const steps: SplayStep[] = [];
 if (!root)
 return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
 cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск унёлся в ${last.key}

```

```

const diff = diffEdges(before, collectEdges(root));
steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: label,
 note: `Поворот: ${child.key} встал над ${p.key}.`,
 rotCount,
});
};

while (target.parent) {
 const p = target.parent;
 const g = p.parent;
 if (!g) {
 emitRotation(
 target,
 "Zig",
 [p.key, target.key],
 `Zig: у ${target.key} нет деда — хватит одного
);
 continue;
 }
 const pIsLeft = g.left === p;
 const xIsLeft = p.left === target;
 if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа"
 emitRotation(
 p,
 "Zig-Zig",
 [g.key, p.key],
 `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta
 ${g.key}, а ${target.key} пока не трогаем.`
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",

```

```

 rotCount,
 });
 return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number =>
 let d = 0;
 let cur = t;
 while (cur) {
 if (key === cur.key) return d;
 cur = key < cur.key ? cur.left : cur.right;
 d += 1;
 }
 return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [
 ...inorderKeys(t.left),
 t.key,
 ...inorderKeys(t.right)
] : []);

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12];

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
 return t!;
};

const CODE_LINES = [
 { line: 1, text: "while x has parent:" },
 { line: 2, text: " p = parent(x); g = parent(p)" },
 { line: 3, text: " if g is None: # Z" },
 { line: 4, text: " rotate(x over p)" },
 { line: 5, text: " elif p,g,x на одной линии: # " },
 { line: 6, text: " rotate(p over g) # B" },
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z" },
 { line: 9, text: " rotate(x over p) # H" }
];

```

```

/>
<text x="48" y="130" textAnchor="middle" fill="black">
<circle cx="80" cy="126" r="13" fill="#0f172a" stroke="black" stroke-width="2">
<text x="80" y="130" textAnchor="middle" fill="black">
<text x="98" y="92" fill={isCut ? "#f59e0b" : "black"}>
 {isCut ? "β отстёрнута" : "β при x"}
</text>
<text x="16" y="160" fill="#64748b" font-size="12">
</g>
<text x="163" y="96" fill="#10b981" font-size="24">
<g transform="translate(186,6)">
 <text y="10" fill="#94a3b8" font-size="11" font-weight="bold">
 <line x1="70" y1="46" x2="45" y2="86" stroke="black" stroke-width="2">
 <line x1="70" y1="46" x2="95" y2="86" stroke="black" stroke-width="2">
 <line x1="45" y1="86" x2="27" y2="126" stroke="black" stroke-width="2">
 <line
 x1="45" y1="86" x2="63" y2="126"
 stroke={isRot ? "#10b981" : "#64748b"}
 stroke-width={isRot ? 3 : 2}
 />
 <circle cx="70" cy="46" r="15" fill="#059669" stroke="black" stroke-width="2">
 <text x="70" y="50" textAnchor="middle" fill="black">
 <circle cx="45" cy="86" r="15" fill="#1e293b" stroke="black" stroke-width="2">
 <text x="45" y="90" textAnchor="middle" fill="black">
 <circle cx="27" cy="126" r="13" fill="#0f172a" stroke="black" stroke-width="2">
 <text x="27" y="130" textAnchor="middle" fill="black">
 <circle
 cx="63" cy="126" r="13"
 fill={isRot ? "#78350f" : "#0f172a"}
 stroke={isRot ? "#10b981" : "#64748b"}
 stroke-width={isRot ? 2.5 : 1.5}
 />
 <text x="63" y="130" textAnchor="middle" fill="black">
 <circle cx="95" cy="126" r="13" fill="#0f172a" stroke="black" stroke-width="2">
 <text x="95" y="130" textAnchor="middle" fill="black">
 <text x="112" y="92" fill={isRot ? "#10b981" : "black"}>
 {isRot ? "↓ β теперь при p" : "β при p"}
 </text>

```



```

const baseRef = useRef<SN>(initial);
const timer = useRef<ReturnType<typeof setTimeout> | null>(null);

const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
};

useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 baseRef.current = steps[steps.length - 1].root ?? null;
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v + 1), 100);
 return () => {
 if (timer.current) clearTimeout(timer.current);
 };
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length - 1)] : null;
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot]);
const nodes = useMemo(() => [...pos.entries()].map(([key, node]) => node), [pos]);
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
}, [viewRoot]);

```

```

 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e[0] === k) return "#1e293b";
 if (e[1] === k) return "#78350f";
 }
 if (step.kind === "rot" && step.pathKeys?.includes(k))
 return "#0f172a";
};

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind === "search" && step.pathKeys?.includes(k))
 return "#f59e0b";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e.includes(k)) return "#f59e0b";
 }
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 { /* Wanka */ }
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Клик
 </p>
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3" >
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {

```

```

 Перемешать
 </button>
 </div>
 </div>

 { /* Плеер */
 {steps && {
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setIdx(0); setPlaying(true); }}
 lg:bg-slate-700 text-white disabled:opacity-50
 В начало
 </button>
 <button onClick={() => { setPlaying(false); setIdx(0); }}
 lg:text-xs font-bold bg-slate-700 text-white
 ⏮
 </button>
 <button onClick={() => setPlaying(!playing)}
 lg:px-6 lg:py-1.5 rounded-lg text-white
 {playing ? "Пауза" : "▶ Пуск"}
 </button>
 <button onClick={() => { setPlaying(false); setIdx(0); }}
 className="px-3 py-1.5 rounded-lg text-xs font-medium
 ⏭
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && {

 {step.caseLabel}

 }}
 </div>
 }}
}

{ /* Дерево */
<div className="bg-indigo-950/20 rounded-xl p-4 b

```

```

 <g
 key={`n-${n.key}-${treeVers}`}
 transform={`translate(${n.px}, ${n.py})`}
 onClick={() => {
 if (playing) return;
 const has = inorderKeys(baseRef.current);
 if (has) runKey(n.key);
 }}
 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="rgba(0,0,0,0.1)" fontSize="13"
 >
 {n.key}
 </text>
 </g>
);
}}
</svg>

```

```

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row" >
 <div className="flex-1 bg-slate-950/80 p-4 rounded" >
 <p className="font-semibold text-amber-400" >
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "∞ Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 }
 </p>
 </div>
</div>

```

```

 </tr>
 </tbody>
 </table>
) : (
 <p className="text-xs text-slate-400">Запис
 })
 <p className="text-xs text-slate-500 mt-3">Од
 щений дешевлеет.</p>
</div>
<div className="w-full lg:w-1/2 bg-slate-900/60
 <h4 className="text-sm font-bold text-slate-2
 <div className="bg-slate-950/80 rounded-lg p-
 {CODE_LINES.map((l) => {
 const zig = step?.caseLabel === "Zig" &&
 const zz1 = step?.caseLabel === "Zig-Zig"
 const zz2 = step?.caseLabel === "Zig-Zig"
 const za1 = step?.caseLabel === "Zig-Zag"
 const za2 = step?.caseLabel === "Zig-Zag"
 const active = zig || zz1 || zz2 || za1 |
 return (
 <div key={l.line} className={`py-1 px-2
ber-400" : "text-slate-400"}`}>
 <span className="text-slate-600 selec
 {l.text}
 </div>
 </div>
);
 })}
 </div>
</div>
</div>
</div>
);
};

export default SplayWalkViz;

```

---

```

const runtime = useVizRuntime();
const liveStack = vizArray(runtime?.variables?.st) ??
const displayStack: Plate[] = liveStack
 ? liveStack.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: " ",
 color: "from-indigo-500/20 to-indigo-600/20 border",
 isDirty: true,
 animState: "idle",
 }))
 : dirtyStack;

const [cleanRack, setCleanRack] = useState<Plate[]>([]);
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
}, [isWashing]);

const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
counter++;

const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,

```

```

 setCleanRack(prev => [cleanPlate, ...prev].slice(0, cleanPlate.length - 1));
 setDirtyStack(prev => prev.slice(0, prev.length - 1));
 setShowSoap(false);
 setIsWashing(false);
 addLog(`    Готово! Чистая тарелка ${topPlate.emoji}`, 850);
 }, [dirtyStack, isWashing]);

```

```

const reset = () => {
 counter = 0;
 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', state: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
};

```

```

const topIndex = displayStack.length - 1;

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">Правило!</h3>
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно последнюю.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>
 </div>
)

```

```

<div className="md:col-span-7 bg-slate-900 rounded-[380px]">
 <div className="absolute top-4 left-4 text-[10px] text-white">
 Грязная стопка (Стек вызовов / LIFO)
 </div>

 {/* Визуализация раковины */}
 <div className="absolute top-4 right-4 flex items-center justify-center gap-2 bg-slate-800/80 text-[10px] font-mono">

 РАКОВИНА
 </div>

 {/* Мыльные пузыри при мытье */}
 {showSoap && {
 <div className="absolute inset-0 pointer-events-none">
 <div className="absolute w-6 h-6 rounded-full bg-white"></div>
 <div className="absolute w-4 h-4 rounded-full bg-white"></div>
 <div className="absolute w-5 h-5 rounded-full bg-white"></div>
 <div className="absolute text-4xl anim-crash"></div>
 </div>
)}

 {displayStack.length === 0 ? {
 <div className="flex-1 flex flex-col items-center justify-center gap-2">
 0
 <p className="text-white font-black text-2xl">Пустая стопка</p>
 <p className="text-slate-500 text-xs mt-1">Создайте задачу</p>
 </div>
 } : {
 <div className={`flex-1 flex flex-col justify-center gap-2`}
 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-center justify-center gap-2"
 >

```





```

const pi = new Array(n).fill(0);
const steps: any[] = [];

steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

for (let i = 1; i < n; i++) {
 let j = pi[i - 1];

 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j = pi[j - 1];
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записываем` });

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true });
 }
}

return { s, steps, patternLength: pattern.length };
}

function generateZSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";

```

```

 steps.push({ i, l, r, z: [...z], desc: ` 0
 }

 if (z[i] === pattern.length) {
 steps.push({ i, l, r, z: [...z], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp"
const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
const [pattern, setPattern] = useState("aba");
const [text, setText] = useState("abacaba");

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

const { s: demoString, steps, patternLength: demoPatternLength } =
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
}, [mode, safePattern, safeText]);
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?.patternLength);
const s = runtimeString ?? demoString;
const patternLength = runtimeString ? runtimeString : demoPatternLength;

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
 </div>
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
 </div>
 </div>
 </div>

 <div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
 <div className="flex gap-1.5 px-4 h-ful
 {s.split("").map((char, index) => {
 const isPattern = index < patte
 const isSeparator = index === p

```

```

 { /* Char Box */
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}
 </div>

 { /* Value array box */
 <div className="text-[1
slate-700/50 font-mono font-bold">
 {(mode === "kmp" ?
 </div>

 { /* KMP fingers */
 {mode === "kmp" && step
 <div className="abs
z-20">
 <span classNam
 </div>
 })
 {mode === "kmp" && step
 <div className="abs
 <span classNam
 </div>
 })
 {mode === "kmp" && step
 <div className="abs
20">
 <span classNam
 </div>
 })

 { /* Z fingers */
 {mode === "z" && step.i
 <div className="abs
e z-20">
 <span classNam
 </div>

```

```

 <button onClick={nextStep} disabled={
 !currentStep || currentStep === 'p-2' text-slate-400 hover:text-white hover:
 <SkipForward strokeWidth={3} size={24} />
 </button>
 <button onClick={reset} aria-label="Reset" className="bg-slate-800 rounded-lg ml-2 border-l border-slate-400"
 <RefreshCw strokeWidth={3} size={24} />
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 border border-slate-800 overflow-hidden">
 <div className="absolute top-0 right-0 text-slate-400">
 {currentStep}
 </div>
 <div className="text-[10px] text-slate-400">
 {steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl">
 {step.title}
 <p className="text-xs text-slate-400">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "применено" : "не найдено"}
 о было найдено!
 Пройди шаги вручную, посмотри, как работает алгоритм
 для пропуска работы (в Z).
 </p>
</div>

{/* Code Sneak Peek */}

```

```

 /** Элемент, на который наводят курсор / фокус. */
 children: React.ReactNode;
 /** С какой стороны показывать. По умолчанию – сверху
 side?: "top" | "bottom";
 className?: string;
 }

 /**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
 export const Tooltip: React.FC<TooltipProps> = ({ content, ...props }) => {
 const [open, setOpen] = useState(false);

 const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

 return (
 <span
 className={`relative inline-flex ${className ?? ""}
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2 top-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px]
 duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full mt-2"
 }
 ${open ? "visible translate-y-0 opacity-100" : "invisible"}
 >
 {content}
 <span
 className={`absolute left-1/2 h-2 w-2 -translate-x-1/2
 side === "top" ? "-bottom-[5px] border-b border-slate-900" : "left-[-5px] border-l border-slate-900"
 }`
 />

```

```

 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное" },
];

```

```

const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];

```

```

interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}

```

```

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];

```



```

 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} о
 u,
);
 }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write i
recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'
 ,
 u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",

```

```

</h3>

<div className="flex items-center gap-2 bg-slate-500" >
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-5 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить сортировку"
 title="Сбросить сортировку"
 className="flex items-center justify-center gap-2 px-3 py-5 500 text-white"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"

```

```

 className="transition-all duration-300"
 />
);
 }}}

```

```

{ /* Nodes */
dagNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited = step.visited.has(node.id);
 const isProcessed = step.fullyProcessed.has(node.id);

```

```

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";
 void textColor;

```

```

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }

```

```

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"

```

```

<div className="bg-slate-900 border border-slate-800 p-2.5 rounded">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded">
 pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
 </div>
 </div>

```

```

{/* Color key */}
<div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">


```

ФАЙЛ 78/111: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useContext, useEffect, useMemo, useState } from "react";
import { Terminal } from "lucide-react";
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from "viz-react";
```

```
/**
```

```
 * Интерактивный Treap – шесть режимов:
```

```
 * build – сборка: ВИДИМАЯ сортировка точек (и чем сортируем, тем быстрее)
```

```
 * координатная плоскость НАВЕРХУ (ассоциативная)
```

```
 * чек-лист инвариантов (автопроверка, как у AVL)
```

```
 * split – разрез по x с двумя корзинами (грамотная сортировка)
```

```
 * merge – склейка двух деревьев по y.
```

```
 * erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x, удалить по y.
```

```
 * layers – миф «дети в массиве по $2^k/2^{k+1}$ »: работает
```

```
 * search – «отсортирую бинарным поиском»: поиск не сортировкой
```

```
 * сортировка платит $\geq n \cdot \log n$ сравнений, сортировка
```

```
 *
```

```
 * Канонический набор: все x и все y различны $\Rightarrow$ дерево
```

```
 * Узел идентифицируется парой $id = "x:y"$.
```

```
 */
```

```
export interface Pair {
```

```
 key: number;
```

```
 grade: number;
```

```
}
```

```
type NodeId = string;
```

```
const pairId = (key: number, grade: number): NodeId =>
```

```
interface TNode {
```

```
 id: NodeId;
```

```
 key: number;
```

```
 grade: number;
```

```
 left: TNode | null;
```

```
 right: TNode | null;
```

```
}
```

```

export function buildTree(ps: Pair[]): TNode | null {
 let root: TNode | null = null;
 for (const p of sortedByY(ps)) root = insert(root, p);
 return root;
}
```

```
export interface SplitResult {
 l: TNode | null;
 r: TNode | null;
 trace: { atId: NodeId; goLeft: boolean; note: string }
}
export function splitTree(root: TNode | null, x: number): SplitResult {
 const trace: SplitResult["trace"] = [];
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
 if (!t) return { l: null, r: null };
 if (t.key <= x) {
 const sub = go(t.right);
 trace.push({ atId: t.id, goLeft: true, note: `(${t.id} делит
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.id} делит
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
 };
 const res = go(root);
 return { l: res.l, r: res.r, trace };
}
```

```
export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosen: boolean }
}
export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
 const trace: MergeResult["trace"] = [];
 const go = (t: TNode | null, x: number): TNode | null {
 if (!t) return null;
 if (t.key <= x) {
 const sub = go(t.right, x);
 trace.push({ aId: t.id, bId: null, chosen: true, note: `(${t.id} делит
 о правое поддерево.` });
 t.right = sub;
 return t;
 }
 const sub = go(t.left, x);
 trace.push({ aId: t.id, bId: null, chosen: false, note: `(${t.id} делит
 о левое поддерево.` });
 t.left = sub;
 return t;
 };
 const res = go(a, x);
 const res2 = go(b, x);
 return { root: merge(res, res2), trace };
}
```

```

 };
 return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
 t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null

```

/\* ----- валидация treap (для чек-листа и стресс-теста) ----- \*/

```

export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;
 const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null): void => {
 if (!n) return;
 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) heap = false;
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

```

/\* ----- раскладка сцены дерева ----- \*/

```

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 10 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
}

```

```

<div>
 <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full" >
 {edges.map((e) => {
 const a = byId(e.from);
 const b = byId(e.to);
 if (!a || !b) return null;
 return (
 <line
 key={`te-${e.from}-${e.to}`}
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={edgeColor ? edgeColor(e.from, e.to) : "#ccc"}
 strokeWidth={2.5}
 strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
 className="transition-all duration-500"
 />
);
 })}
 {nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate(${p[0]px, ${p[1]px})`} >
 <circle r={pulse ? 24 : 20} fill={nodeColor(n)} stroke="#ccc"
 strokeWidth={pulse ? 4 : 2} className="transition-all duration-500" />
 <text y={-2} fill="#fff" fontSize="13" fontWeight="bold">{n.id}</text>
 <text y={13} fill="#cbd5e1" fontSize="10">{p[0]} : {p[1]}</text>
 </g>
);
 })}
 </svg>
 {subtitle && <p className="text-xs text-slate-400">{subtitle}</p>}
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

```



```

<line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(1) - 8}
{[0, 10, 20, 30, 40].map((k) => {
 <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16}
})}
{[-4, 0, 4, 8].map((g) => {
 <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16}
})}
<text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8"
<text x={PX(1) - 26} y="14" fill="#94a3b8" fontSi

{drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a.key &&
 badEdge.a.grade === e.b.grade) || (badEdge.a.key === e.b.key &&
 badEdge.a.grade === e.a.grade));
 return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 strokeDasharray={isBad ? "4,3" : "none"}
 >
})}

{/* временная линия игры */}
{interactive && selected && cursor && {
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDasharray={[]}
 />
}}

{points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key === p.key && e.b.key === p.key))
 const isSel = selected && selected.key === p.key
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ? () => {
 selected = p
 }}
 onPointerUp={interactive && onPointUp ? () => {
 selected = null
 }}
 style={interactive ? { cursor: "pointer" } : {}}
 >
 <circle
 cx={PX(p.key)} cy={PY(p.grade)}
 r={10}
 fill={isSel ? "#818cf8" : "#94a3b8"}
 stroke={isSel ? "#818cf8" : "none"}
 strokeWidth={isSel ? 2.5 : 0}
 />
 <rect
 x={PX(p.key) - 10} y={PY(p.grade) - 10}
 width={20} height={20}
 fill="none" stroke="none"
 />
 </g>
)}

```

```

const gpid = (p: Pair) => `${p.key}:${p.grade}`;

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge | undefined =>
 edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pair): boolean => {
 let cur: Pair | undefined = from;
 const seen = new Set<string>();
 while (cur) {
 if (sameP(cur, target)) return true;
 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur))?.a;
 }
 return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа. */
export function forestIsBst(edges: GEdge[], all: Pair[]): boolean {
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || []), e]);
 const hasParent = new Set(edges.map((e) => gpid(e.b)));
 const roots = all.filter((p) => !hasParent.has(gpid(p)));
 const ok = (n: Pair | undefined, lo: number, hi: number): boolean => {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
 };
 return roots.every((r) => ok(r, -Infinity, Infinity));
}

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]): boolean {
 const heap = edges.every((e) => e.a.grade > e.b.grade);

```

```

 const nums = chunk.split(/[\sxx*]+/).filter(Boolean)
 if (nums.length !== 2 || nums.some((v) => !Number.isInteger(v))) {
 errors.push(`«${chunk}» – нужно ровно два числа: два числа и пробел`);
 continue;
 }
 const id = `${nums[0]}:${nums[1]}`;
 if (seen.has(id)) {
 errors.push(`точка (${nums[0]}; ${nums[1]}) дважды`);
 continue;
 }
 seen.add(id);
 pairs.push({ key: nums[0], grade: nums[1] });
 }
 const xs = pairs.map((p) => p.key);
 const ys = pairs.map((p) => p.grade);
 return { pairs, errors, equalX: new Set(xs).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0, 1 1, 10 10, 11 11, 12 12, 13 13, 14 14, 15 15, 16 16, 17 17, 18 18, 19 19, 20 20, 21 21, 22 22, 23 23, 24 24, 25 25, 26 26, 27 27, 28 28, 29 29, 30 30, 31 31, 32 32, 33 33, 34 34, 35 35, 36 36, 37 37, 38 38, 39 39, 40 40, 41 41, 42 42, 43 43, 44 44, 45 45, 46 46, 47 47, 48 48, 49 49, 50 50, 51 51, 52 52, 53 53, 54 54, 55 55, 56 56, 57 57, 58 58, 59 59, 60 60, 61 61, 62 62, 63 63, 64 64, 65 65, 66 66, 67 67, 68 68, 69 69, 70 70, 71 71, 72 72, 73 73, 74 74, 75 75, 76 76, 77 77, 78 78, 79 79, 80 80, 81 81, 82 82, 83 83, 84 84, 85 85, 86 86, 87 87, 88 88, 89 89, 90 90, 91 91, 92 92, 93 93, 94 94, 95 95, 96 96, 97 97, 98 98, 99 99, 100 100, 101 101, 102 102, 103 103, 104 104, 105 105, 106 106, 107 107, 108 108, 109 109, 110 110, 111 111, 112 112, 113 113, 114 114, 115 115, 116 116, 117 117, 118 118, 119 119, 120 120, 121 121, 122 122, 123 123, 124 124, 125 125, 126 126, 127 127, 128 128, 129 129, 130 130, 131 131, 132 132, 133 133, 134 134, 135 135, 136 136, 137 137, 138 138, 139 139, 140 140, 141 141, 142 142, 143 143, 144 144, 145 145, 146 146, 147 147, 148 148, 149 149, 150 150, 151 151, 152 152, 153 153, 154 154, 155 155, 156 156, 157 157, 158 158, 159 159, 160 160, 161 161, 162 162, 163 163, 164 164, 165 165, 166 166, 167 167, 168 168, 169 169, 170 170, 171 171, 172 172, 173 173, 174 174, 175 175, 176 176, 177 177, 178 178, 179 179, 180 180, 181 181, 182 182, 183 183, 184 184, 185 185, 186 186, 187 187, 188 188, 189 189, 190 190, 191 191, 192 192, 193 193, 194 194, 195 195, 196 196, 197 197, 198 198, 199 199, 200 200, 201 201, 202 202, 203 203, 204 204, 205 205, 206 206, 207 207, 208 208, 209 209, 210 210, 211 211, 212 212, 213 213, 214 214, 215 215, 216 216, 217 217, 218 218, 219 219, 220 220, 221 221, 222 222, 223 223, 224 224, 225 225, 226 226, 227 227, 228 228, 229 229, 230 230, 231 231, 232 232, 233 233, 234 234, 235 235, 236 236, 237 237, 238 238, 239 239, 240 240, 241 241, 242 242, 243 243, 244 244, 245 245, 246 246, 247 247, 248 248, 249 249, 250 250, 251 251, 252 252, 253 253, 254 254, 255 255, 256 256, 257 257, 258 258, 259 259, 260 260, 261 261, 262 262, 263 263, 264 264, 265 265, 266 266, 267 267, 268 268, 269 269, 270 270, 271 271, 272 272, 273 273, 274 274, 275 275, 276 276, 277 277, 278 278, 279 279, 280 280, 281 281, 282 282, 283 283, 284 284, 285 285, 286 286, 287 287, 288 288, 289 289, 290 290, 291 291, 292 292, 293 293, 294 294, 295 295, 296 296, 297 297, 298 298, 299 299, 300 300, 301 301, 302 302, 303 303, 304 304, 305 305, 306 306, 307 307, 308 308, 309 309, 310 310, 311 311, 312 312, 313 313, 314 314, 315 315, 316 316, 317 317, 318 318, 319 319, 320 320, 321 321, 322 322, 323 323, 324 324, 325 325, 326 326, 327 327, 328 328, 329 329, 330 330, 331 331, 332 332, 333 333, 334 334, 335 335, 336 336, 337 337, 338 338, 339 339, 340 340, 341 341, 342 342, 343 343, 344 344, 345 345, 346 346, 347 347, 348 348, 349 349, 350 350, 351 351, 352 352, 353 353, 354 354, 355 355, 356 356, 357 357, 358 358, 359 359, 360 360, 361 361, 362 362, 363 363, 364 364, 365 365, 366 366, 367 367, 368 368, 369 369, 370 370, 371 371, 372 372, 373 373, 374 374, 375 375, 376 376, 377 377, 378 378, 379 379, 380 380, 381 381, 382 382, 383 383, 384 384, 385 385, 386 386, 387 387, 388 388, 389 389, 390 390, 391 391, 392 392, 393 393, 394 394, 395 395, 396 396, 397 397, 398 398, 399 399, 400 400, 401 401, 402 402, 403 403, 404 404, 405 405, 406 406, 407 407, 408 408, 409 409, 410 410, 411 411, 412 412, 413 413, 414 414, 415 415, 416 416, 417 417, 418 418, 419 419, 420 420, 421 421, 422 422, 423 423, 424 424, 425 425, 426 426, 427 427, 428 428, 429 429, 430 430, 431 431, 432 432, 433 433, 434 434, 435 435, 436 436, 437 437, 438 438, 439 439, 440 440, 441 441, 442 442, 443 443, 444 444, 445 445, 446 446, 447 447, 448 448, 449 449, 450 450, 451 451, 452 452, 453 453, 454 454, 455 455, 456 456, 457 457, 458 458, 459 459, 460 460, 461 461, 462 462, 463 463, 464 464, 465 465, 466 466, 467 467, 468 468, 469 469, 470 470, 471 471, 472 472, 473 473, 474 474, 475 475, 476 476, 477 477, 478 478, 479 479, 480 480, 481 481, 482 482, 483 483, 484 484, 485 485, 486 486, 487 487, 488 488, 489 489, 490 490, 491 491, 492 492, 493 493, 494 494, 495 495, 496 496, 497 497, 498 498, 499 499, 500 500, 501 501, 502 502, 503 503, 504 504, 505 505, 506 506, 507 507, 508 508, 509 509, 510 510, 511 511, 512 512, 513 513, 514 514, 515 515, 516 516, 517 517, 518 518, 519 519, 520 520, 521 521, 522 522, 523 523, 524 524, 525 525, 526 526, 527 527, 528 528, 529 529, 530 530, 531 531, 532 532, 533 533, 534 534, 535 535, 536 536, 537 537, 538 538, 539 539, 540 540, 541 541, 542 542, 543 543, 544 544, 545 545, 546 546, 547 547, 548 548, 549 549, 550 550, 551 551, 552 552, 553 553, 554 554, 555 555, 556 556, 557 557, 558 558, 559 559, 560 560, 561 561, 562 562, 563 563, 564 564, 565 565, 566 566, 567 567, 568 568, 569 569, 570 570, 571 571, 572 572, 573 573, 574 574, 575 575, 576 576, 577 577, 578 578, 579 579, 580 580, 581 581, 582 582, 583 583, 584 584, 585 585, 586 586, 587 587, 588 588, 589 589, 590 590, 591 591, 592 592, 593 593, 594 594, 595 595, 596 596, 597 597, 598 598, 599 599, 600 600, 601 601, 602 602, 603 603, 604 604, 605 605, 606 606, 607 607, 608 608, 609 609, 610 610, 611 611, 612 612, 613 613, 614 614, 615 615, 616 616, 617 617, 618 618, 619 619, 620 620, 621 621, 622 622, 623 623, 624 624, 625 625, 626 626, 627 627, 628 628, 629 629, 630 630, 631 631, 632 632, 633 633, 634 634, 635 635, 636 636, 637 637, 638 638, 639 639, 640 640, 641 641, 642 642, 643 643, 644 644, 645 645, 646 646, 647 647, 648 648, 649 649, 650 650, 651 651, 652 652, 653 653, 654 654, 655 655, 656 656, 657 657, 658 658, 659 659, 660 660, 661 661, 662 662, 663 663, 664 664, 665 665, 666 666, 667 667, 668 668, 669 669, 670 670, 671 671, 672 672, 673 673, 674 674, 675 675, 676 676, 677 677, 678 678, 679 679, 680 680, 681 681, 682 682, 683 683, 684 684, 685 685, 686 686, 687 687, 688 688, 689 689, 690 690, 691 691, 692 692, 693 693, 694 694, 695 695, 696 696, 697 697, 698 698, 699 699, 700 700, 701 701, 702 702, 703 703, 704 704, 705 705, 706 706, 707 707, 708 708, 709 709, 710 710, 711 711, 712 712, 713 713, 714 714, 715 715, 716 716, 717 717, 718 718, 719 719, 720 720, 721 721, 722 722, 723 723, 724 724, 725 725, 726 726, 727 727, 728 728, 729 729, 730 730, 731 731, 732 732, 733 733, 734 734, 735 735, 736 736, 737 737, 738 738, 739 739, 740 740, 741 741, 742 742, 743 743, 744 744, 745 745, 746 746, 747 747, 748 748, 749 749, 750 750, 751 751, 752 752, 753 753, 754 754, 755 755, 756 756, 757 757, 758 758, 759 759, 760 760, 761 761, 762 762, 763 763, 764 764, 765 765, 766 766, 767 767, 768 768, 769 769, 770 770, 771 771, 772 772, 773 773, 774 774, 775 775, 776 776, 777 777, 778 778, 779 779, 780 780, 781 781, 782 782, 783 783, 784 784, 785 785, 786 786, 787 787, 788 788, 789 789, 790 790, 791 791, 792 792, 793 793, 794 794, 795 795, 796 796, 797 797, 798 798, 799 799, 800 800, 801 801, 802 802, 803 803, 804 804, 805 805, 806 806, 807 807, 808 808, 809 809, 810 810, 811 811, 812 812, 813 813, 814 814, 815 815, 816 816, 817 817, 818 818, 819 819, 820 820, 821 821, 822 822, 823 823, 824 824, 825 825, 826 826, 827 827, 828 828, 829 829, 830 830, 831 831, 832 832, 833 833, 834 834, 835 835, 836 836, 837 837, 838 838, 839 839, 840 840, 841 841, 842 842, 843 843, 844 844, 845 845, 846 846, 847 847, 848 848, 849 849, 850 850, 851 851, 852 852, 853 853, 854 854, 855 855, 856 856, 857 857, 858 858, 859 859, 860 860, 861 861, 862 862, 863 863, 864 864, 865 865, 866 866, 867 867, 868 868, 869 869, 870 870, 871 871, 872 872, 873 873, 874 874, 875 875, 876 876, 877 877, 878 878, 879 879, 880 880, 881 881, 882 882, 883 883, 884 884, 885 885, 886 886, 887 887, 888 888, 889 889, 890 890, 891 891, 892 892, 893 893, 894 894, 895 895, 896 896, 897 897, 898 898, 899 899, 900 900, 901 901, 902 902, 903 903, 904 904, 905 905, 906 906, 907 907, 908 908, 909 909, 910 910, 911 911, 912 912, 913 913, 914 914, 915 915, 916 916, 917 917, 918 918, 919 919, 920 920, 921 921, 922 922, 923 923, 924 924, 925 925, 926 926, 927 927, 928 928, 929 929, 930 930, 931 931, 932 932, 933 933, 934 934, 935 935, 936 936, 937 937, 938 938, 939 939, 940 940, 941 941, 942 942, 943 943, 944 944, 945 945, 946 946, 947 947, 948 948, 949 949, 950 950, 951 951, 952 952, 953 953, 954 954, 955 955, 956 956, 957 957, 958 958, 959 959, 960 960, 961 961, 962 962, 963 963, 964 964, 965 965, 966 966, 967 967, 968 968, 969 969, 970 970, 971 971, 972 972, 973 973, 974 974, 975 975, 976 976, 977 977, 978 978, 979 979, 980 980, 981 981, 982 982, 983 983, 984 984, 985 985, 986 986, 987 987, 988 988, 989 989, 990 990, 991 991, 992 992, 993 993, 994 994, 995 995, 996 996, 997 997, 998 998, 999 999, 1000 1000, 1001 1001, 1002 1002, 1003 1003, 1004 1004, 1005 1005, 1006 1006, 1007 1007, 1008 1008, 1009 1009, 1010 1010, 1011 1011, 1012 1012, 1013 1013, 1014 1014, 1015 1015, 1016 1016, 1017 1017, 1018 1018, 1019 1019, 1020 1020, 1021 1021, 1022 1022, 1023 1023, 1024 1024, 1025 1025, 1026 1026, 1027 1027, 1028 1028, 1029 1029, 1030 1030, 1031 1031, 1032 1032, 1033 1033, 1034 1034, 1035 1035, 1036 1036, 1037 1037, 1038 1038, 1039 1039, 1040 1040, 1041 1041, 1042 1042, 1043 1043, 1044 1044, 1045 1045, 1046 1046, 1047 1047, 1048 1048, 1049 1049, 1050 1050, 1051 1051, 1052 1052, 1053 1053, 1054 1054, 1055 1055, 1056 1056, 1057 1057, 1058 1058, 1059 1059, 1060 1060, 1061 1061, 1062 1062, 1063 1063, 1064 1064, 1065 1065, 1066 1066, 1067 1067, 1068 1068, 1069 1069, 1070 1070, 1071 1071, 1072 1072, 1073 1073, 1074 1074, 1075 1075, 1076 1076, 1077 1077, 1078 1078, 1079 1079, 1080 1080, 1081 1081, 1082 1082, 1083 1083, 1084 1084, 1085 1085, 1086 1086, 1087 1087, 1088 1088, 1089 1089, 1090 1090, 1091 1091, 1092 1092, 1093 1093, 1094 1094, 1095 1095, 1096 1096, 1097 1097, 1098 1098, 1099 1099, 1100 1100, 1101 1101, 1102 1102, 1103 1103, 1104 1104, 1105 1105, 1106 1106, 1107 1107, 1108 1108, 1109 1109, 1110 1110, 1111 1111, 1112 1112, 1113 1113, 1114 1114, 1115 1115, 1116 1116, 1117 1117, 1118 1118, 1119 1119, 1120 1120, 1121 1121, 1122 1122, 1123 1123, 1124 1124, 1125 1125, 1126 1126, 1127 1127, 1128 1128, 1129 1129, 1130 1130, 1131 1131, 1132 1132, 1133 1133, 1134 1134, 1135 1135, 1136 1136, 1137 1137, 1138 1138, 1139 1139, 1140 1140, 1141 1141, 1142 1142, 1143 1143, 1144 1144, 1145 1145, 1146 1146, 1147 1147, 1148 1148, 1149 1149, 1150 1150, 1151 1151, 1152 1152, 1153 1153, 1154 1154, 1155 1155, 1156 1156, 1157 1157, 1158 1158, 1159 1159, 1160 1160, 1161 1161, 1162 1162, 1163 1163, 1164 1164, 1165 1165, 1166 1166, 1167 1167, 1168 1168, 1169 1169, 1170 1170, 1171 1171, 1172 1172, 1173 1173, 1174 1174, 1175 1175, 1176 1176, 1177 1177, 1178 1178, 1179 1179, 1180 1180, 1181 1181, 1182 1182, 1183 1183, 1184 1184, 1185 1185, 1186 1186, 1187 1187, 1188 1188, 1189 1189, 1190 1190, 1191 1191, 1192 1192, 1193 1193, 1194 1194, 1195 1195, 1196 1196, 1197 1197, 1198 1198, 1199 1199, 1200 1200, 1201 1201, 1202 1202, 1203 1203, 1204 1204, 1205 1205, 1206 1206, 1207 1207, 1208 1208, 1209 1209, 1210 1210, 1211 1211, 1212 1212, 1213 1213, 1214 1214, 1215 1215, 1216 1216, 1217 1217, 1218 1218, 1219 1219, 1220 1220, 1221 1221, 1222 1222, 1223 1223, 1224 1224, 1225 1225, 1226 1226, 1227 1227, 1228 1228, 1229 1229, 1230 1230, 1231 1231, 1232 1232, 1233 1233, 1234 1234, 1235 1235, 1236 1236, 1237 1237, 1238 1238, 1239 1239, 1240 1240, 1241 1241, 1242 1242, 1243 1243, 1244 1244, 1245 1245, 1246 1246, 1247 1247, 1248 1248, 1249 1249, 1250 1250, 1251 1251, 1252 1252, 1253 1253, 1254 1254, 1255 1255, 1256 1256, 1257 1257, 1258 1258, 1259 1259, 1260 1260, 1261 1261, 1262 1262, 1263 1263, 1264 1264, 1265 1265, 1266 1266, 1267 1267, 1268 1268, 1269 1269, 1270 1270, 1271 1271, 1272 1272, 1273 1273, 1274 1274, 1275 1275, 1276 1276, 1277 1277, 1278 1278, 1279 1279, 1280 1280, 1281 1281, 1282 1282, 1283 1283, 1284 1284, 1285 1285, 1286 1286, 1287 1287, 1288 1288, 1289 1289, 1290 1290, 1291 1291, 1292 1292, 1293 1293, 1294 1294, 1295 1295, 1296 1296, 1297 1297, 1298 1298, 1299 1299, 1300 1300, 1301 13
```

```

const comp = new Map<string, string>(points.map((p) => {
const find = (x: string): string => {
 while (comp.get(x) !== x) {
 comp.set(x, comp.get(x)!);
 x = comp.get(x)!;
 }
 return x;
});
for (const e of edges) {
 const ra = find(pid(e.a));
 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
}
const comps = new Set(points.map((p) => find(pid(p))))
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v
 let left: Pair | null = null;
 let right: Pair | null = null;

```

```

const rootPair = roots.length ? roots[0].a : edges.l
if (!rootPair) return null;
const build = (p: Pair): TNode => {
 const node = make(p);
 nodes.set(pid(p), node);
 const kids = edges.filter((e) => pid(e.a) === pid(p)
 if (kids.length > 2) return node; // невалидно – по
 kids.sort((u, v) => u.key - v.key);
 const left = kids.filter((k) => k.key <= p.key); //
 const right = kids.filter((k) => k.key > p.key);
 if (left.length > 1 || right.length > 1) return node
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
};
return build(rootPair);
}

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const live = useTreapLive();

 // набор точек из Python-кода пересобирает плоскость
 useEffect(() => {
 if (!live.pairsText || live.pairsText === raw) return
 setRaw(live.pairsText);
 setPhase("connect");
 setOrder("y-desc");
 setSortIdx(3);
 setConnIdx(10000); // все рёбра проведены – ср
 setPlaying(false);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [live.pairsText]);
 const [phase, setPhase] = useState<BuildPhase>("sort"
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);

```

```

const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>();
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>();
const [gameMsg, setGameMsg] = useState<string | null>();
const [badEdge, setBadEdge] = useState<GameEdge | null>();
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};

const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree

```

13

```
{parsed.errors.length > 0 && (
 <div className="mt-2 text-xs text-rose-300 bg
 {parsed.errors.map((e, i) => <div key={i}>x
 </div>
```

```

 })
 {parsed.errors.length === 0 && (parsed.equalX |
 <div className="mt-2 text-xs text-amber-300 b

```

```

 </button>
 <button
 onClick={() => startConnect("left-right")}
 className={`px-3 py-1.5 rounded-lg text-white
 "bg-emerald-600 text-white" : "bg-slate-700 text-white"
 }>
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-white
 "go-600 text-white" : "bg-indigo-500/80 text-white"
 }>
 соединить сам
 </button>
 </>
)}
 </div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : phaseEdges}
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-slate-800/50
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} пар)
 </>
) : sortIdx < 3 ? (

```



```

 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно <b
 </>
 })

{phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE
 white disabled:opacity-30">↶ Убрать линию</
 <button onClick={resetGame} disabled={!gameE
 ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
 te disabled:opacity-30"> Показать решение<
 </div>
)}
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
 <div className="bg-emerald-950/20 rounded-xl p-
 <h4 className="text-sm font-bold text-emerald
 <TreeView root={gameTree} subtitle="Слева-нап
 ществует." />
 </div>
)}

{/* Дерево */}
{order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={userTree} subtitle="Слева-нап

```

```

const rTree = split.r;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Разрез
 <select value={x0} onChange={(e) => { setX0(Num
 -700 rounded-lg px-3 py-1.5 text-sm font-mono
 {[2, 3, 6, 7, 9, 12, 45].map((k) => <option k
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {i
 </div>

 <div className="bg-slate-900/40 rounded-xl p-4 bo
 <h4 className="text-sm font-bold text-slate-300
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 />
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
 </div>

 {done && (
 <div className="grid grid-cols-1 md:grid-cols-2
 <div className="bg-sky-950/20 rounded-xl p-3 l
 <h5 className="text-sm font-bold text-sky-3
 <TreeView root={lTree} width={480} height={
 </div>

```

```

 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {id
 </div>

 <div className="grid grid-cols-1 md:grid-cols-2 g
 <div className="bg-sky-950/20 rounded-xl p-3 bo
 <h5 className="text-sm font-bold text-sky-300
 <TreeView root={l} width={480} height={220} n
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 b
 <h5 className="text-sm font-bold text-rose-300
 <TreeView root={r} width={480} height={220} n
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {cur?.chosenId ? {
 <? {cur.note}</?
 } : {
 <?Готово: {done ? "деревья склеены обратно в
 />
 }}
 </div>

 {done && (
 <div className="bg-emerald-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-emerald
 <TreeView root={merged.root} subtitle="merge(
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-9
 Грамотно про «слияние»: у корней сравни
 в (то, что сохраняет порядок x) отправляется ,
 </div>
</div>

```

```

 Удалить
 <select value={key} onChange={(e) => { setKey(N
 te-700 rounded-lg px-3 py-1.5 text-sm font-mo
 {POINTS.map((p) => <option key={p.key} value=
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {i
 </div>

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4
 <TreeView
 root={fullTree}
 nodeColor={(id) => (pathSet.has(id) ? "#783
 nodeStroke={(id) => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только вер
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={res.root} subtitle="Валидность
 <ul className="mt-2 space-y-1 text-sm">
 обход слева-направо по-прежнему сорти
 каждое ребро вниз по y (куча целая) –

 </div>
))

 <div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-9
 Ответ на «убрать можно только самый верхний

```

```

 <h4 className="text-sm font-bold text-slate-300">
 4>
 <div className="flex gap-2 overflow-x-auto mb-4">
 {heapArr.slice(1).map((v, i) => {
 <div key={i} className="px-3 py-2 rounded">
 <div className="text-slate-500 text-[10px]">{v}</div>
 <div className="text-white font-bold">{i}</div>
 <div className="text-emerald-400 text-[10px]">{v}</div>
 </div>
 })}
 </div>
 <p className="text-xs text-slate-400">Полное $2i/2i+1$ попадает в узел.</p>
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-rose-300">
 >
 <TreeView root={full} subtitle="Попробуй «посмотреть»" />
 <div className="flex gap-2 overflow-x-auto mt-2">
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((v, i) => {
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded border-slate-700`} : "bg-rose-950/40 border-rose-950">
 <div className="text-slate-500 text-[10px]">{v}</div>
 {id ? <div className="text-white font-[10px]">{v}</div> : null}
 </div>
);
 })}
 </div>
 <p className="text-xs text-rose-300 mt-2">Смотри, что левая ветка ушла на 4 уровня и заняла два места в индексах указателей (left/right), а не арифметикой.</p>
 </div>
)}

```

```

 else { steps.push({ lo, hi, mid, found: false, no
 ` }); hi = mid - 1; }
 }
 return steps;
}, [sortedByKey]);

```

```

const wrongMid = Math.floor((shuffled.length - 1) / 2)
const comparisons = stage === 2 ? bidx + 1 : 0;

```

```

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти y для x=7 бинарным по
 </button>
)}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsStep
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0
 Сброс</button>
 </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p, i) => {
 const highlight = stage === 1 && i === wrongMid
 const inBs = stage === 2 && (() => { const s = sortedByKey[
 const isMid = stage === 2 && (() => { const s = sortedByKey[
 const isFound = stage === 2 && bidx >= bsStep
 return (
 <div key={` ${p.key}: ${p.grade} - ${i} `} class="border-1
 nsition-all ${highlight ? "border-rose-500 border-amber-500
 r-amber-500 bg-amber-950/30" : inBs ? "border-amber-500

```

```

const v = gameVerdict(edges, POINTS);

const click = (p: Pair) => {
 if (!sel) {
 setSel(p);
 setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кл
 return;
 }
 if (sameP(sel, p)) {
 setSel(null);
 return;
 }
 const res = tryConnect(edges, sel, p);
 if (res.ok) {
 const parent = sel.grade > p.grade ? sel : p;
 const child = sel.grade > p.grade ? p : sel;
 setEdges([...edges, { a: parent, b: child }]);
 setMsg(` Линия (${parent.key}; ${parent.grade})
 setSel(null);
 } else {
 setMsg("x " + res.reason);
 setSel(null);
 }
};

const win = v.win;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0
 rounded-lg text-xs font-bold bg-slate-700 tex
 <button onClick={() => { setEdges([]); setSel(n
 3 py-1.5 rounded-lg text-xs font-bold bg-slate
 <button onClick={() => { setEdges(CANON_GAME.ma
); }} className="px-3 py-1.5 rounded-lg text-
 линий:
 </div>

```

```

 });
 }}}}
</svg>
</div>

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
ate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {v.heap ? " " : " "} каждое ребро – вниз
 {v.oneParent ? " " : " "} у каждой точки
 {v.acyclic ? " " : " "} циклов нет
 {v.bst ? " " : " "} обход слева-направо –
 {v.complete ? " " : " "} линий $n-1 = \{POI$

 {win && {
 <p className="mt-3 text-sm font-bold text-еме
 ения просто не существует.</p>
 }}
</div>

 <div className="text-xs text-slate-300 bg-slate-9
</div>
 });
};

/* ===== СВЯЗЬ С КОМПИЛЯТОРОМ =====

/**
 * Живые переменные Python-кода страницы treap:
 * points / pairs – набор пар (x; y): по нему ПЕРЕСТРАИ
 * mid – разрез для вкладки Split;
 * key – удаляемая точка для вкладки Erase;
 * step / i – номер шага (листаает кадры Split / E
 * height, n – высота дерева и число узлов (показ
 */
const useTreapLive = () => {

```



```

 { id: "layers", label: "Миф: 2k/2k+1" },
 { id: "search", label: "Поиск ≠ сортировка" },
 { id: "game", label: "Игра" },
];

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");
 const chapterId = useContext(VizChapterContext);
 const live = useTreapLive();

 // вкладка = подрежим страницы: панель Python подстав.
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Пар.
 </p>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}

```

```
 {tab === "search" && <SearchMode />}
 {tab === "game" && <GameMode />}
 </div>
);
};
```

---

ФАЙЛ 79/111: src/components/VizLiveStatus.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";

interface VizLiveStatusProps {
 /** Название демонстрации – подписывается в режиме вс
 demoTitle?: string;
 className?: string;
}

/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",

/**
 * Строка состояния связи «компилятор → демонстрация» д
 *
 * Раньше компонент молча проигрывал собственный встроен
 * редакторе Python был пустой код, код с другими именам
 * размерности, – выглядело так, будто компилятор исполн
 * картинка живёт отдельной жизнью. Теперь источник данн
 * «данные из компилятора (i, j, dist 5x5)», «компилятор
 * пока нет» (отладчик стоит до первой строки) или «встр
 */
export function VizLiveStatus({ demoTitle, className =
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const names = vars ? Object.keys(vars).filter((name)
```

```

import FloydViz from "./FloydViz";
import { GraphComponentsViz } from "./GraphComponentsViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { ComplexityViz } from "./ComplexityViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import { PlanarColoringViz } from "./PlanarColoringViz";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 },

```

```

 },
 "dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
 },
 "splay-tree": {
 title: "Splay: механика → большое дерево → собери с...",
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
 },
 "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: ... },
 "top-sort": { title: "Топологическая сортировка", Component: ... },
 "scc-kosaraju": { title: "Компоненты сильной связности", Component: ... },
 "graph-articulation": { title: "Точки сочленения", Component: ... },
 "bridges-code": { title: "Мосты: DFS + tin/low", Component: ... },
 "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: ... },
 "planarity-euler-formula": {
 title: "Планарность и покраска: $K_5/K_{3,3}$, $\chi \leq \Delta+1$, д...
 hint: "Четыре вкладки: непланарные K_5 и $K_{3,3}$, жадна...
 ашего кода красит вершины напрямую.",
 Component: PlanarColoringViz,
 },
 dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем...",
 Component: DijkstraViz,
 },
 "bellman-ford": {
 title: "Форд–Беллман",
 Component: BellmanFordViz,
 },
 floyd: {
 title: "Флойд–Уоршелл",
 Component: FloydViz,
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: ... }

```

```
 },
 };

export const getViz = (id?: string): VizEntry | undefined
```

---

ФАЙЛ 81/111: src/components/VizSourceBadge.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { Cpu, Film } from "lucide-react";
import { Tooltip } from "../Tooltip";
```

```
interface VizSourceBadgeProps {
```

```
 /**
```

```
 * true — картинку рисуют переменные из выполненного кода
```

```
 * false — компонент показывает собственную встроенную картинку
```

```
 */
```

```
 linked: boolean;
```

```
 /**
```

```
 * Компилятор уже выполнил код и прислал шаг, но переменные ещё не обновлены
```

```
 * (отладчик стоит ДО первой строки или в коде другие переменные не изменились)
```

```
 * подпись должна объяснять, почему картинка ещё не инициализирована
```

```
 */
```

```
 awaiting?: boolean;
```

```
 /** Строка пользовательского кода, на которой стоит отладчик
```

```
 line?: number;
```

```
 /** Короткая подпись данных компилятора: «матрица 5×5»
```

```
 detail?: string;
```

```
 /** Как называется встроенная демонстрация (для подписи)
```

```
 demoTitle?: string;
```

```
 /** Размеры демо-сетки и данных пользователя не совпадают
```

```
 mismatch?: string | null;
```

```
 className?: string;
```

```
}
```

```
/**
```

```

 сценарий: {demoTitle}

) : null}

 {mismatch ? (
 <Tooltip side="bottom" content={mismatch}>
 <span
 tabIndex={0}
 className="cursor-help rounded-full border border-slate-500 p-2"
 >
 размерность не совпадает

 </Tooltip>
) : null}
 </div>
);
}
```

---

ФАЙЛ 82/111: `src/components/WaterfallAnimationWidget.tsx`  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from '../context';

// Структура заранее подготовленного красивого бора для визуализации
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
```

```

export const WaterfallAnimationWidget: React.FC = () =>
 // Переключатель режимов: 'training' (Полный BFS обхо
 const [activeMode, setActiveMode] = useState<'trainin

 /* — связь с Python-компилятором (PAGE_SYNC["aho-cor
 text — сканируемый текст, state — вершина автомата
 matches — найденные вхождения (индекс конца, слово
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
 }, [chapterId]);

 const liveText = vars ? vizString(vars.text) ?? vizSt
 const liveState = vars ? vizNumber(vars.state) ?? viz
 const liveI = vars ? vizNumber(vars.i) ?? vizNumber(v
 const liveMode = vars ? vizString(vars.mode) : null;
 const liveMatchesRaw = vars ? vizArray(vars.matches)
 const liveMatchesSig = useMemo(() => (liveMatchesRaw
 const liveFail = vars ? vars.fail : undefined;

 useEffect(() => {
 if (!vars) return;
 if (liveMode && /search|поиск/i.test(liveMode)) set
 else if (liveMode && /train|обуч|bfs/i.test(liveMod
 if (liveText) {
 const clean = liveText.toUpperCase().replace(/[^A
 if (clean && clean !== textToScan) {
 setTextToScan(clean);
 setCurrentIndex(0);
 setCurrentNode(0);
 setFoundMatches([]);
 setIsSearchDone(false);
 setJumpPath(null);
 setActiveMode("search");
 }
 }
 });

```

```

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInp
 const clean = e.target.value.toUpperCase().replace(
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся н
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось дово
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];

```



```

 if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWords}
 setActiveSearchCodeLine(4);
 }

 setCurrentNode(curr);
 setCurrentIndex(currentIndex + 1);
 setSearchLog(logMsg);
 if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
 }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обр
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },

```

```

 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины :
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 },

```

```

 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p-2
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 `}
 }
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
</button>
<button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
</button>
</div>
</div>

```

```

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3
 Показаны все шаги очереди BFS (начиная с
 ар 8 (SHE → HE), где наглядно видно
 </p>
 }
 </div>
 </div>
} : {
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3
 {/* Управление шагами поиска */}
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Поиск в тексте (поиск по шагам)
 </h5>
 <p className="text-xs text-slate-300 mb-3
 Показаны все шаги очереди поиска (начиная с
 ар 8 (SHE → HE), где наглядно видно
 </p>
 }
 </div>
 </div>
}
}

```

```

 { /* Блок с кодом построения fail-ссылки и опи
 <div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 { Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
 ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex ite
 l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
 т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex ite
 r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
 /span>
 {currentTrainInfo.codeLine === 4 && <

```

```

 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
 idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
 'bg-slate-800 text-slate-500'}
 >
 {char}

)}
 </div>

 ГОТОВО ✓

 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-weight-normal
 ${isSearchDone || textToScan.length === 0 ? 'bg-slate-700 text-slate-500 cursor-not-allowed' :
 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 >
 Поиск
 </button>
 </div>
 </div>
</div>

```

```

 </div>
 </div>

 <div>
 <h5 className="text-slate-300 font-bold m-0">
 <AlertCircle className="w-3.5 h-3.5 text-slate-300"/>
 </h5>
 <div className="bg-slate-800 p-3.5 rounded-lg items-center">
 {searchLog}
 </div>
 </div>
 </div>
 </div>
)}

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-gradient-to-b from-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-1/2">
 <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
 <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>

 <div className="flex items-center justify-between p-4">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
 </div>
</div>

```

```

 return Object.entries(children).map(([char, children]) => {
 const toNode = WATERFALL_NODES[toId];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ||
 activeMode === 'normal';

 // Подсветка в режиме обучения (когда и
 let isTrainingExamined = activeMode === 'training';
 let isMatchBranch = isTrainingExamined &

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}>
 {/* Линия течения воды */}
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 className={isHighlighted ? 'highlighted' : ''}
 />
 {/* Символ перехода */}
 <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'none'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2} + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'none'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 {char}
 </g>
);
 });
 }

 {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
```

```

 <path
 d={`M ${node.x} ${node.y} Q ${cx} $
 fill="none"
 stroke={isHighlighted ? '#f43f5e' :
 strokeWidth={isHighlighted || isJustE
 strokeDasharray="6,6"
 className={isHighlighted || isJustE
 opacity={isHighlighted || isJustEst
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b
 >
 fail[{node.label}] = {targetNode.
 </text>
)}
 </g>
);
}}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

 return (
 <g key={`node-${node.id}`} className="node" style={nodeStyle} >
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={node.radius + 2} fill="none" stroke="red" stroke-width={2} />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={node.radius + 2} fill="none" stroke="blue" stroke-width={2} />
)}

 {/* Основной круг вершины */}
 <circle
 cx={node.x}

```



```

 }}
 </g>
);
 }}}

 { /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
 <g>
 { /* Круг-подложка под рыбу */
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5'
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 { /* Сама рыба */
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
 </g>

 </svg>
</div>
</div>

```

```

 { /* Список найденных совпадений (только в режиме
 {activeMode === 'search' && {
 <div className="mt-6 bg-slate-900/50 p-5 rounded

```

```

 {liveState !== null && {
 <span className="text-[11px] font-mono bg-
 state = {liveState}{WATERFALL_NODES[liveState]}

 }}
 {liveMatchesSig && <span className="text-[11px] font-mono bg-
 d px-2 py-1">matches: {foundMatches.map((m) => {
 {liveFail !== undefined && <span className="text-[11px] font-mono bg-
 ed-md px-2 py-1">fail = {JSON.stringify(liveState)}

 }) : {

 открой панель Python и запусти Ахо–Корасик:
 "font-mono text-cyan-300">state</code>, <span className="text-[11px] font-mono bg-
 an-300">matches</code> поведут лосося по этому пути

 }}
 </div>
 </div>
);
 };
};

```

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 83/111: src/components/visualizers/PrefixSumViz.tsx  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 466 | РАЗМЕР: 1.5 КБ

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import {
 emitVizDemo,
 liveMatrix,
 pickLiveVar,
 useVizRuntime,
} from '@visuallib/core';

```

```

<div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800" >
 <div className="flex gap-2 mb-4 overflow-x-auto" >
 {{
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
 }.map(({key, label}) => {
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm:text-sm
 mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
 }` >
 {label}
 </button>
 })}
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]], []),
 []
);
};

```

```

/** Что сейчас под курсором: префикс P[i] или элемент A[i]
const [hover, setHover] = useState<{ kind: "pref" | "elem" }>({ kind: "pref" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);

```

```

const isDrivenAdd = shownDriven !== null &&
return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r
 className={`${cellBase} cursor-pointe
 isDrivenAdd
 ? "bg-amber-500 text-black ring-2
 : inCover
 ? "bg-indigo-500 text-white rin
 : inRange
 ? "bg-indigo-900/70 text-indi
 : "bg-slate-800 text-slate-300
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
}})
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" &&
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const liveValue = liveP?.[i];
 const blank = hasLiveP && (liveValue ===
 const masked = !hasLiveP && shownDriven !
 const isDrivenCur = shownDriven !== null &
 const shownValue = hasLiveP ? liveValue :
 return (
 <div

```

```

) : hover?.kind === "pref" ? (
 hover.i === 0 ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0
 отрезка, начинающегося с нуля.
 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i} - 1] = {A1.slice(0, hove

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
)
 })
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву

```

---

```

* Матрицы из кода пользователя рисуются В СВОЕЙ разме
*
* Раньше значения S[i][j] протискивались в демо-сетку
* размере матрицы часть ячеек становилась «пустыми»,
* шаги компилятора не совпадали с картинкой.
*/

```

```

const livePrefSource = pickLiveVar(vars, ["S", "P", "g"]);
const liveS = liveMatrix(livePrefSource?.value);
const hasLiveS = liveS !== null;

```

```

const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);

```

```

const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
 ? Array.from({ length: prefRows }, (_, i) =>
 Array.from({ length: prefCols }, (_, j) => liveS[i][j]
)
 : P;

```

```

const srcRows = liveA?.rows ?? n;
const srcCols = liveA?.cols ?? m;
const srcGrid: (number | null)[][] = liveA
 ? Array.from({ length: srcRows }, (_, i) => Array.from(
 { length: srcCols }, (_, j) => liveA[i][j]
)
 : A2;

```

```

const cellAt = (i: number, j: number): number | null {
 if (i >= 0 && j >= 0 && i < prefRows && j < prefCols) return liveS[i][j];
 return null;
}

```

```

const sizeMismatch =
 hasLiveS && liveS && (liveS.rows !== P.length || liveS.cols !== P[0].length)
 ? `Демо построено на матрице ${n}x${m} (сетка пре
 мет имеет размер ${liveS.sizeLabel}. Ниже нарисована
 : null;

```

```

useEffect(() => {
 const r1 = vizNumber(vars?.r1);

```

```

 </p>
) : null}
 <div className="grid gap-5 lg:grid-cols-2">
 { /* Исходная матрица */ }
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Исходная матрица {liveA ? `${liveArrSource?`
 чтобы задать прямоугольник
 </div>
 <div className="inline-block">
 {srcGrid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= r1 && i <= r2 &&
 return {
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1
 ? { r1: i, c1: j, r2: Math.max(i, j), c2: Math.min(i, j) }
 : { ...r, r2: i, c2: j }
)
 }
 className={` ${cellBase} cursor-pointer
 inRect ? "bg-indigo-500 text-white" : ""
 }`}
 >
 {v === null ? "." : v}
 </div>
)};
 })}
 </div>
)}}
 </div>
 </div>
 { /* Матрица префиксных сумм */ }

```

```

 {corner && !isHover && {
 <span className="absolute -top-
ht">
 {corner}

 }}
 </div>
);
 }}}
</div>
)))
</div>
</div>
</div>

```

```

<div className="min-h-[54px] bg-slate-900 border
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 {liveS ? livePrefSource?.name ?? "S" : "P
 {" "}
 – сумма всего прямоугольника от левого верх
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор
 }}
</div>

```

```

<div className="bg-slate-900 border border-indigo
 <p className="font-mono text-sm sm:text-base te
 сумма = A<
 C + <s
 ={" "}
 {
 </p>
 <p className="font-mono text-[11px] text-slate-
 прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · c
 </p>

```



```
const st1D: number[][] = Array.from({ length: N }, () =>
 for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
 for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)][j - 1]);
 }
 }
}
```

// Data for 2D Matrix (Sparse Table 2D)

```
const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];
```

```
const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);
```

```
for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}
```

```
for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
```

```

 </button>
 <button
 onClick={() => setTab("2d_build")}
 className={`px-3 sm:px-4 py-2 rounded-lg text
 digo-600 text-white" : "bg-slate-800 text-s
 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text
 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
 </div>
);
};

```

```

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number}>({ i: 0, j: 0 });
 const runtime = useVizRuntime();
 const runtimeVars = runtime?.variables;
 const liveI = vizNumber(runtimeVars?.i);
 const liveJ = vizNumber(runtimeVars?.j);
 const liveSource = vizArray(runtimeVars?.a);

```

```

const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 {dimMismatch ? (
 <p className="mt-2 max-w-[560px] rounded-lg bg-slate-800 text-white">
 {dimMismatch}
 </p>
) : null}
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:{step === 0}"
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500 disabled:{step === maxStep}"
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:{step === 0}"
 >

```

```


 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\text{hover.j}}$ = {1 << hover.j}:{" "}

 min({arr1D.slice(covered.from, covered.to)}

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
 </div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-500">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 <div className="font-mono text-slate-500">
 /* Headers */
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^3 (L=8)
 </div>
 </div>
 </div>
</div>

```

```

// Готовые вычисленные уровни демо не
// уровень j=0 всегда берём из уже сущ
const shownValue = compilerLinked ? r
const displayValue =
 shownValue === undefined || shownVa
 ? "."
 : typeof shownValue === "object"
 ? JSON.stringify(shownValue)
 : String(shownValue);

return (
 <div key={j} className="relative fl
 {!isValid ? (
 <div className="w-[clamp(30px,9
) : (
 <motion.div
 onMouseEnter={() => (compiler
 onMouseLeave={() => setHover(
 onClick={() => (compilerLinker
)}}

 initial={{ opacity: 0, scale:
 animate={{
 // При st = [] каркас уже с
 opacity: compilerLinked ? 1
 scale: compilerLinked || is
 }}
 className={`w-[clamp(30px,9vw
clamp(11px,3vw,17px)] font-bold transition-
 hover && hover.i === i && h
 ? "bg-sky-500 text-white
 : isActive
 ? "bg-indigo-500 text-whi
 : isSrc1
 ? "bg-emerald-500 text-r
 : isSrc2
 ? "bg-amber-500 text-r
 : isVisible
 ? "bg-slate-800 tex

```



```

const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'liveTable');
const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
const codeCell = liveR !== null && liveC !== null ? {
 code: liveTable[liveK] || liveTable[0],
 compilerLinked: codeCell !== null || liveK !== 0,
 shownK: pinnedK !== null
 ? pinnedK
 : liveK !== null
 ? Math.max(0, Math.min(3, Math.trunc(liveK)))
 : hasLiveSource
 ? 0
 : k;
} : null;

/** 2D-демонстрация нарисована на фиксированной сетке */
const liveSource2D = liveMatrix(vars?.A);
const dimMismatch2D = liveSource2D
 ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
 : `Демо 2D построено на матрице 8x8, а в вашем коде матрица 8x8
 не её не отрисовываются, значения вашей матрицы`;
const dimMismatch2DText = dimMismatch2D ? null : null;

// Уровень и клетка берутся прямо из k/r/c; номер строки
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 shadow">
 <h3 className="text-xl font-bold text-indigo-400">Матрица
 <p className="text-sm text-slate-400 mb-6 text-slate-400">
 Для наглядности показываем только <b className="text-slate-400">


```

```

<h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
</h4>
<div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 const liveValue = liveTable?.[
 // Базовый слой (блоки 1x1) – э
 // одного элемента равен ему сам
 // показываем из A сразу, как A
 // цикла построения st2. Уровни
 const sourceValue = step === 0
 const shownValue = compilerLink
 const isBlank = shownValue ===
 let highlightClass = isBlank
 ? 'bg-slate-950 text-slate-600'
 : isCurr
 ? 'bg-slate-800/80 text-slate-300'
 : 'bg-slate-800/50 text-slate-300'
 let zIndex = 'z-0';

 const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 // Значения в ячейке ещё не
 // контур «сюда смотрит код
 // залитая пустая клетка вы
 highlightClass = isBlank

```





```

<p className="text-slate-300 mb-3 t
 Пример для ячейки:
 {locked && <span className="text
ded border border-rose-500/20"><Lock size={
</p>
<p className="text-indigo-300 borde
 <span className="font-bold text-w
}}[{{shownK}}][{{shownK}}]

</p>

<div className="space-y-2 pt-3 pl-2
 {{{}} => {
 const prevL = 1 << (shownK -
 return (
 <>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{{activeC
 <span className="text
tiveCell.r}[activeCell.c][shownK-1][shownK-
 </div>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{{activeC
 <span className="text
tiveCell.r}[activeCell.c + prevL][shownK-1]
 </div>
 <div className="flex ite
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{{activ
 <span className="text

```

```

const nr1 = vizNumber(vars?.r1);
const nc1 = vizNumber(vars?.c1);
const nr2 = vizNumber(vars?.r2);
const nc2 = vizNumber(vars?.c2);
if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
}, [runtime]));

const queryVars = runtime?.variables;
const liveCoords = ["r1", "c1", "r2", "c2"].map((na
const queryMismatch = liveCoords.some((v) => v !== n
 ? `Координаты запроса в вашем коде выходят за дем
] ?? "-"}, r2=${liveCoords[2] ?? "-"}, c2=${live
 : null;

const H = Math.max(1, r2 - r1 + 1);
const W = Math.max(1, c2 - c1 + 1);
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
 <div className="flex flex-col xl:flex-row gap-6
 <div className="flex-1 min-w-0 bg-slate-900
 <h3 className="text-xl font-bold text-ro
 <p className="text-slate-400 text-[13px]
 еpx, r2, c2 - правый низ).</p>
 {queryMismatch ? (
 <p className="mb-3 rounded-lg border b
 {queryMismatch}
 </p>
) : null}

```

```

 <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)` , width: `calc(${(Ly / 8) * 100}% - 16
 <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)` , width: `calc(${(Ly / 8) * 100
 </div>

 <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 </div>
</div>
</div>

<div className="flex-1 bg-slate-900 p-6 rou
 <h3 className="text-xl font-bold text-em
 <p className="text-sm font-sans text-sla
 Мы знаем, что максимальная степень дв
rounded text-white">{Lx}. Аналогично
te">{Ly}.

```

```

и влево от c2)

 >r2 - Lx + 1][
 {'}'}));
 </div>

 <div className="w-full flex justify-center">
 <div className="flex flex-col items-center">
 <h4 className="text-slate-400 font-bold">
 Откуда берутся эти 4 числа: матрица
 </h4>
 <div className="grid gap-[2px] p-2.5">
 Columns: `repeat(${matCols}, 1fr)`}}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-slate-200";
 let txt = ST2D[r][c][kx][ky];

 let badge = null;
 if (isTL) { color = "bg-rose-500 text-rose-100 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-rose-500 text-rose-100 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-300 text-emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-300 text-amber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color} style={{padding: 5px, text-align: center}}>
 {txt}
 {badge}
 </div>
);
 })}
 </div>
 </div>
 </div>
 </div>
);
}

```

ФАЙЛ 85/111: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 const aside = document.querySelector('aside');
 const headerMain = document.querySelector('header main');
 const questionsContainer = document.getElementById('questions-container');
 const wtfContainer = document.getElementById('wtf-container');

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode');
 if (aside) aside.style.display = 'none';
 if (headerMain) headerMain.style.display = 'none';
 if (questionsContainer) questionsContainer.style.display = 'none';
 if (wtfContainer) wtfContainer.classList.remove('hidden');

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname);
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden');
 }
 }

```

```
const closeDrawerBtn = document.getElementById('close-drawer')
const drawerOverlay = document.getElementById('drawer-overlay')

function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('closed')
 if (isClosed) {
 mobileDrawer.classList.remove('closed')
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden')
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 100)
 }
 document.body.style.overflow = 'hidden'
 } else {
 mobileDrawer.classList.add('closed')
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden')
 setTimeout(() => drawerOverlay.classList.add('hidden'), 100)
 }
 document.body.style.overflow = ''
 }
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener('click', () => toggleDrawer())
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener('click', () => toggleDrawer())
}
if (drawerOverlay) {
 drawerOverlay.addEventListener('click', () => toggleDrawer())
}

// Close on link click
document.querySelectorAll('#mobile-drawer a').forEach(link => {
 link.addEventListener('click', () => {
 if (mobileDrawer && !mobileDrawer.classList.contains('closed')) {
 toggleDrawer()
 }
 })
})
```

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</t
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js
 <script id="MathJax-script" async src="https://cdn.
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']]
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "./src/index.css";
 .math-font { font-family: 'Fira Code', monospac
 .uno-card {
 width: 70px; height: 105px; border-radius:
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); posi
 align-items: center; justify-content: cente
 color: white; overflow: hidden; flex-shrink
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #0
 font-family: 'Arial Black', Impact, sans-se
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #0
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; tra
 .uno-card.mini { width: 45px !important; height
```



```

 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::after {
 width: 100%; height: 100%;
 }

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !important; }
 px; }
 }
 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }
 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card-
 .card-green { background-color: #339e35; } .card-
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;

```

```

 .creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
 }
 .creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
 </style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 Темы
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setMode('normal')" id="btn-normal"
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf"
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 Темы
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setTheme('dark')" id="btn-dark"
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light"
 transition-colors">Светлая</button>
 </div>
 </div>
 </div>

```

```

<g id="lego-red">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="20"
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#333"
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55"
 <circle cx="25" cy="50" r="15" fill="#f
</g>
<g id="lego-green">
 <rect x="0" y="10" width="20" height="10"
 <rect x="3" y="5" width="14" height="5"
</g>
<g id="lego-white">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
</g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->

```

```
<div class="pl-6 space-y-2 text
 <a href="#section-1-1" class="b
 <a href="#section-1-2" class="b
 <a href="#section-1-3" class="b
 <a href="#section-1-4" class="b
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-2" class=
er-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#section-2-1" class="b
 <a href="#section-2-2" class="b
 <a href="#section-2-3" class="b
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-3" class=
er-teal-500 pl-3 font-bold text-teal-300">
 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#section-3-1" class="b
 <a href="#section-3-2" class="b
 <a href="#section-3-3" class="b
 <a href="#section-3-4" class="b
 </div>
</details>

<details class="group">
```

```
<div class="pl-6 space-y-2 text
<a href="#q6-1" class="block ho
<a href="#q6-2" class="block ho
<a href="#q6-3" class="block ho
<a href="#q6-4" class="block ho
<a href="#q6-5" class="block ho
<a href="#q6-6" class="block ho
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числ

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
 <a href="#q7-3" class="bloc
 <a href="#q7-4" class="bloc
 <a href="#q7-5" class="bloc
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-8" class=
er-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля В

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q8-1" class="bloc
 <a href="#q8-2" class="bloc
 <a href="#q8-3" class="bloc
 <a href="#q8-4" class="bloc
```



```
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч.

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-light text-slate-500 pl-3 text-slate-400">


```

```

const observer = new IntersectionObserver(
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;

 // Highlight active section
 document.querySelectorAll('a').forEach(a => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith(visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details
 const correspondingId = `#${visibleId}-details`;
 if (correspondingLink) {
 const details = document.querySelector(correspondingId);
 if (details && !details.open) {
 details.open = true;
 document.querySelector('a').forEach(a => {
 if (a.href.endsWith(correspondingId)) {
 a.click();
 }
 });
 }
 }
 }
 });
}, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
});

sections.forEach(section => {

```



```
const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';
```

```
const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';
```

```
const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = 'white';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(contentRaw);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};
```

```
const closeBtn = document.createElement('button');
closeBtn.innerText = 'Заккрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = 'white';
```

```

 text = text.replace(/\n{3,}/g,
 '\n\n\n');

 if (window !== window.top) {
 fallbackModal(text, 'Markdown');
 return;
 }
 const blob = new Blob([text], {
 type: 'text/markdown'
 });
 const url = URL.createObjectURL(blob);
 const link = document.createElement('a');
 link.href = url;
 link.download = 'vyisshaya_algebra.md';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-900');

 document.getElementById('theme-dark').checked = theme === 'dark';
 document.getElementById('theme-light').checked = theme === 'light';
 document.getElementById('theme-auto').checked = theme === 'auto';

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-900');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `

```

```

body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`;
} else if (theme === 'terminal')
body.classList.add('bg-black')
document.getElementById('theme')
style.innerHTML = `
body.bg-black .bg-slate-950 {
background-color: #000000;
}
body.bg-black section h1,
, body.bg-black button {
color: #4ade80 !important;
border-color: #22c55e !important;
font-family: monospace;
}
body.bg-black svg path,
stroke: #22c55e !important;
fill: transparent !important;
}
`;
}

```

```

 ">* </button>
 <button id="theme-terminal"
- center justify-center text-green-400 hover:
терминала"> </button>
 </div>
 </div>
 </div>
 </aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16
-base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 Высшая алгебра: теория и практика

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-center">
 <h2 class="font-bold text-xl mb-4">Print Table of Contents
 <ul class="text-base space-y-2">
 1. Алгебра
 2. Коммутативная алгебра
 3. Топология
 4. Статистика
 5. Группы
 6. Дифференциальная геометрия
 7. Алгебраические кривые
 8. Квантовая механика
 9. Комплексные функции
 10. Теория вероятностей
 11. Теория чисел
 12. Теория игр

```

```

<a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

<!-- Row 2: Базовые операции / Евклид
<a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

<a href="#question-10" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

<a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

<!-- Row 3: НОД и НОК / Алгоритм Ев
<a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

<a href="#question-12" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

<!-- Empty 3rd row cell -->
<div class="hidden md:block"></div>

<!-- Row 4: Факторизация / Структур

```

```

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setl
-l-4 border-orange-500 hover:bg-slate-700 t
<div class="text-xs text-orange
<div class="font-bold text-slate

<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

<a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
<div class="text-xs text-teal-5
<div class="font-bold text-slate

<a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
<div class="text-xs text-blue-5
<div class="font-bold text-slate

<a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
<div class="text-xs text-emeral
<div class="font-bold text-slate

<a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
<div class="text-xs text-rose-5
<div class="font-bold text-slate

<a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
<div class="text-xs text-indigo
<div class="font-bold text-slate

```

```
 <h2 class="text-2xl font-bold b
(Меметика)</h2>
```

```
 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-blur
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
k$).
 <span class="text-w
 <span class="text-w
ово.

 </div>
```

```
 <!-- От противного -->
 <div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-fuch
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
i>
 <span class="text-w
т.д.).
 <span class="text-w
что так и задумано: "Ой, противоречие, теор

 </div>
```

```
 <!-- Единственность -->
 <div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-eme
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
, r_2$).
 <span class="text-w
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 со следующим. Если в конце 0 – пациент мертв.
 </p>
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 c. Пока получается 0 – корень жив. Как то.
 корня минус один.</p>
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 ициентов (они делятся на p), кроме старшего.
 зложить.</p>
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 . Чаще всего после этого Фейс-контроль p .
 </div>
 </div>
```

```
 <h2 class="text-2xl font-bold bg-slate-800 p-4">
 (Правила экстрасенса)</h2>
 <p class="text-sm text-slate-400">
 тношения), а ты их не помнишь. Юзай базовые
 нейность) для делимости.</p>
```



```


 </div>
 </section>
</div>
```

```
<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 87/111: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 88/111: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 341 | РАЗМЕР: 15.5 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";
```

```

.export-head { border-bottom: 1px solid #1e293b; padding: 1rem 0 0 0; }
.export-note { background: #0f172a; border: 1px solid #0f172a; border-radius: .5rem; padding: .85rem 1rem; font-size: .95rem; }
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid #0f172a; padding-top: 1rem; }
.export-gloss h2 { color: #fff; font-size: 1.25rem; font-weight: bold; }
.export-term { background: #0f172a; border: 1px solid #0f172a; padding: 1rem; }
.export-term h3 { color: #c7d2fe; font-size: .95rem; font-weight: bold; }
.export-term p { margin: 0 0 .3rem 0; font-size: .85rem; }
.export-term .formal { color: #94a3b8; font-size: .8rem; }
.export-term .cx { color: #6ee7b7; font-size: .78rem; font-weight: bold; }
.export-foot { margin-top: 3rem; border-top: 1px solid #0f172a; padding-top: 1rem; }
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65; }
.export-shell pre { font-size: 12px; line-height: 1.5; }
.export-figs { margin: 0 0 1.6rem 0; }
.export-figs-title { color: #e2e8f0; font-size: .95rem; font-weight: bold; }
.export-fig { margin: 0 0 .9rem 0; }
.export-fig img { width: 100%; border: 1px solid #1e293b; }
.export-fig figcaption { color: #94a3b8; font-size: .75rem; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
`
;

```

```

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string, opts: { selfId?: string;
 body: string;

```

```

 el.replaceWith(link);
 });

 // <details> в файле лучше раскрыть: иначе код и разб
 host.querySelectorAll("details").forEach((d) => d.set

 return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[], book = false) {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
<h2>${book ? "Словарик терминов пособия" : "Словарик
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
 ны сюда.</p>
 ${items}
</section>`;
}

function wrap(opts: { title: string; subtitle: string;
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">

```

```

const imgs = Array.from(doc.querySelectorAll("img[src
await Promise.all(
 imgs.map(async (img) => {
 const src = img.getAttribute("src") ?? "";
 if (!src || src.startsWith("data:")) return;
 try {
 const res = await fetch(src);
 if (!res.ok) return;
 const blob = await res.blob();
 if (blob.size > 4_000_000) return;
 const data = await new Promise<string>((resolve
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
));
 img.setAttribute("src", data);
 } catch {
 /* картинка недоступна — глава останется со ссы
 }
 })
);
return doc.body.innerHTML;
}

```

/\*\*

\* Кадры визуализаций: книга встраивает снимки демонстр  
 \* файл показывал, что крутилось в приложении. Снимки д  
 \* (capture-viz-shots.mjs) и публикует в /export/viz-sh  
 \* без них (офлайн-сборка, локальный дев) книга остаётс  
 \* нормально и молча пропускается.  
 \*/

```

async function vizFigures(chapterId: string): Promise<s
 try {
 const base = import.meta.env.BASE_URL || "/";
 const manifestRes = await fetch(`${base}export/viz-
 if (!manifestRes.ok) return "";
 const manifest = (await manifestRes.json()) as { fi

```

## Универсальное пособие • полный исходный код

```

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
 const num = title.match(/^s*(\d+)/)?.[1];
 const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-": ""}${base}`;
}
```

```
/** Собрать автономный HTML одной главы. */
export async function buildChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body: rawBody, termIds } = prepareBody(chapter);
 const body = (await vizFigures(chapter.id)) + (await vizText(chapter.id));
 return wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
}
```

```
/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 triggerDownload(await buildChapterHtml(chapter), safeName(chapter.title, chapter.id));
}
```

```
/** Собрать книгу: всё пособие одним файлом со сквозным индексом.
export async function buildBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `-
 .join("\n");

 for (const c of chapters) {
```

```

import { useEffect } from "react";
import { chapterTopics } from "../data/content";

/**
 * Перекрёстные ссылки между билетами.
 *
 * В тексте страниц десятки упоминаний вида «...берите Фор
 * или «сердце Краскала (билет 18)». Раньше это был про
 * номер, но попасть на него не мог — ссылка была «битой
 * в никуда. Здесь такие упоминания превращаются в настоя
 *
 * • в приложении — `

```

```

 const closing = part.startsWith("</");
 const tag = part.slice(closing ? 2 : 1).split(/
 if (tag === "pre" || tag === "code" || tag ===
 inCode = Math.max(0, inCode + (closing ? -1 :
 } else if (tag === "a") {
 inLink = Math.max(0, inLink + (closing ? -1 :
 }
 return part;
}
if (inCode > 0 || inLink > 0) return part;
return part.replace(TICKET_RE, (whole, a: string,
 const from = Number(a);
 const to = b ? Number(b) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) return whole;
 return `<a class="ticket-link" data-goto="${id}
 });
});
.join("");
}

/**
 * DOM-версия — для интерактивной страницы. Проходит то
 * (как `annotateTerms`), поэтому атрибуты и разметка не
 * Возвращает число созданных ссылок — его проверяют DOM
 */
export function annotateTicketLinks(
 root: HTMLElement,
 opts: { selfId?: string; lite?: boolean } = {}
): number {
 if (typeof document === "undefined") return 0;
 const walker = document.createTreeWalker(root, SHOW_TEXT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
 const text = node.nodeValue ?? "";
 if (/билет/i.test(text)) return FILTER_REJECT;

```

```

}

/** Размечает ссылки при смене главы и подстраховывается
export function useTicketLinks(
 ref: React.RefObject<HTMLElement | null>,
 deps: unknown[],
 opts: { selfId?: string; lite?: boolean } = {}
) {
 useEffect(() => {
 const el = ref.current;
 if (el) annotateTicketLinks(el, opts);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, deps);
}
```

## =====

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

## =====

-----  
ФАЙЛ 90/111: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

-----

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(im
```



```

const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && l
const gapViz = rows.filter((r) => r.hasAnalogy && !has2
const orphanViz = [...vizIds].filter((id) => !chapters.

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) =>

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |
 console.log("| --- | --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy
 ge)} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

```

 .replace(/<[^>]+>/g, " ");

const perTerm = new Map();
const perSurface = new Map();
let highlights = 0;
const ids = new Set();

for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
}

if (ids.size === 0) bad.push(c.title);
console.log(
 String(ids.size).padStart(3), "|", String(highlights).padStart(3),
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 47).join(", ")
);
}
console.log("\nГлав без единой подсказки:", bad.length, "шт.");
console.log("Терминов в глоссарии:", GLOSSARY.length, "шт.");
console.log("Не встретились:", GLOSSARY.filter((g) => !bad.includes(g)).length, "шт.");

```

```

nodes: [
 { id: "A", label: "A", x: 40, y: 60, dist: "0", s
 { id: "B", label: "B", x: 130, y: 30, dist: "3",
 { id: "C", label: "C", x: 130, y: 90, dist: "2",
 { id: "G", label: "G", x: 220, y: 90, dist: "5",
 { id: "D", label: "D", x: 220, y: 30, dist: "6",
 { id: "E", label: "E", x: 310, y: 90, dist: "7",
 { id: "F", label: "F", x: 400, y: 60, dist: "8",
],
edges: [
 { u: "A", v: "B", w: 4, highlighted: true },
 { u: "A", v: "C", w: 2, highlighted: true },
 { u: "C", v: "B", w: 1, highlighted: true },
 { u: "B", v: "D", w: 5 },
 { u: "C", v: "G", w: 3, highlighted: true },
 { u: "C", v: "E", w: 8 },
 { u: "G", v: "D", w: 1, highlighted: true },
 { u: "G", v: "E", w: 2, highlighted: true },
 { u: "D", v: "F", w: 2 },
 { u: "E", v: "F", w: 1, highlighted: true },
],
table: [
 ["Вершина", "Старт", "Кратчайшее dist", "Предшест
 ["Пиццерия А", "s = A", "0", "-", "зафиксирована"
 ["Парк С", "из А", "2", "А", "зафиксирована"],
 ["Дом 1 В", "из С", "3", "С", "зафиксирована"],
 ["Метро G", "из С", "5", "С", "зафиксирована"],
 ["Офис D", "из G", "6", "G", "зафиксирована"],
 ["Дом 2 E", "из G", "7", "G", "зафиксирована"],
 ["ТЦ F", "из E", "8", "E", "зафиксирована"],
],
},
"bellman-ford": {
 title: "Векторный граф релаксации (Форд–Беллман)",
 nodes: [
 { id: "S", label: "S", x: 40, y: 60, dist: "0", s
 { id: "A", label: "A", x: 130, y: 30, dist: "4",
 { id: "B", label: "B", x: 130, y: 90, dist: "5",

```

```

 title: "Минимальный остов (Краскал / Прим / Боровка
 nodes: [
 { id: "1", label: "1", x: 50, y: 30, status: "done" },
 { id: "2", label: "2", x: 150, y: 30, status: "done" },
 { id: "3", label: "3", x: 250, y: 30, status: "done" },
 { id: "4", label: "4", x: 50, y: 90, status: "done" },
 { id: "5", label: "5", x: 150, y: 90, status: "done" },
 { id: "6", label: "6", x: 250, y: 90, status: "done" },
 { id: "7", label: "7", x: 350, y: 60, status: "done" },
],
 edges: [
 { u: "1", v: "2", w: 2, highlighted: true },
 { u: "2", v: "3", w: 3, highlighted: true },
 { u: "1", v: "4", w: 1, highlighted: true },
 { u: "4", v: "5", w: 4, highlighted: true },
 { u: "2", v: "5", w: 5 },
 { u: "3", v: "6", w: 2, highlighted: true },
 { u: "5", v: "6", w: 6 },
 { u: "6", v: "7", w: 3, highlighted: true },
],
 table: [
 ["Ребро", "Вес w", "Краскал (DSU)", "Прим (Куча)"],
 ["(1, 4)", "1", "добавлено (comp 1-4)", "добавлено"],
 ["(1, 2)", "2", "добавлено (comp 1-2-4)", "добавлено"],
 ["(3, 6)", "2", "добавлено (comp 3-6)", "добавлено"],
 ["(2, 3)", "3", "добавлено (объединение)", "добавлено"],
 ["(6, 7)", "3", "добавлено (финиш)", "добавлено в"],
],
 },
};

/** A4 в пунктах. */
const PAGE = { width: 595.28, height: 841.89, margin: 40 };
const CONTENT_W = PAGE.width - PAGE.margin * 2;
const TOP = PAGE.margin;
const BOTTOM = PAGE.height - PAGE.margin - 16;

/* ----- шрифты -----

```

```

const mod10 = Math.abs(n) % 10;
const mod100 = Math.abs(n) % 100;
if (mod10 === 1 && mod100 !== 11) return one;
if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 10))
 return many;
};

const plain = (text) =>
 String(text ?? "")
 .replace(UNSUPPORTED, "")
 .replace(/\s{2,}/g, " ")
 .replace(/\u0000/g, "")
 .trim();

/* ----- данные приложения (через esbuild) ----- */

async function loadAppData() {
 ensureDir(path.dirname(ENTRY));
 fs.writeFileSync(
 ENTRY,
 [
 'export { chapters, chapterTopics } from "../src/chapters";',
 'export { PAGE_SYNC } from "../src/data/vizSync";',
 'export { VIZ_REGISTRY } from "../src/components/vizRegistry";',
 'export { quizzes } from "../src/data/quizzes";',
].join("\n")
);
};

const res = await build({
 entryPoints: [ENTRY],
 bundle: true,
 write: false,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 logLevel: "silent",
 loader: { ".ts": "ts", ".tsx": "tsx", ".jpg": "dataurl", ".css": "empty" },
});

```

\*

\* Важно: на время печати нижнее поле страницы уменьш

\* решает, что текст не влез, и молча добавляет пусты

\*/

```
stampFooter() {
 if (this.pages === 0 || !this.doc.page) return;
 const prevBottom = this.doc.page.margins.bottom;
 this.doc.page.margins.bottom = 16;
 const y = PAGE.height - PAGE.margin + 4;
 this.doc.font(this.fonts.regular).fontSize(7.6).fil
 this.doc.text("Универсальное пособие • алгоритмы и
 width: CONTENT_W * 0.62,
 align: "left",
 lineBreak: false,
 ellipsis: true,
 });
 this.doc.text(`${this.footerTitle} • ${this.pages}`
 width: CONTENT_W * 0.38,
 align: "right",
 lineBreak: false,
 ellipsis: true,
 });
 this.doc.page.margins.bottom = prevBottom;
 this.doc.y = this.cy;
}
```

```
newPage(footerTitle) {
 this.stampFooter();
 if (footerTitle !== undefined) this.footerTitle = f
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
 this.cy = TOP;
}
```

```
firstPage(footerTitle) {
 if (footerTitle !== undefined) this.footerTitle = f
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
```

```

 current = sentence;
 continue;
 }
 for (const word of sentence.split(/\s+/)) {
 const withWord = current ? `${current} ${word}` : word;
 if (measure(withWord) <= max) current = withWord;
 else {
 flush();
 current = word;
 }
 }
 flush();
 return chunks.length ? chunks : [source];
}

rule(color = "#c7d2fe", width = 1) {
 const y = this.cy;
 this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin + width, y);
 this.cy = y + 8;
}

h1(rawText, { size = 20, color = "#111827", rule = true } = {}) {
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + (rule ? 14 : 6) + gapBefore, 22);
 this.cy += gapBefore;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
 this.cy = Math.max(this.cy, this.doc.y) + 2;
 if (rule) this.rule();
}

h2(rawText, { size = 13.5, color = "#1e293b" } = {}) {
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + 6, 20);
}

```

```

const marker = ordinal ? `${ordinal}.` : "*";
const width = CONTENT_W - indent - 4;
const opts = { font: this.fonts.regular, size, width };
const chunks = this.splitToFit(text, opts, BOTTOM);
chunks.forEach((chunk, index) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 2.5, 6);
 const y0 = this.cy;
 if (index === 0) {
 this.doc.font(this.fonts.regular).fontSize(size).text(
 marker, { align: "right", lineGap: 1 });
 }
 this.doc.font(this.fonts.regular).fontSize(size).text(chunk);
 this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
 if (this.cy > BOTTOM) this.newPage();
});
}

```

```

/** Код: режим на фрагменты по высоте страницы, каждый фрагмент
code(rawText, { size = 8.0 } = {}) {
 const font = this.fonts.mono;
 const inner = CONTENT_W - 22;
 const lines = String(rawText ?? "").replace(UNSUPPORTED_CHARS, "");
 // собираем фрагменты, которые влезают в одну рамку
 let i = 0;
 while (i < lines.length) {
 // Если у низа страницы осталось меньше минимальной высоты
 // СРАЗУ: иначе рамка уедет за BOTTOM, и pdfkit сглотит ошибку
 // в обход рендерера (без колонтитула и нумерации)
 if (BOTTOM - this.cy - 16 < 60) this.newPage();
 const avail = BOTTOM - this.cy - 16;
 const maxH = avail;
 let chunk = [];
 let h = 0;
 while (i < lines.length) {
 const candidate = chunk.concat(lines[i]).join(" ");
 const ch = this.height(candidate, { font, size, width });
 if (chunk.length && ch > maxH) break;

```



```

 const x1 = PAGE.margin + fromNode.x;
 const y1 = startY + fromNode.y;
 const x2 = PAGE.margin + toNode.x;
 const y2 = startY + toNode.y;

 const strokeColor = e.highlighted ? "#059669" : "#1a3a5a";
 const strokeWidth = e.highlighted ? 2 : 1;

 this.doc
 .moveTo(x1, y1)
 .lineTo(x2, y2)
 .lineWidth(strokeWidth)
 .strokeColor(strokeColor)
 .stroke();

 if (e.w !== undefined) {
 const mx = (x1 + x2) / 2;
 const my = (y1 + y2) / 2;
 this.doc
 .circle(mx, my, 7)
 .fillAndStroke("#ffffff", strokeColor);
 this.doc
 .font(this.fonts.bold)
 .fontSize(7)
 .fillColor("#0f172a")
 .text(String(e.w), mx - 7, my - 3.5, { width: 14 });
 }
 }
}

// Вершины графа
for (const n of nodes) {
 const nx = PAGE.margin + n.x;
 const ny = startY + n.y;
 const r = 12;

 const fillColor = n.status === "done" ? "#dcf7e7" : "#f7f7f7";
 const strokeColor = n.status === "done" ? "#16a34a" : "#1a3a5a";

```

```

 return max + pad * 2;
};

const drawRow = (cells, y, isHeader, zebra) => {
 const h = rowHeight(cells);
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).fill(
 for (let i = 0; i < cols; i++) {
 const x = PAGE.margin + widths.slice(0, i).reduce(
 this.doc
 .font(isHeader ? this.fonts.bold : this.fonts
 .fontSize(size)
 .fillColor("#1f2937")
 .text(String(cells[i] ?? ""), x, y + pad, { w
 }
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineW
 return h;
};

```

```

this.ensure(rowHeight(header) + 8, 14);
let y = this.cy;
rows.forEach((cells, idx) => {
 const row = [];
 for (let i = 0; i < cols; i++) row.push(cells[i]
 const h = rowHeight(row);
 if (y + h > BOTTOM) {
 this.cy = y;
 this.newPage();
 y = this.cy;
 if (idx > 0) y += drawRow(header, y, true, false
 }
 y += drawRow(row, y, idx === 0, idx % 2 === 1);
});
this.cy = y + 8;
}

```

```

quote(rawText) {
 const text = plain(rawText);
 if (!text) return;

```

```

 this.cy = Math.max(this.cy, this.doc.y) + 6;
 }
 return true;
}

/** Бокс с заголовком и строками; длинные режутся на
box(rawTitle, rawLines, { color = "#eef2ff", border =
 const title = plain(rawTitle);
 const lines = rawLines.map((l) => plain(l)).filter(
 const inner = CONTENT_W - 20;
 let rest = lines.slice();
 let first = true;
 while (rest.length || first) {
 first = false;
 const headH = this.height(title, { font: this.fon
 this.ensure(headH + 24, 16);
 const y0 = this.cy;
 // сколько строк влезет на эту страницу
 const avail = BOTTOM - y0 - headH - 16;
 const taken = [];
 let h = 0;
 while (rest.length) {
 const ch = this.height(rest[0], { font: this.fon
 if (taken.length && h + ch > avail) break;
 taken.push(rest.shift());
 h += ch;
 }
 const boxH = headH + h + 12;
 this.doc.roundedRect(PAGE.margin, y0, CONTENT_W,
 this.doc.font(this.fonts.bold).fontSize(9.2).fill
 let y = y0 + 7 + headH;
 for (const l of taken) {
 this.doc.font(this.fonts.regular).fontSize(size
 ;
 y += this.height(l, { font: this.fonts.regular,
 }
 this.cy = y0 + boxH + 7;
 if (rest.length) this.newPage();
 }
}

```

```

/* — обложка — */
R.firstPage("Обложка");
R.cy = TOP + 90;
R.h1("Универсальное пособие", { size: 26, rule: false });
R.h1("Алгоритмы и структуры данных", { size: 17, color: "#4b5563" });
R.para(
 `Билеты 1-${totalTopics} · ${chapters.length} ${plus}
 Count, "интерактивная демонстрация", "интерактивная синхронизация",
 { size: 10.5, color: "#4b5563" }
);
R.para(
 "Автоматический компактный сборник: весь текст тематический анализ, синхронизация. Навигация приложения и код панели Python Pages.",
 { size: 9.6, color: "#374151" }
);
R.para(`Дата сборки: ${today}`, { size: 9, color: "#6b728b" });

const missingViz = chapters.filter((c) => !VIZ_REGISTERED[c.id]);
R.box("Как устроено пособие", [
 "• Страница = один или несколько билетов: текст, анимация, видео, аудио, интерактив",
 "• У каждой страницы есть интерактивная демонстрация, которую можно посмотреть, кликнув на неё",
 "• В этом PDF демонстрации показаны снимками (когда они есть) и видео, которые можно посмотреть, кликнув на них",
 missingViz.length
 ? `• Страницы без демонстрации: ${missingViz.map(c => c.id).join(", ")}`
 : "• Демонстрация и синхронизация с компилятором Python Pages"
]);

/* — содержание — */
R.newPage("Содержание");
R.h1("Содержание", { size: 18 });
chapters.forEach((c, i) => {
 const topics = chapterTopics[c.id] ?? [];
 R.para(
 `${String(i + 1).padStart(2, "0")}. ${c.title}${topics.length > 0 ? " — " + topics.join(", ") : ""}`
);
});

```

```

 R.para(b.text, { size: 8.3, color: "#6b7280", i
 } else {
 R.para(b.text);
 }
}

/* иллюстрация страницы */
const img = CHAPTER_IMAGES[chapter.id];
if (img && fs.existsSync(path.join(ROOT, img))) {
 R.h2("Иллюстрация-аналогия");
 const captions = {
 dijkstra: "✂ Декстер Дуглас у CRT-монитора и ви
 "bellman-ford": " Бэтмен патрулирует Готэм в м
 floyd: " Все вершины связаны со всеми",
 };
 R.image(path.join(ROOT, img), { maxHeight: 220, s
}

/* демонстрация: интерактивная схема (векторные гра
const viz = VIZ_REGISTRY[chapter.id];
const vecViz = VECTOR_VISUALIZERS[chapter.id];
const syncKeys = Object.keys(PAGE_SYNC).filter((k)

if (viz || syncKeys.length || vecViz) {
 R.h2("Демонстрация и интерактивная схема");
 if (viz) {
 R.para(viz.title, { size: 10.2, color: "#1e293b
 if (viz.hint) R.para(viz.hint, { size: 9.1, col
 }

 if (vecViz) {
 if (vecViz.nodes && vecViz.edges) {
 R.vectorGraph(vecViz.nodes, vecViz.edges, { h
 }
 if (vecViz.table) {
 R.table(vecViz.table);
 }
 }
}

```

```
// Самопроверка: страниц в файле должно быть ровно столько, сколько
// рендерер. Расхождение означает, что pdfkit создал файл с лишними
// в файл без нижнего колонтитула и без нумерации.
const pdfLib = await import("pdf-lib");
const physical = (await pdfLib.PDFDocument.load(fs.readFileSync(
console.log(` страниц: ${R.pages} (рендер) / $
if (R.pages > 100) console.warn(` ВНИМАНИЕ: ${R.pages}
if (physical !== R.pages) {
 console.error(` ОШИБКА: pdfkit добавил ${physical} страниц
 process.exitCode = 1;
}
}

main().catch((err) => {
 console.error("Ошибка сборки PDF пособия:", err.message);
 process.exit(1);
});
```

---

ФАЙЛ 93/111: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

---

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,
 ensureFile } from "../utils";

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };
```

```

 stream.on("error", reject);
 });

 console.log("[3/3] png -> pdf");
 console.log(`      страниц: ${pages.length}`);
 console.log(`      выход:   ${path.relative(ROOT, PDF)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

-----  
ФАЙЛ 94/111: scripts/export/capture-viz-shots.mjs  
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 122 | РАЗМ: 1.5 КБ  
-----

```
/**
 * Снимки визуализаций для PDF-конспекта: tmp/viz-shots.
 *
 * Headless-браузер открывает каждую страницу с демонстрацией
 * блок #chapter-viz. У страниц с вкладками (sparse-table)
 * снимается каждая вкладка — файл получает имя `<chapter>-<tab>`.
 * и PDF подписывает его как «режим «вкладка»».
 *
 * npm run export:shots # сервер уже поднят
 * SHOTS_BASE_URL=http://127.0.0.1:4173 npm run export:shots
 *
 * В CI это делает rebuild-pdf.yml: сборка → vite preview →
 * npm run export:guide. Локально без браузера скрипт просто
 * честно пишет, что снимки добавляются при сборке в CI.
 */
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import puppeteer from "puppeteer";
```

```

const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями:

const browser = await puppeteer.launch({
 headless: true,
 args: ["--no-sandbox", "--disable-setuid-sandbox", "--no-zygote-sandbox"],
});
const page = await browser.newPage();
await page.setViewport({ width: 1400, height: 1000, deviceScaleFactor: 1 });

let done = 0;
let failed = 0;
for (const id of ids) {
 try {
 await page.goto(`${BASE_URL}/?topic=${id}`, { waitUntil: 'load' });
 await page.waitForSelector("#chapter-viz", { timeout: 10000 });
 // демо-анимации: даём кадру собраться до осмысленного состояния
 await new Promise((r) => setTimeout(r, 1600));
 const viz = await page.$("#chapter-viz");
 if (!viz) throw new Error("нет #chapter-viz");
 await viz.screenshot({ path: path.join(OUT_DIR, `${id}.png`), fullPage: true });
 done += 1;

 for (const tab of TABS[id] ?? []) {
 try {
 const clicked = await page.evaluate((label) => {
 const root = document.querySelector("#chapter-viz");
 if (!root) return false;
 const btn = Array.from(root.querySelectorAll("button")).find(
 (b) => b.textContent ?? "" === label.trim()
);
 if (!btn) return false;
 btn.dispatchEvent(new MouseEvent("click", { bubbles: true }));
 return true;
 }, tab);
 if (!clicked) continue;
 await new Promise((r) => setTimeout(r, 1200));
 } catch (e) {
 failed += 1;
 }
 }
 } catch (e) {
 failed += 1;
 }
}
console.log(`[export:shots] страниц со визуализациями: ${done} (ошибок: ${failed})`);

```



## Универсальное пособие • полный исходный код

```

*
* Список файлов берётся из git (чтобы ничего не забыть
* с фоллбэком на обход файловой системы, если git недо
*/
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в
 [/^src\/components\/$/, "Интерактивные компоненты и си
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
```

```

 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 });
 }
}

```

/\*\* Достаём заголовки глав из данных пособия (без запус

```

function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
));
}

```

```

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

```

```

 const out = [];
 const push = (s = "") => out.push(s);

```

```

current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel));
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`файлов: ${files.length}`);
console.log(`глав: ${chapters.length}`);
console.log(`строк: ${txt.split("\n").length}`);
console.log(`выход: ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

```
 pointsize: 8,
 /** Максимум символов в строке (далее – жёсткий пере
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапк
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toF
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore"
 return cmd;
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-ge
);
}
```

```

*/
export function parseHtml(html) {
 const root = { tag: "#root", attrs: {}, children: [] };
 const stack = [root];
 const re = /<!--[\s\S]*?-->|<\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)\s*/;
 let m;
 while ((m = re.exec(html)) !== null) {
 const [full, closeTag, openTag, attrsStr, selfClose] = m;
 if (full.startsWith("<!--")) continue;

 if (closeTag) {
 const tag = closeTag.toLowerCase();
 // закрываем до совпадения (прощает незакрытые
 и т.п.)
 for (let i = stack.length - 1; i >= 1; i--) {
 if (stack[i].tag === tag) {
 stack.length = i;
 break;
 }
 }
 continue;
 }

 if (openTag) {
 const tag = openTag.toLowerCase();
 const attrs = {};
 const attrRe = /([a-zA-Z-_:]+)\s*=\s*"([^"]*)"|'([^']*)'|([a-zA-Z-_:]+)\s*/;
 let a;
 while ((a = attrRe.exec(attrsStr || "")) !== null) {
 const node = { tag, attrs, children: [] };
 stack[stack.length - 1].children.push(node);
 if (!VOID.has(tag) && !selfClose) stack.push(node);
 continue;
 }

 if (textChunk !== undefined && textChunk.length) {
 stack[stack.length - 1].children.push({ tag: "#text", text: textChunk });
 }
 }
 }
}

```

```

 out += c.text;
 } else if (SKIP.has(c.tag)) {
 continue;
 } else if (BLOCK.has(c.tag)) {
 continue;
 } else if (c.tag === "br") {
 out += "\n";
 } else {
 out += inlineText(c);
 }
}
return clean(out);
}

/** Весь текст поддерева (для ячеек таблиц и подписей).
function collectText(node) {
 if (node.tag === "#text") return clean(node.text);
 if (SKIP.has(node.tag)) return "";
 if (node.tag === "table") return tableRows(node).map(
 return clean(node.children.map(collectText).join(" "))
)
}

/** Сырой текст внутри <pre> – переносы строк важны. */
function collectRawPre(node) {
 if (node.tag === "#text") return node.text;
 if (node.tag === "br") return "\n";
 if (SKIP.has(node.tag) && node.tag !== "img") return
 return node.children.map(collectRawPre).join("");
}

/** Таблица → массив строк из массивов ячеек. */
function tableRows(node) {
 const rows = [];
 const visit = (n) => {
 if (n.tag === "tr") {
 const cells = n.children.filter((c) => c.tag ===
 if (cells.some((c) => c.length)) rows.push(cells)
 return;

```

```

const walk = (node, listDepth = 0) => {
 if (node.tag !== "#root" && SKIP.has(node.tag)) return;

 if (node.tag === "table") {
 const rows = tableRows(node);
 if (rows.length) push({ type: "table", rows });
 return;
 }
 if (node.tag === "pre") {
 const text = decodeEntities(collectRawPre(node));
 if (text.trim()) push({ type: "code", text: text });
 return;
 }

 // есть блочные дети: идём по порядку, инлайн-пробе
 if (hasBlockChild(node)) {
 let buf = "";
 const flush = () => {
 const t = clean(buf);
 if (t) push({ type: "p", text: t });
 buf = "";
 };
 for (const c of node.children) {
 if (c.tag === "#text") {
 buf += c.text;
 continue;
 }
 if (SKIP.has(c.tag)) continue;
 if (!BLOCK.has(c.tag)) {
 buf += inlineText(c) + " ";
 continue;
 }
 if (c.tag === "ul" || c.tag === "ol") {
 flush();
 emitList(c, listDepth);
 continue;
 }
 }
 }
}

```

```

 return lines.join("\n").replace(/\n{3,}/g, "\n\n").trim()
}
```

-----  
ФАЙЛ 98/111: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 3.5 КБ

```

/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(/
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent
```



```

 });

 const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
 "-pointsize", String(PAGE.pointsized),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

 const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
 let done = 0;
 let cursor = 0;

 async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`отрендерено ${done} страниц\n`);
 }
 }
 }

 await Promise.all(Array.from({ length: concurrency }, worker));
 process.stdout.write("\n");

 const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
 if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);
 }
}

```

```

 group: string;
 name: string;
 ok: boolean;
 ms: number;
 detail: string;
 consoleErrors: string[];
}

const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;

/* -----
/* утилиты
/* -----

const sleep = (ms: number) => new Promise((r) => setTime

async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
 });
}

function bodyText(): string {
 return (document.body.textContent ?? "").replace(/\s+/g, ' ');
}

function allButtons(): string[] {
 return Array.from(document.querySelectorAll("button"))
 .filter((b) => (b.getAttribute("aria-label") || b.getAttribute("title")));
}

/** Кнопки без доступного имени: ни aria-label, ни title
function namelessButtons(scope: ParentNode = document): string[] {
 return Array.from(scope.querySelectorAll("button"))
 .filter((b) => {

```

```
function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window, 'innerText')
 if (!setter) throw new Error("нет нативного setter'a")
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubble: true }));
 });
}
```

```
async function key(k: string, init: KeyboardEventInit = {}) {
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("keydown", init));
 await sleep(20);
 });
}
```

```
/** Ждём появления признака в DOM (текст/предикат), перепроверяем каждые 20мс */
async function waitFor(pred: () => boolean, timeoutMs = 1000) {
 const deadline = Date.now() + timeoutMs;
 while (Date.now() < deadline) {
 if (pred()) return true;
 await settle(step);
 }
 return pred();
}
```

```
const RUN_DONE = [
 "Трасса готова автоматически",
 "Нет шагов трассы",
 "Пустой код",
 "Код не выполнялся",
 "Выполнение остановлено после",
 "Не удалось выполнить код",
 "Ошибка:",
];
```

```
let root: Root | null = null;
```

```

const entry = VIZ_REGISTRY[vizId];
if (!entry) throw new Error(`в реестре нет визуализации`);
const Comp = entry.Component;
root = createRoot(host);
await act(async () => {
 root!.render(
 <VizChapterContext.Provider value={vizId}>
 <VizLiveStatus demoTitle={entry.title} />
 <Comp />
 </VizChapterContext.Provider>
);
});
await settle(60);
return host;
}

```

```

const BAD_DOM = [
 { re: /\bNaN\b/, why: "NaN в разметке" },
 { re: /\bundefined\b/, why: "undefined в разметке" },
 { re: /\[object Object\]/, why: "[object Object] в разметке" },
];

```

/\*\*

\* Битые подстановки видны в КОРОТКИХ текстовых узлах (листья  
 \* вершины): «undefined», «NaN», «[object Object]». В длинных  
 \* псевдокода те же слова законны — их не трогаем.  
 \*/

```

function domDefects(scope: HTMLElement | null): string[] {
 if (!scope) return [];
 const found: string[] = [];
 const walker = document.createTreeWalker(scope, 4 /* NodeFilter.SHOW_TEXT */);
 let node = walker.nextNode() as Text | null;
 while (node) {
 const text = (node.nodeValue ?? "").replace(/\s+/g, " ");
 if (text.length > 0 && text.length < 60) {
 for (const bad of BAD_DOM) {
 if (bad.re.test(text)) found.push(`${bad.why}: ${text}`);
 }
 }
 node = walker.nextNode();
 }
 return found;
}

```

```

const onlyChapters = (process.env.HAR_CHAPTERS || "").split(",");
const skipGroups = new Set((process.env.HAR_SKIP || "").split(","));
/** HAR_SYNC=floyd,dijkstra – прогнать только эти записи */
const onlySync = new Set((process.env.HAR_SYNC || "").split(","));

export async function runSuite({ py }: { py: Py }): Promise<void> {
 const chapterList = onlyChapters.length ? chapters.filter(chapter => onlyChapters.includes(chapter.id)) : chapters;
 const want = (group: string) => !skipGroups.has(group);
 // перехват console.error приложения (React пишет сюда)
 console.error = (...args: unknown[]) => {
 consoleErrors.push(args.map(a => a instanceof Error ? a : null));
 };

 /* — 1. Панель Python: реальный запуск на каждой главе */
 for (const chapter of chapterList) {
 const group = "A. страницы";
 await test(group, `${chapter.id}: монтируется, бейдж`);
 await mountApp(chapter.id);
 const txt = bodyText();
 if (!txt.includes(chapter.title.slice(0, 18))) throw new Error("некорректная загрузка");
 if (!VIZ_REGISTRY[chapter.id]) return "визуализация не найдена";
 if (!/встроенное демо|данные из компилятора/.test(txt)) return "некорректная загрузка";
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в DOM: ${defects.join(", ")}`);
 const nameless = namelessButtons(host!);
 if (nameless.length) throw new Error(`кнопок без имени: ${nameless.join(", ")}`);
 return `бейдж «встроенное демо» на месте, кнопок: ${nameless.length}`;
 });

 await test(group, `${chapter.id}: панель открывается`);
 await mountApp(chapter.id);
 const toggle = findButton("Боковая панель Python");
 await click(toggle, "кнопка панели Python в навбаре");
 const ta = textarea();
 if (!ta) throw new Error("редактор (textarea) не найден");
 if (!ta.value.includes("# «")) return `редактор не настроен`;
 const eye = findButton(/эта?лонный код/i);
 if (!eye) throw new Error("нет кнопки-«глаза» (встроенное демо)");
 await click(eye, "кнопка «эта?лонный код»");
 const code = bodyText();
 if (!code.includes(chapter.title)) return "некорректная загрузка";
 if (!code.includes("VIZ_REGISTRY")) return "некорректная загрузка";
 if (!code.includes("const defects")) return "некорректная загрузка";
 if (!code.includes("const nameless")) return "некорректная загрузка";
 if (!code.includes("return `бейдж «встроенное демо»")) return "некорректная загрузка";
 return `корректная загрузка`;
}

```

```

 { name: "пустая матрица", vars: { n: 0, k: 0, i: 0,
 { name: "матрица под именем D и k/i/j вне диапазона
 [10, 11, 12, 0]] }, expect: "4x4" },
];

for (const c of floydCases) {
 await test("В. Флойд: размерности", c.name, async () => {
 await mountViz("floyd");
 await act(async () => {
 emitVizState("floyd", { line: 3, func: "<module
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\\s
 if (!/данные из компилятора/.test(txt)) throw new
 if (!txt.includes(c.expect)) throw new Error(`в р
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в
 return `бейдж: ${txt.match(/данные из компилятор
 });
}

/* — 3. Префиксные суммы 2D и разреженная таблица —
const gridCases: { viz: string; chapter: string; name
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 9, 10, 11, 12]], S: [[1, 3, 6, 10], [6, 14, 24, 3
 тора/ },
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 pref: [[1, 3], [4, 10]], r1: 0, c1: 0, r2: 1, c2
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 9]], S: [[1, 3, 6], [5, 12, 21], [12, 27, 45]],
 { viz: "sparse-table", chapter: "sparse-table", nam
 [3, 3, 1, 1, 0], [1, 1, 0, 0, 0]] }, expect: /не
 { viz: "sparse-table", chapter: "sparse-table#2d-bu
 0, i: 2, j: 3, st: Array.from({ length: 4 }, () :
 pa/ },
 { viz: "sparse-table", chapter: "sparse-table#2d-qu
 30, j2: 40, res: 2 }, expect: /не совпадает|данны
};

```

```

 throw new Error(`пустая активная клетка нарисована
 }
 if (domDefects(host).length) throw new Error(`артеф
 return "оригинал из A с первого шага, вкладка «Шаг
});

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин
for (const [viz, vars] of [
 ["dijkstra", { u: 0, v: 3, w: 4, dist: [0, 4, 2, 6,
 ["bellman-ford", { n: 5, m: 4, i: 1, u: 2, v: 4, w:
] as [string, Record<string, unknown>][]) {
 await test("D. графы путей", `${viz}: вершины 0...4 в
 await mountViz(viz);
 const before = allButtons().length;
 const playEnabled = (findButton(/^Пуск|Пуск$/)) as
 await act(async () => {
 emitVizState(viz, { line: 5, func: "<module>",
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\\s
 if (!/данные из компилятора/.test(txt)) throw new
 if (!/ваши расстояния|dist/i.test(txt)) throw new
 const step = findButton(/^Шаг$|Шаг /) as HTMLButt
 if (step && !step.disabled) throw new Error("кноп
 if (domDefects(host).length) throw new Error(`арт
 await act(async () => {
 clearVizState(viz);
 await sleep(30);
 });
 const step2 = findButton(/^Шаг$|Шаг /) as HTMLBut
 if (step2 && step2.disabled) throw new Error("кноп
 return `кнопок до/после: ${before}/${allButtons()}
 });
}

```

```

/* — 5. Все референсные коды: настоящий Python → нас
const syncKeys = Object.keys(PAGE_SYNC).filter((k) =>
for (const key of syncKeys) {

```

```
}
```

```
/* — 6. Поведение редактора: пустой код, ошибки, гла
const ch = "dijkstra";
```

```
await test("F. редактор", "только комментарии: редакт
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "# просто комментариев\n# и ещё один\n"
 const done = await waitFor(() => bodyText().include
 if (!done) throw new Error(`не дождались сообщения
 if (!textarea()) throw new Error("редактор исчез по
 if (!/встроенное демо/.test(bodyText())) throw new
 return "редактор на месте, бейдж «встроенное демо»"
});
```

```
await test("F. редактор", "синтаксическая ошибка: ред
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "def broken(:\n pass\n");
 const done = await waitFor(() => /Код не выполнен
 if (!done) throw new Error(`ошибка синтаксиса не по
 if (!textarea()) throw new Error("редактор исчез на
 if (!/invalid syntax|SyntaxError/i.test(bodyText()))
 return "ошибка показана, редактор остался";
});
```

```
await test("F. редактор", "гигантский print обрезаетс
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, 'dist = {0: 0}\nprint("x" * 200000)\np
 const ready = await waitFor(() => bodyText().include
```



```

// — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор
await click(findButton("Вставить эталонный код"), "
const trace = await waitFor(() => !!findButton(/Оста
if (!trace) throw new Error(`эталон не запустился.
if (!textarea()) throw new Error("редактор исчез во
if (/Ошибка:/.test(panelText())) throw new Error(`э
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.
if (!ta2?.value.includes(firstRefLine)) {
 throw new Error(`выполняется не тот текст, что ле
 0, 100)}`);
}
const mustBe = [
 { what: "пуск/автопроход", re: /Запустить код|Про
 { what: "копирование кода", re: /Скопировать код|
 { what: "закрыть панель", re: /Закрыть компилятор
 { what: "шаг вперёд", re: /Шаг вперёд|вперёд/i },
 { what: "шаг назад", re: /Шаг назад|назад/i },
 { what: "свернуть листинг", re: /Свернуть листинг
];
const btns = allButtons().join(" | ");
for (const m of mustBe) {
 if (!m.re.test(btns)) throw new Error(`в панели н
}
// На первом шаге переменных ещё нет (отладчик вста
// поэтому связь с демонстрацией проверяем, дойдя д
await click(findButton("В конец трассы"), "в конец
const linked = await waitFor(() => /данные из компи
if (!linked) throw new Error(`эталон выполнен до ко

// — глаз ВЫКЛ: кода нет вовсе, ничего не выполняе
await click(findButton("Убрать эталонный код"), "гл
const cleared = await waitFor(() => bodyText().incl
if (!cleared) throw new Error(`после выключения гла
const ta3 = textarea();
if (!ta3) throw new Error("редактор пропал после вы
if (ta3.value.trim() !== "") throw new Error(`глаз
if (!/встроенное демо/.test(bodyText())) throw new

```

```

 if (!done) throw new Error(`трасса не собралась. Па
 if (!textarea()) throw new Error("во время прогона
 if (!findButton(/Остановить прогон трассы/)) throw
 // Листинг сворачивается (освобождая место редактор
 // и редактор остаются на месте.
 const codeLines = () => panel()?.querySelectorAll("
 if (codeLines() === 0) throw new Error("в раскрытой
 await click(findButton("Свернуть листинг трассы"),
 await settle(80);
 if (!textarea()) throw new Error("после сворачивани
 if (codeLines() !== 0) throw new Error(`листинг не
 if (!findButton("В конец трассы")) throw new Error(
 await click(findButton("Показать листинг трассы"),
 await settle(80);
 if (codeLines() === 0) throw new Error("листинг не
 // Esc: прогон остановлен, трасса убрана, редактор
 await key("Escape");
 await settle(150);
 if (!textarea()) throw new Error("Esc убрал редактор
 if (findButton(/Остановить прогон трассы/)) throw n
 if (!/встроенное демо/.test(bodyText())) throw new
 return "редактор всегда на месте: трасса рядом, сво
 });

```

```

await test("F. редактор", "закрытие панели во время пр
 await mountApp("floyd"); // 869 шагов: прогон долги
 await click(findButton("Боковая панель Python"), "п
 await waitFor(() => panel() !== null, 4000);
 await click(findButton("Заккрыть компилятор"), "крес
 await settle(150);
 if (panel()) throw new Error("панель не закрылась")
 const backToDemo = await waitFor(() => /встроенное
 if (!backToDemo) throw new Error(`после закрытия па
 await settle(3000); // ждём, пока догоняющий прогон
 if (/данные из компилятора/.test(bodyText())) {
 throw new Error("закрытая панель всё-таки опублик
 }
 if (!/встроенное демо/.test(bodyText())) throw new

```

```

const linked = await waitFor(() => /данные из компи.
if (!linked) throw new Error("в конце трассы визуал
// 4) сняли связь (Esc + пустой код) – снова демо
await key("Escape");
await settle(80);
const ta = textarea();
if (!ta) throw new Error("редактор не вернулся по E
typeInto(ta, "");
const cleared = await waitFor(() => /встроенное дем
if (!cleared) throw new Error("после очистки кода б
if (/переменных пока нет/.test(bodyText())) throw n
return "демо → выполнено/переменных нет → данные из
});

```

/\* — 7. Текстовый режим для PDF/парсеров —————

/\* — 9. Ссылки между билетами, разделы содержания, б.

```

await test("Н. доступность", "содержание сжимается в
await mountApp("bridges-code");
const root = () => document.querySelector<HTMLElemen
const toggle = () => findButton(/Сжать содержание|C
if (!root()) throw new Error("нет сайдбара");
if (!/lg:w-56/.test(root()!.className)) throw new E
await click(toggle(), "сжать");
if (!/lg:w-44/.test(root()!.className)) throw new E
const items = Array.from(root()!.querySelectorAll("
if (items.length < chapters.length) throw new Error
if (root()!.querySelector("input")) throw new Error
await click(toggle(), "скрыть");
if (!/lg:w-0/.test(root()!.className)) throw new Er
await click(toggle(), "развернуть");
if (!/lg:w-56/.test(root()!.className)) throw new E
return "широкое → узкое (темы на месте, поиска нет)
});

```

await test("J. шапка", "кнопки загрузки не прячутся и

```

await mountApp("segment-trees");

```

```

const menu = document.querySelector("#download-menu

```

```

 return `ссылка на страницу: ${links.length}, переход
 });

```

```

await test("I. ссылки", "содержание: разделы по порядку");
await mountApp("bridges-code");
const sidebar = document.querySelector("#sidebar-content");
if (!sidebar) throw new Error("нет сайдбара содержания");
const headers = Array.from(sidebar.querySelectorAll("h2"))
 .map((el) => (el.textContent ?? "").trim())
 .filter((t) => SECTIONS.some((sec) => sec.title === t));
if (headers.length !== SECTIONS.length) {
 throw new Error(`в содержании ${headers.length} разделов`);
}
const expected = SECTIONS.map((sec) => sec.title).join(" | ");
if (headers.join(" | ") !== expected) throw new Error("неверный порядок");
// 11 «Мосты» и 12 «Точки сочленения» читаются парой
const items = Array.from(sidebar.querySelectorAll("li"));
const iBridges = items.findIndex((t) => t.includes("Мосты"));
const iArtic = items.findIndex((t) => t.includes("Точки сочленения"));
if (iBridges < 0 || iArtic < 0) throw new Error("неверный порядок");
if (Math.abs(iArtic - iBridges) !== 1) throw new Error("неверный порядок");
// 2 «разреженная» и 5 «префиксные» – в одном разделе
if (sectionOf("sparse-table").id !== sectionOf("prefixes")) {
 throw new Error("билеты 2 и 5 попали в разные разделы");
}
if (sectionOf("bridges-code").id !== sectionOf("graphs")) {
 throw new Error("билеты 11 и 12 попали в разные разделы");
}
// порядок чтения: номера билетов сверху вниз не убывающий
const nums = Array.from(sidebar.querySelectorAll("b"))
 .map((b) => (b.querySelector("span")?.textContent ?? ""))
 .filter((t) => /^\\d/.test(t))
 .map((t) => Number.parseInt(t, 10));
const ticketed = chapters.filter((c) => (chapterTopic === c.title));
if (nums.length !== ticketed) {
 throw new Error(`номера билетов видны у ${nums.length} билетов`);
}
for (let i = 1; i < nums.length; i += 1) {

```

```

const shown = () => (section.textContent ?? "").rep
if (!shown().includes(list[0].question.slice(0, 24))
 throw new Error(`первый вопрос не показан: ${list[0].question}`);
}
const wrong = list[0].options.findIndex((_, i) => i !== correctIndex);
await click(ask(list[0].options[wrong].slice(0, 20), "неверно"));
await settle(80);
if (!/Неверно/.test(shown())) throw new Error("на неверном ответе");
if (!shown().includes(list[0].explanation.slice(0, 24)))
 throw new Error("после ответа нет разбора – пользуетесь подсказкой");
}
const nextBtn = ask("Следующий вопрос");
if (!nextBtn) throw new Error("нет кнопки «Следующий вопрос»");
await click(nextBtn, "дальше");
await settle(80);
if (!shown().includes(list[1].question.slice(0, 24)))
 // верный ответ
 await click(ask(list[1].options[list[1].correctIndex].slice(0, 20), "верно"));
 await settle(80);
 if (!/Верно/.test(shown())) throw new Error("на верном ответе");
 if (!/вопрос 2\\./.test(shown())) throw new Error(`с ${list[1].question} не совпало`);
 return `блиц из ${list.length} вопросов отвечает, продолжайте`;
});

await test("I. факты", "память: разреженная жирная, префиксная", async () => {
 const memoryRow = async (topic: string) => {
 await mountApp(topic);
 const rows = Array.from(document.querySelectorAll("table tbody tr"));
 const row = rows.find((r) => /^\\s*Память/.test(r.textContent));
 if (!row) throw new Error(`на странице «${topic}» не найдено «Память»`);
 return Array.from(row.querySelectorAll("td")).map((td) => td.textContent);
 };
 const sparse = await memoryRow("sparse-table");
 const prefix = await memoryRow("prefix-sums-2d");
 // билет 2: у префиксной N × M, у разреженной – с логарифмом
 if (!/log/.test(sparse.join(" "))) throw new Error("не найдено «log»");
 const prefixCell = sparse.find((c) => !/log/.test(c));

```

```
 return "билет 2: N×M×logN×logM против N×M; билет 5:
 });
```

/\* — 10. Сквозной путь: свой код чужой размерности, 1

```
await test("К. свой код", "Флойд 4×4 из редактора вед
 await mountApp("floyd");
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(
 ta,
 [
 "n = 4",
 "INF = 999",
 "d = [[0, 3, INF, 7], [8, 0, 2, INF], [5, INF,
 "k = 0",
 "i = 0",
 "j = 0",
 "for k in range(n):",
 " for i in range(n):",
 " for j in range(n):",
 " if d[i][k] + d[k][j] < d[i][j]:",
 " d[i][j] = d[i][k] + d[k][j]",
 "print(d[0], d[1][0])",
].join("\n")
);
 const ready = await waitFor(() => bodyText().includ
 if (!ready) throw new Error(`своя программа не выпол
 if (!bodyText().includes("[0, 3, 5, 6]")) {
 throw new Error(`Флойд на своей матрице 4×4 посчи
 }
 if (!textarea()) throw new Error("редактор исчез, п
 await click(findButton("В конец трассы"), "в конец
 const linked = await waitFor(() => /данные из компи
 if (!linked) throw new Error("демонстрация не приня
 const page = bodyText();
 if (!/4×4/.test(page)) throw new Error(`демонстраци
```

```

 await mountApp("scc-kosaraju");
 await click(findButton("Боковая панель Python"), "п
 if (!textarea()) throw new Error("нет редактора");
 await click(findButton("Вставить эталонный код"), "
 const ready = await waitFor(() => bodyText().includ
 if (!ready) throw new Error(`эталон Косарайю не вып
 if (/Ошибка:/.test(panelText())) throw new Error(`э
 const out = panelText();
 if (!/порядок выхода:/.test(out)) throw new Error(`
 // граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SC
 if (!/компонент: 2/.test(out)) throw new Error(`в гр
 if (!textarea()) throw new Error("редактор исчез, п
 await click(findButton("В конец трассы"), "в конец
 const linked = await waitFor(() => /данные из компи
 if (!linked) throw new Error("демонстрация не приня
 const defects = domDefects(document.querySelector("
 if (defects.length) throw new Error(`артефакты в де
 return "эталон Косарайю: order считается DFS-ом, на
 });

```

```

await test("G. lite-режим", "?lite=1 отдаёт текст без
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 window.history.replaceState({}, "", "/?lite=1&topic
 root = createRoot(host);
 await act(async () => {
 root!.render(<App />);
 });
 await settle(120);
 const txt = bodyText();
 if (txt.length < 2000) throw new Error(`lite-режим
 if (/main\.py|Pyodide/.test(txt)) throw new Error("

```

```

 g[r.ok ? "ok" : "fail"]++;
 byGroup.set(r.group, g);
 }
 return { total: results.length, passed: results.filter
 .fromEntries(byGroup) };
}
```

```
/** Инициализация страницы из vizSync – нужна для проверки
export const initTemplateOf = (id: string) => buildInit
```

-----  
ФАЙЛ 100/111: scripts/verify/check-links.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 199 | РАЗМЕР: 1000

```

/**
 * Аудит внутренних ссылок пособия: ничего не должно вешаться
 *
 * npm run verify:links
 *
 * Проверяет (по настоящим модулям приложения, собранным)
 * 1. билеты 1–24 закрыты страницами, id в chapterTopic
 * 2. разделы содержания: каждая страница в своём разделе
 * 3. упоминания «билет N» в тексте ведут на существующие
 * 4. все data-goto / href="?topic=..." указывают на существующие
 * 5. якоря href="#..." есть в той же главе;
 * 6. glossary.simulator существует в реестре и (кроме того)
 * имеет свою страницу – иначе модалка не получит данных
 * 7. glossary.demo – работающая мини-демо;
 * 8. у каждого ключа PAGE_SYNC есть запись в реестре
 * 9. у каждой страницы есть визуализация;
 * 10. страницы связаны перекрёстными ссылками (темы не исключение)
 */
import * as esbuild from "esbuild";
import { mkdirSync, writeFileSync } from "fs";

const OUT = "tmp/verify/links-entry.mjs";
```



```

const chapterIds = new Set(chapters.map((c) => c.id));
const registryIds = Object.keys(VIZ_REGISTRY);

/** Демо без собственной страницы: открываются из подсказки */
const EXTRA_DEMOS = new Set(["stack-dfs", "queue-bfs",

/* 1. билеты 1–24 */
const covered = new Set();
for (const [id, tickets] of Object.entries(chapterTopics)) {
 ok(chapterIds.has(id), `chapterTopics ссылается на не-
 tickets.forEach((n) => covered.add(n));
}
for (let n = 1; n <= 24; n += 1) {
 ok(covered.has(n), `билет ${n} не закрыт ни одной стр.
}

/* 2. разделы содержания */
for (const chapter of chapters) {
 const sec = sectionOf(chapter.id);
 ok(!!sec, `страница «${chapter.id}» не попала ни в од
 const tickets = chapterTopics[chapter.id] ?? [];
 ok(
 tickets.every((n) => n >= sec.from && n <= sec.to),
 `билеты страницы «${chapter.id}» (${tickets.join(",
 });
}
{
 const bySection = SECTIONS.map((s) => ({
 s,
 nums: chapters.flatMap((c) => (sectionOf(c.id).id =
 }));
 for (let i = 1; i < bySection.length; i += 1) {
 const prevMax = Math.max(...bySection[i - 1].nums);
 const curMin = Math.min(...bySection[i].nums);
 ok(curMin > prevMax, `разделы пересекаются: «${bySe
 }
 const all = bySection.flatMap((x) => x.nums);
 for (let n = all[0]; n <= all[all.length - 1]; n += 1

```

```

// 5. якоря
for (const m of html.matchAll(/href="#([^\"]+)/g)) {
 ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
}

if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

/* 6–7. глоссарий */
const demoKinds = new Set(MINI_DEMO_KINDS);
for (const term of GLOSSARY) {
 if (term.simulator) {
 ok(registryIds.includes(term.simulator), `глоссарий
 ok(
 chapterIds.has(term.simulator) || EXTRA_DEMOS.has
 `глоссарий «${term.id}»: simulator "${term.simula
 ые компилятора`
);
 }
 if (term.demo) ok(demoKinds.has(term.demo), `глоссарий
}

/* 8–9. реестр и синхронизация */
for (const key of Object.keys(PAGE_SYNC)) {
 const base = key.split("#")[0];
 ok(registryIds.includes(base), `PAGE_SYNC["${key}"]:
}
for (const id of registryIds) {
 if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
 FAILS.push(`реестр: визуализация "${id}" не являетс
 }
}
for (const chapter of chapters) {
 ok(registryIds.includes(chapter.id), `страница «${cha
}

/* 96. файлы, на которые ведут кнопки шапки, действител
{

```

```

* Каждый референсный Python-код из PAGE_SYNC выполняет
* (pyodide из node_modules) под тем же трейсером, что и
* проверяется:
* 1. код не падает и не обрывается по лимиту шагов;
* 2. трасса не пустая (иначе панель показала бы пустую);
* 3. каждая переменная, обещанная странице, реально
* иначе визуализация не может за ним следовать;
* 4. в снимках нет «мусорных» значений, которые уронят
*/
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
mkdirSync(OUT_DIR, { recursive: true });

writeFileSync(
 `${OUT_DIR}/contract-entry.ts`,
 `import { buildDebugRunner } from "../../src/data/debug";
import { PAGE_SYNC } from "../../src/data/vizSync";
import { chapters } from "../../src/data/content";
export const runnerOf = (src: string) => buildDebugRunner(src);
export const sync = Object.fromEntries(Object.entries(PAGE_SYNC));
export const declared = Object.fromEntries(Object.entries(declared));
export const chapterIds = chapters.map((c) => c.id);
`
);

const built = await esbuild.build({
 entryPoints: [`${OUT_DIR}/contract-entry.ts`],
 bundle: true,
 outfile: `${OUT_DIR}/contract-bundle.mjs`,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 sourcemap: false,
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
});
```

```

 .filter((a) => a.some(ident))
 .filter((a) => !a.some((x) => ident(x) && seen.has(x)))
 .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обеща
/*
 * Обещанная переменная не должна быть пустой на ВСЕХ
 * объявил и не наполнил – визуализации нечего показы
 * «защитый» результат вместо работы алгоритма. Именн
 * Косарайю: order был дан готовым списком, а comp та
 */
const isEmpty = (v) =>
 Array.isArray(v) ? v.length === 0
 : typeof v === "string" ? v.length === 0
 : typeof v === "object" && v !== null ? Object.keys
 : false;
const neverFilled = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .map((a) => a.find((x) => ident(x) && seen.has(x)))
 .filter(Boolean)
 .filter((name) => {
 let sawValue = false;
 let sawNonEmpty = false;
 for (const step of [...result.steps, { values: re
 const v = step.values?.[name];
 if (v === undefined || v === null) continue;
 sawValue = true;
 if (!isEmpty(v)) sawNonEmpty = true;
 }
 return sawValue && !sawNonEmpty;
 }));
if (neverFilled.length) issues.push(`объявлены, но ни

const stderr = printed.filter((p) => p.startsWith("[s
if (stderr.length) issues.push(`предупреждения: ${std

if (issues.length) problems++;
rows.push({ key, steps: result.steps.length, vars: se

```

```

const jsdomErrors = [];
virtualConsole.on("jsdomError", (error) => {
 jsdomErrors.push(error?.message || String(error));
 onError?.(error);
});
// console.error/warn приложения собираем отдельно —
const consoleErrors = [];
virtualConsole.on("error", (...args) => consoleErrors.push(...args));
virtualConsole.on("warn", () => {});

const dom = new JSDOM(
 `<!doctype html><html lang="ru"><head><meta charset="utf-8"><body><div id="root"></div></body></html>`,
 { url, pretendToBeVisual: true, runScripts: "outside-only" },
);
const { window } = dom;

window.ResizeObserver = window.ResizeObserver ?? OBSERVER_API.ResizeObserver;
window.IntersectionObserver = window.IntersectionObserver ?? OBSERVER_API.IntersectionObserver;
window.matchMedia =
 window.matchMedia ??
 ((media) => ({
 matches: false,
 media,
 onchange: null,
 addListener() {},
 removeListener() {},
 addEventListener() {},
 removeEventListener() {},
 dispatchEvent: () => false,
 }));
window.scrollTo = () => {};
window.HTMLElement.prototype.scrollIntoView = () => {};
window.HTMLElement.prototype.focus = window.HTMLElement.prototype.blur = () => {};
window.URL.createObjectURL = window.URL.createObjectURL = () => {};
window.URL.revokeObjectURL = window.URL.revokeObjectURL = () => {};
Object.defineProperty(window, "innerWidth", { value: 1000 });
Object.defineProperty(window, "innerHeight", { value: 1000 });

```

```
 "FileReader",
];
 for (const key of copy) {
 try {
 Object.defineProperty(globalThis, key, { value: w
 } catch {
 /* read-only в Node – не критично */
 }
 }
 globalThis.IS_REACT_ACT_ENVIRONMENT = true;

 return { dom, window, document: window.document, cons
}
```

---

ФАЙЛ 103/111: scripts/verify/live-matrix.test.mjs  
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМ

---

```
/**
 * Модульные проверки разбора «живых» данных компилятор
 *
 * npm run verify:unit
 *
 * liveMatrix() – то, благодаря чему визуализации матриц
 * суммы 2D, разреженная таблица) принимают данные ЛЮБОГО
 * пользователя вместо захардкоженной демо-сетки.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
writeFileSync(
 `${OUT_DIR}/unit-entry.ts`,
 `export { liveMatrix, pickLiveVar, clampIndex } from
`);
```

```

check("at() с отрицательным индексом → null", m([[1, 2]
check("список словарей с числовыми ключами", cells([{"
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1:
check('ключи-пары "{i, j}" после снапшота tuple', cells
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5
```

```
/* — liveMatrix: что матрицей НЕ является -----
check("плоский список чисел → null", m([1, 2, 3, 4, 5])
check("словарь по именам → null", m({ dist: 1, prev: 2
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);
check("null → null", m(null), null);
check("число → null", m(42), null);
check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m([["a", "b"], ["c"
```

```
/* — liveMatrix: особые значения -----
check('"inf" → +Infinity', cells([["inf", 1], [2, "-inf"
check('числа-строки "42" → 42', cells([["1", "2"], ["3"
check("булевы значения ячейкой не считаются", m([[true,
check("обрывок до 64×64", shape(Array.from({ length: 80
 4", numbers: true }));
check("hasNumbers=false невозможна для живой матрицы",
```

```
/* — pickLiveVar -----
check("первое найденное имя по приоритету", pickLiveVar
check("берёт следующее имя, если первого нет", pickLive
check("ничего не найдено → null", pickLiveVar({ x: 1 },
check("нет снимка → null", pickLiveVar(undefined, ["d"]
check("ключ со значением undefined всё равно считается"
```

```
/* — clampIndex -----
check("null остаётся null", clampIndex(null, 5), null);
check("размер 0 → null", clampIndex(2, 0), null);
check("обрезка сверху", clampIndex(99, 5), 4);
check("обрезка снизу", clampIndex(-7, 5), 0);
```

```

const f = divs.find((d) => d.className.includes("text
return (f?.textContent ?? "").replace(/\s+/g, " ").sl
}
function state(tag: string) {
 const p = panel();
 const ed = editor();
 const txt = (p?.textContent ?? "").replace(/\s+/g, "
 const step = txt.match(/шаг[^\s]*\s*\d+\s*\s*\s*\d+/i)
 process.stdout.write(
 ` [${tag.padEnd(9)}] редактор: ${ed ? "ДА" : "нет
 step ? step[0] : "-" } | футер: ${footer()}\n`
);
 process.stdout.write(` бейдж страницы: ${
 /встроенное демо/.test(document.body.textContent ??
}
function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubble
 }));
}
async function click(el: Element | null | undefined, label: string) {
 if (!el) throw new Error(`нет кнопки: ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { bubble
 await sleep(20);
 }));
}
function findButton(match: RegExp): HTMLButtonElement | undefined {
 return Array.from(document.querySelectorAll("button"))
 .filter(b => match.test([b.getAttribute("aria-label"), b.getAttribute("value")
) as HTMLButtonElement | undefined;
}

export async function probe() {
 const host = document.createElement("div");
 document.body.appendChild(host);

```



```

state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
}

process.stdout.write("\n=== 4. очищаем редактор полно
const ed4 = editor();
if (ed4) {
 typeInto(ed4, "");
 state("очистили");
 for (const t of [200, 800, 2000]) {
 await settle(t);
 state(`t+${t}`);
 }
} else {
 process.stdout.write(" редактора нет (листинг) – ж
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("
 await sleep(30);
 });
 state("после Esc");
}
}

```

---

ФАЙЛ 105/111: scripts/verify/README.md

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

---

# Проверки приложения (`npm run verify`)

Три уровня проверок, которые можно гонять локально и в чужих браузерах. Для локального запуска нужен: DOM даёт `jsdom`, Python – `pyodide` из `node\_modules`. Для проверки в браузере пользователя, только без

```bash

автозапуск инициализации настоящим Python не роняет |
`[object Object]`; нет кнопок без имени |
| В. Флойд: размерности (10) | 5×5, 10×10 (больше демо-
й список, 80×80 (обрезка), пустая матрица, чужое имя
| С. матричные страницы (6) | префиксные суммы 2D и раз
| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами
гравания гаснут и оживают обратно |
| Е. эталонный код (46) | прогон кода в CPython → финал
е показать мусор |
| F. редактор (6) | код из одних комментариев, синтаксис
ляет эталон и возвращает инициализацию; код не перете
| G. lite-режим (1) | `?lite=1` отдаёт текст без навига
| Н. доступность (2) | в панели подписаны все кнопки в
очника |

Фильтры: `HAR\_CHAPTERS=floyd,dijkstra`, `HAR\_SKIP=A,E`,

Как это заведено в приложение

- * `window.\_\_algv0PythonRuntime` — точка подмены интерпретатора
`PythonCompiler.tsx`. В браузере пользователя поле не
грузит Pyodide с CDN как раньше; харнес кладёт туда л
- * Состояние визуализаций передаётся через `emitVizState`
(`src/data/vizStepBus.ts`) — те же события, что шлёт
идёт по боевому пути, а не через внутренние API компо

ФАЙЛ 106/111: scripts/verify/run-browser-checks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 96 | РАЗМ

/**

* DOM-проверки приложения: jsdom + настоящий Pyodide и

*

* npm run verify:dom

*

* Фильтры:

```

// Vite-специфичные глобалы, которых нет вне dev-серв
define: {
  "process.env.NODE_ENV": '"development"',
  "import.meta.env.BASE_URL": '"/'',
  "import.meta.env.MODE": '"development"',
  "import.meta.env.DEV": "true",
  "import.meta.env.PROD": "false",
  "import.meta.env.SSR": "false",
},
loader: { ".css": "empty", ".png": "dataurl", ".jpg":
logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/b

process.on("unhandledRejection", (reason) => {
  jsdomErrors.push(`unhandledRejection: ${reason instan
});

log("прогоняю сценарии...\n");
const results = await mod.runSuite({ py, window, document
const sum = mod.summary();

writeFileSync(`${OUT_DIR}/report.json`, JSON.stringify(

const failed = results.filter((r) => !r.ok);
console.log("\n" + "=".repeat(96));
console.log("СВОДКА");
console.log("=".repeat(96));
for (const [group, g] of Object.entries(sum.byGroup)) {
  console.log(`  ${group.padEnd(26)} ok: ${String(g.ok)
}
console.log(`\n  ВСЕГО ${sum.total}:  прошло ${sum.pass
if (failed.length) {
  console.log("\nПРОВАЛЫ:");
  for (const f of failed) console.log(`  x [${f.group}]
}
if (jsdomErrors.length) {
  console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченны

```

```
-----
log("поднимаю jsdom...");
const { window, document } = createDom();
window.__algv0PythonRuntime = py;

await esbuild.build({
  entryPoints: ["scripts/verify/probe-compiler.tsx"],
  bundle: true,
  outfile: `${OUT_DIR}/probe-bundle.mjs`,
  format: "esm",
  platform: "browser",
  target: "es2022",
  jsx: "automatic",
  sourcemap: true,
  define: {
    "process.env.NODE_ENV": '"development"',
    "import.meta.env.BASE_URL": '"/'',
    "import.meta.env.DEV": "true",
    "import.meta.env.PROD": "false",
  },
  loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
  logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/probe-bundle.mjs`);
log("зондирую панель...");
await mod.probe();
process.exit(0);
-----
```

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

ФАЙЛ 108/111: [.github/workflows/deploy-pages.yml](#)
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

name: Deploy app to GitHub Pages

```
node-version: 24
```

```
cache: npm
```

- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Build for the repository subpath
env:
 VITE_BASE: /algv0/
run: npm run build
- name: Make GitHub Pages fallback
run: cp dist/index.html dist/404.html
- name: Upload Pages artifact
uses: actions/upload-pages-artifact@v5
with:
 path: ./dist

deploy:

```
name: Publish to GitHub Pages
```

```
needs: build
```

```
runs-on: ubuntu-latest
```

```
environment:
```

```
  name: github-pages
```

```
  url: ${ steps.deployment.outputs.page_url }
```

```
steps:
```

```
# Ранняя и понятная ошибка вместо невнятного «Fail
```

```
# когда в репозитории Pages не переключён на сбор
```

- name: Preflight – Pages собирается через GitHub

```
env:
```

```
  GH_TOKEN: ${ secrets.GITHUB_TOKEN }
```

```
run: |
```

```
  bt=$(gh api "repos/${ github.repository }/p
```

```
  echo "Pages build_type: ${bt}"
```

```
  if [ "${bt}" != "workflow" ]; then
```

```
    echo "::error title=Pages не настроен::Sett
```

```
    те этот запуск. См. комментарий в начале de
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot: ')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends
          # На Ubuntu политика ImageMagick иногда запрещает
          # разрешаем то, что нужно пайплайну (text -> pdf)
          for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
```

```
npm run export:guide
# снимки публикуются вместе с приложением: HTML
# в автономный файл прямо из /export/viz-shots
mkdir -p public/export/viz-shots
cp tmp/viz-shots/*.png tmp/viz-shots/manifest
```

- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full\_code\_all\_pages` | "
 echo "| \`public/export/full\_code\_all\_pages` | "
 echo "| \`public/export/guide\_lite.pdf\` | "
 echo "| Снимков визуализаций | \$(ls tmp/viz-shots | wc -l) | "
 echo "| PNG-страниц | \$(ls tmp/export-pages | wc -l) | "
 } >> "\$GITHUB_STEP_SUMMARY"
- name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 if-no-files-found: error
 retention-days: 30
- name: Upload artifacts (PNG-страницы)
uses: actions/upload-artifact@v7
with:
 name: full-code-pages-png

```
-   "**"
  paths-ignore:
    - "public/export/**"
    - "**/*.md"
  pull_request:
  workflow_dispatch:

concurrency:
  group: verify-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  verify:
    name: unit + эталонный код + DOM
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      # puppeteer здесь не нужен (он для растеризации Р
      # тянет Chromium – пропускаем, чтобы установка не
      - name: Зависимости
        run: npm ci
        env:
          PUPPETEER_SKIP_DOWNLOAD: 1
          PUPPETEER_SKIP_CHROMIUM_DOWNLOAD: 1

      - name: Типы и сборка
        run: |
          npx tsc --noEmit -p tsconfig.json
          npm run build
```


1. Объект главы

```
```ts
import { Chapter } from "../../types";

export const myChapters: Chapter[] = [
 {
 id: "dijkstra", // = ключ VIZ_REGISTRY и
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html", // всегда "html": страница
 category: "Графы. Пути", // группа в навигации
 content: `...HTML ниже...`,
 },
];
```
```

Правила:

- `id` – kebab-case, уникальный, совпадает с ключом `VIZ\_REGISTRY`, `id` подставляется автоматически, вручную `div#slot-\*` вставляются только такие заглушки остались в старых главах и считаются мусором.
- `title` начинается с номера билета: сортировка оглавления по номеру цифре заголовка. Одна страница может закрывать несколько билетов (например `mst` → 18, 19, 20) – тогда номер в заголовке один, билет объявляется в `chapterTopics` (`src/data/content.ts`).
- `content` – шаблонная строка. Внутри **никаких** `<div>` и `<div>` экранирования: это сломает сборку.

2. Каркас страницы

```
```html
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <!-- шапка: бейдж билета + заголовков -->
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
```html
<p class="text-slate-300 text-sm mb-4">
  Текст с <b>акцентом</b> и инлайн-кодом
  <span class="font-mono text-emerald-300">dist[v]</span>
</p>
```
```

Инлайн-«пилюля» для имени переменной/константы:

```
```html
<span class="bg-slate-950 px-2 py-0.5 rounded text-xs f
```
```

### ### 3.3 Сетка ключевых чисел (3 ячейки)

```
```html
<div class="grid grid-cols-1 sm:grid-cols-3 gap-3 my-6">
  <div class="bg-slate-900 border border-slate-700 rounded">
    <p class="text-[10px] uppercase tracking-wider text">
    <p class="font-mono text-emerald-300 text-sm">0{(V+H
  </div>
  <!-- 2-я и 3-я ячейки — тот же markup -->
</div>
```
```

### ### 3.4 Сетка аналогий — **\*\*обязательна\*\*** на каждой странице

Слева — неформальная аналогия, справа — техническая аль

```
```html
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border border">
    <div class="absolute -top-3 left-4 bg-slate-700 text
      ald-500 shadow-md">
      Аналогия 1: Позитивное планирование
    </div>
    <p class="text-slate-300 text-sm mb-4">...</p>
  </div>
  </div>
```
```

### ### 3.6 Спойлер ``<details>`` – **обязателен** для кода,

```
```html
<details class="bg-slate-900/40 rounded-lg border border-
  <summary class="p-4 font-bold text-slate-400 outline-l
    e-700/50">
    ▶ Код: Дейкстра на куче (Python)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <pre class="bg-slate-950 p-4 rounded overflow-x-aut
...</code></pre>
    <p class="text-slate-300 text-sm">Комментарий после
  </div>
</details>
```
```

- Вложенные списки внутри спойлера: ``<ul class="list-di`
- Подзаголовок внутри блока: ``<h4 class="text-lg font-b`
- или ``<p class="text-lg font-bold text-white mt-8 mb-3`

### ### 3.7 Ловушки / частые ошибки

```
```html
<div class="bg-rose-950/30 p-5 rounded-lg border border
  <p class="text-sm font-bold text-rose-300 mb-3">⚠ Гд
  <ul class="list-disc pl-5 space-y-2 text-sm text-slat
    <li><b>Отрицательные веса.</b> Дейкстра их не перев
    <li>Переполнение: <span class="font-mono text-emera
  </ul>
</div>
```
```

---

## ## 4. Порядок разделов на странице

1. Шапка: бейдж билета + ``h2``.
2. «Одной фразой» / «Суть алгоритма» + центрированное о

-----  
компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только `h2/h3/h4`; списки – `ul/ol/li`; т.к. код – `pre`; определения – обычный `p` (см. 3.1);
- **\*\*смысл не должен держаться на эмодзи\*\***: в PDF используется эмодзи-глифы – пиктограммы вырезаются, остаётся «    Операции на отрезках» доедет как «Операции на отрезках», а вот «    значит да /    значит нет» потеряет смысл, пиктограммы не нужны;
- спойлеры в PDF раскрываются полностью – не прячьте ту часть, минимум, но и не дублируйте его в основном тексте;
- картинки страниц берутся из `CHAPTER\_IMAGES` (`script` снимки демонстраций – из `tmp/viz-shots/\*.png` (их перенести в `public` `npm run export:shots` в CI); локально без браузера в `public` описание вкладок и переменных из `PAGE\_SYNC`.

Цель по объёму: 40–60 страниц, жёсткий предел – 100.